



Programare orientată pe obiecte

- suport de curs -

**Anca Dobrovăț
Andrei Păun**

**An universitar 2025 – 2026
Semestrul II
Seriile 13, 14 și 15**

Curs 1

Generalități despre curs

1. Curs – luni 11-14 (seria 15), marti 14-17 (seria 14) si vineri 9-12 (seria 13)
2. Laborator - in fiecare saptamana; pe semigrupe
3. Seminar - o data la 2 saptamani (in conexiune cu laboratorul / vom reveni cu detalii)
4. Prezenta la curs nu e obligatorie!

Laborator – OBLIGATORIU

Să ne cunoaștem

Cine predă?

Curs: Anca Dobrovăț (seriile 13 și 15), Andrei Paun (seria 14)

- anca.dobrovat@fmi.unibuc.ro;
apaun@fmi.unibuc.ro

Laboratoare: (se vor completa informații)

Agenda cursului

1. Regulamente UB si FMI
2. Utilitatea cursului de Programare Orientata pe Obiecte
3. Prezentarea disciplinei
4. Primul curs

Agenda cursului

1. Regulamente UB si FMI

2. Utilitatea cursului de Programare Orientata pe Obiecte

3. Prezentarea disciplinei

4. Primul curs

1. Regulamente UB si FMI

Lucruri bine de stiut de studenti:

- <https://fmi.unibuc.ro/regulamente/>
- regulament privind activitatea studenților la UB: https://unibuc.ro/wp-content/uploads/2025/09/RAPS_-2025.pdf
- http://fmi.unibuc.ro/ro/pdf/2015/consiliu/Regulament_etica_FMI.pdf

Se consideră **incident minor** cazul în care un student/ o studentă:
a. preia codul sursă/ rezolvarea unei teme de la un coleg/ o colegă și pretinde că este rezultatul efortului propriu;

Se consideră **incident major** cazul în care un student/ o studentă:
a. copiază la examene de orice tip;

2 incidente minore = un incident major = exmatriculare

Agenda cursului

1. Regulamente UB si FMI

2. Utilitatea cursului de Programare Orientata pe Obiecte

3. Prezentarea disciplinei

4. Primul curs

2. Utilitatea cursului de Programare Orientata pe Obiecte

PYPL PopularitY of Programming Language

Captura din: <http://pypl.github.io/PYPL>

Worldwide, Feb 2024 :				Worldwide, Feb 2025 :					Worldwide, Feb 2026 :				
Rank	Change	Language	Share	Rank	Change	Language	Share	1-ye	Rank	Change	Language	Share	1-year trend
1		Python	28.11 %	1		Python	29.85 %		1		Python	29.97 %	-0.2 %
2		Java	15.52 %	2		Java	15.15 %		2	↑↑	C/C++	15.45 %	+8.3 %
3		JavaScript	8.57 %	3		JavaScript	7.92 %		3	↓	Java	10.21 %	-4.9 %
4	↑	C/C++	6.92 %	4		C/C++	7.19 %		4	↑↑	R	6.89 %	+2.4 %
5	↓	C#	6.73 %	5		C#	6.13 %		5	↓↓	JavaScript	5.09 %	-2.9 %
6	↑	R	4.75 %	6		R	4.55 %		6	↑↑↑↑↑	Swift	4.05 %	+1.6 %
7	↓	PHP	4.57 %	7		PHP	3.72 %		7	↑	Rust	3.24 %	+0.1 %
8		TypeScript	2.78 %	8	↑↑	Rust	3.07 %		8	↓↓↓	C#	3.19 %	-3.0 %
9		Swift	2.75 %	9	↑↑	Objective-C	2.86 %		9	↓↓	PHP	3.19 %	-0.5 %
10		Objective-C	2.37 %	10	↓↓	TypeScript	2.74 %		10	↑↑↑↑↑	Ada	2.93 %	+1.6 %
11		Rust	2.23 %	11	↓↓	Swift	2.46 %						
12		Go	2.04 %	12		Go	2.07 %						

Majoritatea pot fi considerate limbaje OO.

Limbaje destul de cunoscute care nu sunt OO sunt Go, Julia și Rust

2. Utilitatea cursului de Programare Orientata pe Obiecte

Paradigme de programare → Stil fundamental de a programa

Dictează:

- **Cum se reprezintă datele problemei** (variabile, funcții, obiecte, fapte, constrângeri etc.)
- **Cum se prelucrează reprezentarea** (atribuiri, evaluări, fire de execuție, continuări, fluxuri etc.)
- **Favorizează un set de concepte si tehnici de programare**
- **Influențează felul în care sunt gândiți algoritmi de rezolvare a problemelor**
- **Limbaje – în general multiparadigmă (ex: Python – imperativ, funcțional, orientat pe obiecte)**

Agenda cursului

1. Regulamente UB si FMI
2. Utilitatea cursului de Programare Orientata pe Obiecte
3. Prezentarea disciplinei
 - 3.1 Obiectivele disciplinei
 - 3.2 Programa cursului
 - 3.3 Bibliografie
 - 3.4 Regulament de notare si evaluare
4. Primul curs

3. Prezentarea disciplinei

3.1 Obiectivele disciplinei

- Curs de programare OO
- Oferă o **baza** de pornire pentru alte cursuri
- **Obiectivul general al disciplinei:**
Formarea unei imagini generale, preliminară, despre programarea orientată pe obiecte (POO).
- **Obiective specifice:**
 - 1. Înțelegerea fundamentelor paradigmei programării orientate pe obiecte;
 - 2. Înțelegerea conceptelor de clasă, interfață, moștenire, polimorfism;
 - 3. Familiarizarea cu șabloanele de proiectare;
 - 4. Dezvoltarea de aplicații de complexitate medie respectând principiile de dezvoltare ale POO;
 - 5. Deprinderea cu noile facilități oferite de limbajul C++.

3. Prezentarea disciplinei

3.2 Programa cursului

1. Prezentarea disciplinei.

- 1.1 Principiile programării orientate pe obiecte.
- 1.2. Caracteristici.
- 1.3. Programă cursului, obiective, desfășurare, examinare, bibliografie.

2. Recapitulare limbaj C (procedural) și introducerea în programarea orientată pe obiecte.

- 2.1 Funcții, transferul parametrilor, pointeri.
- 2.2 Deosebiri între C și C++.
- 2.3 Supradefinirea funcțiilor, Operații de intrare/ieșire, Tipul referință, Funcții în structuri.

3. Prezentarea disciplinei

3.2 Programa cursului

3. Proiectarea ascendentă a claselor. Incapsularea datelor în C++.

3.1 Conceptele de clasă și obiect. Structura unei clase.

3.2 Constructorii și destructorul unei clase.

3.3 Metode de acces la membrii unei clase, pointerul this. Modificatori de acces în C++.

3.4 Declararea și implementarea metodelor în clasă și în afara clasei.

4. Supraîncărcarea funcțiilor și operatorilor în C++.

4.1 Clase și funcții friend.

4.2 Supraîncărcarea funcțiilor.

4.3 Supraîncărcarea operatorilor cu funcții friend.

4.4 Supraîncărcarea operatorilor cu funcții membru.

4.5 Observații.

3. Prezentarea disciplinei

3.2 Programa cursului

5. Conversia datelor în C++.

5.1 Conversii între diferite tipuri de obiecte (operatorul cast, operatorul= și constructor de copiere).

5.2 Membrii constanți și statici ai unei clase în C++.

5.3 Modificatorul const, obiecte constante, pointeri constanți la obiecte și pointeri la obiecte constante.

6. Tratarea excepțiilor în C++.

7. Proiectarea descendentă a claselor. Mostenirea în C++.

7.1 Controlul accesului la clasa de bază.

7.2 Constructori, destructori și moștenire.

7.3 Redefinirea membrilor unei clase de bază într-o clasă derivată.

7.4. Declarații de acces.

3. Prezentarea disciplinei

3.2 Programa cursului

8. Funcții virtuale în C++.

8.1 Parametrizarea metodelor (polimorfism la executie).

8.2 Funcții virtuale în C++. Clase abstracte.

8.3 Destructori virtuali.

9. Mostenirea multiplă și virtuală în C++

9.1 Moștenirea din clase de bază multiple.

9.2 Exemple, observații.

10. Controlul tipului în timpul rulării programului în C++.

10.1 Mecanisme de tip RTTI (Run Time Type Identification).

10.2 Moștenire multiplă și identificatori de tip (dynamic_cast, typeid).

3. Prezentarea disciplinei

3.2 Programa cursului

11. Parametrizarea datelor. Șabloane în C++. Clase generice

11.1 Funcții și clase Template: Definiții, Exemple, Implementare.

11.2 Clase Template derivate.

11.3 Specializare.

12. Biblioteca Standard Template Library - STL

12.1 Containere, iteratori și algoritmi.

12.2 Clasele string, set, map / multimap, list, vector, etc.

3. Prezentarea disciplinei

3.2 Programa cursului

13. Șabloane de proiectare (Design Pattern)

13.1 Definiție și clasificare.

13.2 Exemple de șabloane de proiectare (Singleton, Abstract Object Factory).

14. Recapitulare, concluzii, tratarea subiectelor de examen.

3. Prezentarea disciplinei

3.3 Bibliografie

1. Bruce Eckel. Thinking in C++ (2nd edition). Volume 1: Introduction to Standard C++. Prentice Hall, 2000.
2. Bruce Eckel, Chuck Allison. Thinking in C++ (2nd edition). Volume 2: Practical Programming. Prentice Hall, 2003.
3. Bjarne Stroustrup: The C++ Programming Language, Addison-Wesley, 3rd edition, 1997.
4. Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides: Design Patterns. Elements of Reusable Object-Oriented Software. Addison-Wesley, 1995.

3. Prezentarea disciplinei

3.4 Regulament de notare și evaluare

Curs: 3 ore pe săptămână

Laborator: 2 ore pe săptămână.

Seminar: 1 ora pe săptămână => 2 ore, la fiecare 2 săptămâni.

Disciplina: semestrul II, durata de desfășurare de 14 săptămâni.

Materia este de nivel elementar - mediu și se bazează pe cunoștințele de C++ anterior dobândite.

Limbajul de programare folosit la curs și la laborator este **C++**.

3. Prezentarea disciplinei

3.4 Regulament de notare si evaluare

Programa disciplinei este împărțită în 14 cursuri.

Evaluarea studenților se face cumulativ prin:

- 3 lucrări practice (proiecte) – nr de proiecte depinde de tutorii de laborator dar se trece prin mai multe concepte
- Test practic (colocviu)
- Test scris

Toate cele 3 probe de evaluare sunt obligatorii.

Condiții de promovare - minim **nota 5 la fiecare** parte de evaluare enunțată - mai sus se păstrează la oricare din eventualele examene restante ulterioare aferente acestui curs.

3. Prezentarea disciplinei

3.4 Regulament de notare si evaluare

Nota laborator = medie aritmetica a celor 3 note obtinute pe proiecte (sau nota pe proiectul “mare”).

Pentru evidentierea unor lucrari practice, tutorele de laborator poate acorda un bonus de **pana la 2 puncte** la nota pe proiecte astfel calculata.

Studentii care **nu obtin cel putin nota 5 pentru activitatea pe proiecte nu pot intra in examen** si vor trebui sa refaca aceasta activitate, inainte de prezentarea la restanta.

3. Prezentarea disciplinei

3.4 Regulament de notare si evaluare

Testul practic (Colocviu) - in saptamana 14

- Consta dintr-un program care trebuie realizat individual intr-un timp limitat (90 de minute – si va avea un nivel mediu.
- Notare: de la 1 la 10, nu neaparat intregi (pot exista pana la 3 puncte bonus).

Testul practic este obligatoriu.

Studentii care **nu obtin cel putin nota 5 la testul practic de laborator nu pot intra in examen** si vor trebui sa il dea din nou, inainte de prezentarea la restanta.

3. Prezentarea disciplinei

3.4 Regulament de notare si evaluare

Testul scris:

Propunere: **11 iunie 2026** la ora 9:00

Studentii nu pot lua examenul decat daca obtin cel putin nota 5 la testul scris.

3. Prezentarea disciplinei

3.4 Regulament de notare si evaluare

Testul scris:

Consta din:

- 30 intrebari de teorie, grila (unul sau mai multe raspunsuri corecte, adevarat / fals, etc.), fiecare valorand 0,1p.
- 12 subiecte practice (să se decida dacă un algoritm dat este corect (caz în care să menționeze rezultatul), sau este incorect (caz în care trebuie să se recunoască și să se rectifice eroarea)), fiecare valorand 0,5p.
- 1 punct din oficiu.

Studentii nu pot lua examenul decat daca obtin cel putin nota 5 la testul scris.

3. Prezentarea disciplinei

3.4 Regulament de notare si evaluare

Examenul se considera luat daca studentul respectiv a obtinut **cel putin nota 5 la fiecare** dintre cele 3 evaluari (activitatea practica din timpul semestrului, testul practic de laborator si testul scris).

In aceasta situatie, nota finala a fiecarui student se calculeaza ca medie ponderata intre notele obtinute la cele 3 evaluari, ponderile cu care cele 3 note intra in medie fiind:

- 25% - nota pe lucrarile practice (proiecte)
- 25% - nota la testul practic
- 50% - nota la testul scris

3. Prezentarea disciplinei

Modificari

- Laborator: notare mai “clara”
- Seminar: 0.5 bonus la nota de la examenul scris pentru max. 25% din studenti
- Prezenta la curs: 0.5 bonus la nota de la examenul scris pentru primii 25% dintre studenti KAHOOT
- - vom reveni cu detalii saptamana viitoare!!
- bonus dupa ce se promoveaza examenul scris

3. Prezentarea disciplinei

Kahoot

- Se va defini un nume unic de forma 131popescu (unde popescu este numele de familie si 131 este grupa)
- Daca sunt mai multi studenti cu acelasi nume in grupa respectiva (adaugati si initiala / initialele prenumelui)
- 131popescup si 131popescupr

Agenda cursului

1. Regulamente UB si FMI
2. Utilitatea cursului de Programare Orientata pe Obiecte
3. Prezentarea disciplinei
4. Primul curs
 - 4.1 Completări aduse de limbajul C++ față de limbajul C
 - 4.2 Principiile programarii orientate pe obiecte

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

- Bjarne Stroustrup în 1979 la Bell Laboratories in Murray Hill, New Jersey
- 7 revizii: 1998 ANSI+ISO, 2003 (corrigendum), 2011 (C++11/0x), 2014, 2017 (C++ 17/1z), 2020, 2023
- Următoarea plănuită în 2026 (C++2c)
- Versiunea 1998: Standard C++, C++98
- C++ modern: $\geq$ C++11

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

- C++98: a definit standardul inițial, toate chestiunile de limbaj, STL
- C++03: bugfix o unică chestie nouă: value initialization
- C++11: initializer lists, rvalue references, moving constructors, lambda functions, final, nullptr literal, sincronizare cu C99, etc.
- C++14: generic lambdas, binary literals, auto, variable template, etc.

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

C++17:

- If constexpr()
- Inline variables
- Nested namespace definitions
- Class template argument deduction
- Hexadecimal literals
- etc

typename is permitted for template template parameter declarations (e.g.,
template<**template**<**typename**> **typename** X> **struct** ...)

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

- C++20:
 - Concepte: sintaxă mai "simplă" și erori mai "clare" la templates
 - Module
 - Spaceship operator `<=>` și operator `==()` = default
- C++23: `<stacktrace>`, `<expected>`

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

- `<iostream>` (fără `.h`)
- `using namespace std;`
- `cout, cin` (fără `&`)
- `//` comentarii pe o linie
- declarare variabile
- Tipul de date `bool`
- se definesc `true` și `false` (1 si 0);
- C99 nu îl definește ca `bool` ci ca `_Bool` (fără `true/false`)
- `<stdbool.h>` pentru compatibilitate

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Intrări și ieșiri

Obiectele **cin** și **cout**, în plus față de funcțiile `scanf` și `printf` din limbajul C.

Nu necesită specificarea formatelor.

// operator este o functie care are ca nume un simbol (sau mai multe simboluri)

```
int x,y,z;
```

```
cin >>x; // operator>>(cin,x) : intoarce fluxul (prin referinta ) cin din care s-a extras data x
```

```
cin>>y>>z; // operator>>(operator >>(cin,y), z)
```

```
cout<<x; // operator<<(cout,x) : intoarce fluxul cout (prin referinta) in care s-a inserat x
```

```
cout<<y<<z; // operator<<(operator<<(cout,y),z)
```

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

```
#include <iostream>
using namespace std;
int main()
{
    int i;
    cout << "This is output.\n"; // this is a single line comment
    /* you can still use C style comments */
    // input a number using >>
    cout << "Enter a number: ";
    cin >> i;
    // now, output a number using <<
    cout << i << " squared is " << i*i << "\n";
    return 0;
}
```

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

```
#include <iostream>
using namespace std;
int main( )
{
    float f;
    char str[80];
    double d;
    cout << "Enter two floating point numbers: ";
    cin >> f >> d;
    cout << "Enter a string: ";
    cin >> str;
    cout << f << " " << d << " " << str;
    return 0;
}
```

citirea string-urilor se face până la primul caracter alb
se poate face afișare folosind toate caracterele speciale \n, \t, etc.

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Variabilele locale

```
/* Incorrect in C89. OK in C++. */  
int f()  
{  
    int i;  
    i = 10;  
    int j; /* aici problema de compilare in C */  
    j = i*2;  
    return j;  
}
```

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Supraîncărcarea funcțiilor (un caz de *Polimorfism la compilare*)

Utilizarea mai multor funcții care au același nume

Identificarea se face prin numărul de parametri și tipul lor. **Tipul de întoarcere nu e suficient pentru a face diferența**

Simplicitate/corectitudine de cod

La compilare: transformat în nume unic prin name mangling

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Compiler de C

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void f(int x){printf("1");}
5  void f(int x, int y){printf("2");}
6
7  int main()
8  {
9      f(1);
10     f(2,3);
11     return 0;
12 }
```

Logs & others		
Code::Blocks Search results Cccc Build log Build messages		
File	L...	Message
=== Build: Debug in testC (compiler: GNU GCC Compiler) ===		
C:\Proie...	5	error: conflicting types for 'f'
C:\Proie...	4	note: previous definition of 'f' was here

Compiler de C++

```
main.cpp x
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void f(int x){printf("1");}
5  void f(int x, int y){printf("2");}
6
7  int main()
8  {
9      f(1);
10     f(2,3);
11     return 0;
12 }
```

```
12
Process returned 0 (0x0)
```

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

```
#include <iostream>
using namespace std;

// abs is overloaded three ways
int abs(int i);
double abs(double d);
long abs(long l);

int main()
{
    cout << abs(-10) << "\n";
    cout << abs(-11.0) << "\n";
    cout << abs(-9L) << "\n";
    return 0;
}

int abs(int i)
{
    cout << "Using integer abs()\n";
    return i < 0 ? -i : i;
}
```

```
double abs(double d)
{
    cout << "Using double abs()\n";
    return d < 0.0 ? -d : d;
}

long abs(long l)
{
    cout << "Using long abs()\n";
    return l < 0 ? -l : l;
}
```

```
Using integer abs()
10
Using double abs()
11
Using long abs()
9
```


4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Supraîncărcarea funcțiilor

tipuri diferite pentru parametrul i

```
double myfunc(double i) { return i; }
```

```
int myfunc(int i) { return i; }
```

```
int main() {  
    cout << myfunc(10);  
    cout << myfunc(5.4);  
    return 0;  
}
```

numar diferit de parametri

```
int myfunc(int i) {return i;}
```

```
int myfunc(int i, int j) {return i*j;}
```

```
int main()  
{  
    cout << myfunc(10) ;  
    cout << myfunc(4, 5);  
    return 0;  
}
```

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Supraîncărcarea funcțiilor

Ambiguitati pentru polimorfism de functii

- erori la compilare – daca diferenta este doar in tipul de date intors sau sunt tipuri care par sa fie diferite
- majoritatea datorita conversiilor implicite

int myfunc(**int** i); // Error: differing return types are
float myfunc(**int** i); // insufficient when overloading.

void f(**int** *p);
void f(**int** p[]); // error, *p is same as p[]

void f(**int** x);
void f(**int**& x);

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Supraîncărcarea funcțiilor

Ambiguitati pentru polimorfism de functii

Obs.

```
int myfunc(double d); // ...  
cout << myfunc('c'); // not an error, conversion applied
```

Dar...

```
float myfunc(float i){ return i;}  
double myfunc(double i){ return -i;}  
  
int main(){  
    cout << myfunc(10.1) << " "; // unambiguous, calls myfunc(double)  
    cout << myfunc(10); // ambiguous  
    return 0; }
```

- problema nu e de definire a functiilor myfunc,
- problema apare la apelul functiilor

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Supraîncărcarea funcțiilor

Ambiguitati pentru polimorfism de functii

ambiguitate intre char si unsigned char

```
char myfunc(unsigned char ch){ return ch-1;}  
char myfunc(char ch){return ch+1;}
```

```
int main(){  
    cout << myfunc('c'); // this calls myfunc(char)  
    cout << myfunc(88) << " "; // ambiguous  
    return 0;  
}
```

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Funcții cu valori implicite

Într-o funcție se pot declara valori implicite pentru unul sau mai mulți parametri.

La apel se poate omite specificarea valorii pentru acei parametri formali care au declarate valori implicite.

Argumentele cu valori implicite trebuie să fie amplasate la sfârșitul listei.

```
void f (int a, int b = 12){ cout<<a<<" - "<<b<<endl;}
```

```
int main(){  
    f(1);  
    f(1,20);  
  
    return 0;  
}
```

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Funcții cu valori implicite

Valorile implicite se specifică o singură dată în definiție (de obicei în prototip).

```
#include <iostream>
using namespace std;
```

```
void f (int a, int b = 12); // prototip cu mentionarea valorii implicite pentru b
```

```
int main()
{
    f(1);
    f(1,20);
    return 0;
}
```

```
void f (int a, int b) { cout<<a<<" - "<<b<<endl; }
```

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Funcții cu valori implicite

Atentie la ambiguitate!

```
int myfunc(int i) { return i; }
```

```
int myfunc(int i, int j = 0) { return i*j; }
```

```
int main()
{
    cout << myfunc(4, 5) << " "; // unambiguous
    cout << myfunc(10); // ambiguous
    return 0;
}
```

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Cand tipul returnat de o functie nu este declarat explicit, i se atribuie automat int.

Tipul trebuie cunoscut inainte de apel.

```
f (double x)
{
    return x;
}
```

Prototipul unei functii: permite declararea in afara si a numarului de parametri / tipul lor:

```
void f(int); // antet / prototip
int main() { cout<< f(50); }
void f( int x)
{
    // corp functie;
}
```


4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Pointerii in C/C++

- O variabilă care ține o adresă din memorie
- Are un tip, compilatorul știe tipul de date către care se pointează
- Operațiile aritmetice țin cont de tipul de date din memorie
- `Pointer ++ == pointer+sizeof(tip)`
- Definiție: `tip *nume_pointer;`
 - Merge și `tip* nume_pointer;`

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Pointerii in C/C++

Operatori pe pointeri

- *, &, schimbare de tip
- *== “la adresa”
- &==“adresa lui”

```
int i=7, *j;
```

```
j=&i;
```

```
*j=9;
```

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Pointerii in C/C++

Compiler de C

```
main.c x
#include <stdio.h>
#include <stdlib.h>

int main()
{
    float x = 12.34, y;
    int *p;
    p = &x;
    y = *p;
    printf("%d\n", y);
    return 0;
}
```

"C:\Proiecte Codeblocks\testC" x

536870912

Process returned 0 (0x0)

Compiler de C++

```
main.cpp x
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      float x = 12.34, y;
8      int *p;
9      p = &x;
10     y = *p;
11     cout<<y;
12     return 0;
13 }
```

blocks x Search results x Cccc x Build log x Build message

L... Message

In function 'int main()':

9 error: cannot convert 'float*' to 'int*' in assignment

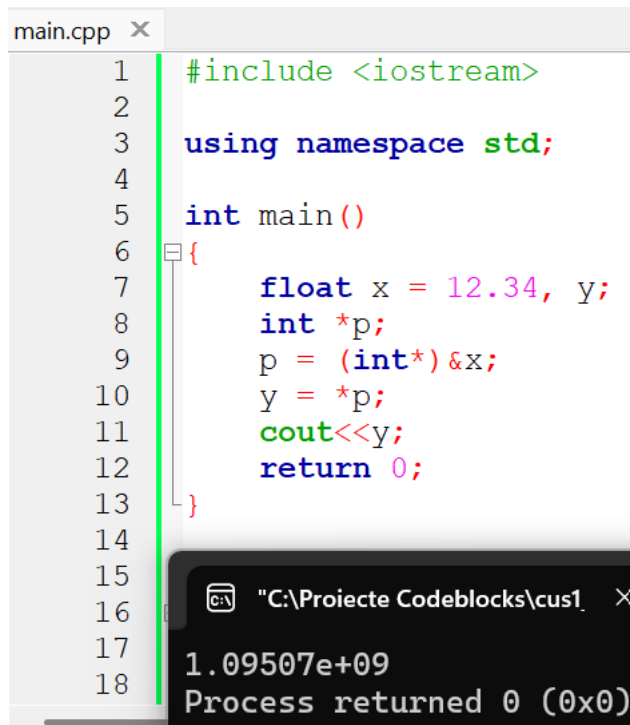
- atentie la corectitudine conversiilor!

în C++ conversiile trebuie făcute cu schimbarea de tip

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Pointerii in C/C++



```
main.cpp x
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      float x = 12.34, y;
8      int *p;
9      p = (int*) &x;
10     y = *p;
11     cout<<y;
12     return 0;
13 }
14
15
16
17
18
```

"C:\Proiecte Codeblocks\cus1" x

1.09507e+09
Process returned 0 (0x0)

- Schimbarea de tip nu e controlată de compilator
- În C++ conversiile trebuie făcute cu schimbarea de tip
- Atentie la corectitudinea conversiilor

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Pointerii in C/C++

Aritmetica pe pointeri

- `pointer++`; `pointer--`;
- `pointer+7`;
- `pointer-4`;
- `pointer1-pointer2`; întoarce un întreg
- comparații: `<`, `>`, `==`, etc.

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Pointerii in C/C++

pointeri și array-uri

- numele array-ului este pointer
- `lista[5]==*(lista+5)`
- array de pointeri, numele listei este un pointer către pointeri (dublă indirectare)
- `int **p;` (dublă indirectare)

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Pointerii in C/C++

- în C++ tipurile pointerilor trebuie să fie la fel

```
int *p;
```

```
float *q;
```

```
p=q; //eroare
```

se poate face cu schimbarea de tip (type casting) dar ieșim din verificările automate făcute de C++

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Atentie cand lucram cu pointeri!

Ce problema apare in codul de mai jos?

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      for(int i = 0; i<5; i++)
8      {
9          int* p=new int();
10         (*p)++;
11         cout<<p<<" "<<*p<<endl;
12     }
13     return 0;
14 }
```

```
0xdb1780 1
0xdb17a0 1
0xdb1900 1
0xdb1920 1
0xdb1940 1
```

“Memory leak” – pointerul nu se dezaloca

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

O rezolvare in C++ prin utilizare de smart pointers (detalii mai tarziu)

```
1  #include <iostream>
2  #include <memory>
3
4  using namespace std;
5
6  int main()
7  {
8      for(int i = 0; i<5; i++)
9      {
10         shared_ptr<int> p(new int());
11         (*p)++;
12         cout<<p<<" "<<*p<<endl;
13     }
14     return 0;
15 }
16
```

```
0xec1780 1
0xec1780 1
0xec1780 1
0xec1780 1
0xec1780 1
```

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Alocare dinamica în C

Funcția malloc

Prototipul funcției:

void * malloc(int dimensiune);

unde:

- ***dimensiune*** = numărul de octeți ceruți a se alocă

1. Dacă există suficient spațiu liber în HEAP atunci un bloc de memorie continuu de dimensiunea specificată va fi marcat ca ocupat, iar funcția **malloc va returna un pointer ce conține adresa de început a aceluiași bloc**. Dacă nu există suficient spațiu liber funcția **malloc** întoarce NULL.

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Alocare dinamica în C Funcția calloc

Prototipul funcției:

void * calloc(int numar, int dimensiune);

unde:

- *numar* reprezintă numărul de blocuri/elemente a se alocă
 - *dimensiune* = numărul de octeți ceruți pentru fiecare bloc
1. Dacă există suficient spațiu liber în HEAP atunci un bloc de memorie continuu de dimensiunea specificată va fi marcat ca ocupat, iar funcția **calloc va returna un pointer ce conține adresa de început a aceluiași bloc**. Dacă nu există suficient spațiu liber funcția *calloc* întoarce NULL.
 2. Diferența față de *malloc*: funcția *calloc* inițializează toate blocurile cu 0 (vector de frecvențe).

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Alocare dinamica în C

Funcția realloc

Prototipul funcției:

void * realloc(void *p, int dimensiune);

unde:

- ***p*** reprezintă un pointer (începutul unui bloc de memorie pe care vreau să îl redimensionez (de obicei avem nevoie de mai multă memorie))
 - ***dimensiune*** = numărul de octeți ceruți pentru alocare
1. Dacă există suficient spațiu liber în HEAP atunci un bloc de memorie continuu de dimensiunea specificată va fi marcat ca ocupat, iar funcția **realloc va returna un pointer ce conține adresa de început a aceluși bloc. Tot conținutul blocului de memorie inițial se copiază.** Dacă nu există suficient spațiu liber ***realloc*** întoarce NULL.

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Alocare dinamica in C

Funcția free

Prototipul funcției:

void free(void *p);

unde:

- *p* reprezinta un pointer (începutul unui bloc de memorie pe care vrem să-l eliberăm)
1. Funcția **free** eliberează zona de memoria alocată dinamic a cărei adresă de început este dată de *p*. Zona de memorie dezalocată este marcată ca fiind disponibilă pentru o nouă alocare.
 2. Un bloc de memorie nu trebuie eliberat de mai multe ori.

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Alocare dinamica in C++ (se poate utiliza si alocarea din C)

```
int *pi;  
pi=new int;
```

delete pi; // elibereaza zona adresata de pi -o considera neocupata

pi=new int(2);// alocă zona și initializează zona cu valoarea 2

pi=new int[2]; // alocă un vector de 2 elemente de tip întreg

delete [] pi; //elibereaza întreg vectorul

//-pentru new se foloseste delete

//- pentru new [] se foloseste delete []

la eroare se “aruncă” excepția bad_alloc din <new>

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

```
#include <iostream>
#include <new>
using namespace std;

int main()
{
    int *p;
    try {
        p = new int;
// allocate space for an int

    } catch (bad_alloc xa) {
        cout << "Allocation Failure\n";
        return 1;
    }
    *p = 100;
    cout << "At " << p << " ";
    cout << "is the value " << *p << "\n";
    delete p;
    return 0;
}
```

```
#include <iostream>
#include <new>
using namespace std;

int main()
{
    int *p;
    try {
        p = new int(100);
// initialize with 100

    } catch (bad_alloc xa) {
        cout << "Allocation Failure\n";
        return 1;
    }

    cout << "At " << p << " ";
    cout << "is the value " << *p << "\n";
    delete p;
    return 0;
}
```

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Ne vom reaminti de alocarea dinamica, atunci cand vom discuta (mai tarziu) despre:

- vector in STL (Standard Template Library)
- Allocator
- Comanda Push_back

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C Const în C++

- idee: să se elimine comenzile de preprocesor #define
- #define făceau substituție de valoare
- se poate aplica la pointeri, argumente de funcții, param de întoarcere din funcții, obiecte, funcții membru
- fiecare dintre aceste elemente are o aplicare diferită pentru const, dar sunt în aceeași idee/filosofie

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Const in C/C++

```
// Using const for safety
#include <iostream>
using namespace std;

const int i = 100; // Typical constant
const int j = i + 10; // Value from const expr
long address = (long)&j; // Forces storage
char buf[j + 10]; // Still a const expression

int main() {
    cout << "type a character & CR:";
    const char c = cin.get(); // Can't change //valoarea
    nu e cunoscuta la compile time si necesita storage
    const char c2 = c + 'a';
    cout << c2;
    // ...
}
```

- dacă știm că variabila nu se schimbă să o declaram cu const
- dacă încercăm să o schimbăm primim eroare de compilare

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Pointeri si Const in C/C++

- const poate fi aplicat valorii pointerului sau elementului pointat
 - const se alătură elementului cel mai apropiat
- `const int* u;`
- u este pointer către un int care este const
- `int const* v;` la fel ca mai sus

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Pointeri si Const in C/C++

- pentru pointeri care nu își schimbă adresa din memorie

```
int d = 1;
```

```
int* const w = &d;
```

- w e un pointer constant care arată către întregi+inițializare

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Pointeri si Const in C/C++

```
//: C08:ConstPointers.cpp
const int* u;
int const* v;

int d = 1;

int* const w = &d;

const int* const x = &d; // (1)
int const* const x2 = &d; // (2)

int main() {} ///:~
```

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Pointeri si Const in C/C++

- se poate face atribuire de adresă pentru obiect non-const către un pointer const
- nu se poate face atribuire pe adresă de obiect const către pointer non-const

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Pointeri si Const in C/C++

```
int d = 1;
```

```
const int e = 2;
```

```
int* u = &d; // OK -- d not const
```

```
//! int* v = &e; // Illegal -- e const
```

```
int* w = (int*)&e; // Legal but bad practice
```

```
int main() {} ///:~
```

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Const si argumente de funcții, param de întoarcere

- apel prin valoare cu const: param formal nu se schimbă în funcție
- const la întoarcere: valoarea returnată nu se poate schimba
- dacă se transmite o adresă: promisiune că nu se schimbă valoarea la adresa respectivă

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Const si argumente de funcții, param de întoarcere

```
void f1(const int i) {  
i++; // Illegal -- compile-time error  
}
```

cod mai clar echivalent mai jos:

```
void f2(int ic) {  
const int& i = ic;  
i++; // Illegal -- compile-time error  
}
```



1. Scurta recapitulare C++

Const si argumente de funcții, param de întoarcere

```
// Returning consts by value  
// has no meaning for built-in types
```

```
int f3() { return 1; }
```

```
const int f4() { return 1; }
```

```
int main() {
```

```
    const int j = f3(); // Works fine
```

```
    int k = f4(); // But this works fine too!
```

```
}
```

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Tipul referinta

O referință este, in esenta, un pointer implicit, care actioneaza ca un alt nume al unui obiect (variabila).

```
int i;  
int *pi,j;
```

int & ri=i; //ri este alt nume pentru variabila i

```
pi=&i; // pi este adresa variabilei i
```

```
*pi=3; //in zona adresata de pi se afla valoarea 3
```

Pentru a putea fi folosită, o referință trebuie inițializată in momentul declararii, devenind un alias (un alt nume) al obiectului cu care a fost inițializată.

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Tipul referinta

```
#include <iostream>
using namespace std;

int main()
{
    int a = 20;
    int& ref = a;
    cout<<a<<" "<<ref<<endl; // 20 20
    ref++;
    cout<<a<<" "<<ref<<endl; // 21 21
    return 0;
}
```

Obs: spre deosebire de pointeri care la incrementare trec la un alt obiect de acelasi tip, incrementarea referintei implica, de fapt, incrementarea valorii referite.

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Tipul referinta

```
#include <iostream>
using namespace std;

int main()
{
    int a = 20;
    int& ref = a;
    cout<<a<<" "<<ref<<endl; // 20 20
    int b = 50;
    ref = b;
    ref--;
    cout<<a<<" "<<ref<<endl; // 49 49
    return 0;
}
```

Obs: in afara initializarii, nu puteti modifica obiectul spre care indica referinta.

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Tipul referinta

- o referinta trebuie să fie initializata când este definita, dacă nu este membra a unei clase, un parametru de functie sau o valoare returnata;
- referintele nule sunt interzise intr-un program C++ valid.
- nu se poate obtine adresa unei referinte.
- nu se pot crea tablouri de referinte.
- nu se poate face referinta catre un camp de biti.

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Întoarcere de referințe

```
#include <iostream>
using namespace std;

char &replace(int i); // return a reference

char s[80] = "Hello There";

int main()
{
    replace(5) = 'X'; // assign X to space
    after Hello
    cout << s;
    return 0;
}

char &replace(int i)
{
    return s[i];
}
```

- putem face atribuiri către apel de funcție
- replace(5) este un element din s care se schimbă
- e nevoie de atenție ca obiectul referit să nu iasă din scopul de vizibilitate

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Transmiterea parametrilor

C

```
void f(int x){ x = x *2;}
```

```
void g(int *x){ *x = *x + 30;}
```

```
int main()
```

```
{
```

```
    int x = 10;
```

```
    f(x);
```

```
    printf("x = %d\n",x);
```

```
    g(&x);
```

```
    printf("x = %d\n",x);
```

```
    return 0;
```

```
}
```

C++

```
#include <iostream>
```

```
using namespace std;
```

```
void f(int x){ x = x *2;} //prin valoare
```

```
void g(int *x){ *x = *x + 30;} // prin pointer
```

```
void h(int &x){ x = x + 50;} //prin referinta
```

```
int main()
```

```
{
```

```
    int x = 10;
```

```
    f(x);
```

```
    cout<<"x = "<<x<<endl;
```

```
    g(&x);
```

```
    cout<<"x = "<<x<<endl;
```

```
h(x);
```

```
    cout<<"x = "<<x<<endl;
```

```
    return 0;
```

```
}
```


4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Transmiterea parametrilor

Observatii generale

- parametrii formali - sunt creati la intrarea intr-o functie si distrusi la retur;
- apel prin valoare - copiaza valoarea unui argument intr-un parametru formal $\Rightarrow$ modificarile parametrului nu au efect asupra argumentului;
- apel prin referinta - in parametru este copiata adresa unui argument $\Rightarrow$ modificarile parametrului au efect asupra argumentului.
- functiile, cu exceptia celor de tip void, pot fi folosite ca operand in orice expresie valida.

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Functii in structuri

C

```
#include <stdio.h>
#include <stdlib.h>
struct test
{
    int x;
    void afis()
    {
        printf("x= %d",x);
    }
}A;

int main()
{
    scanf("%d",&A.x);
    A.afis(); /* error 'struct test' has no
member called afis() */
    return 0;
}
```

C++

```
#include <iostream>
using namespace std;
struct test
{
    int x;
    void afis()
    {
        cout<<"x= "<<x;
    }
}A;

int main()
{
    cin>>A.x;
    A.afis();
    return 0;
}
```

4. Curs 1

4.1 Completări aduse de limbajul C++ față de limbajul C

Functii in structuri

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct test {
    int x;
    void (*afis)(struct test *this);
};
```

```
void afis_implicit(struct test *this) {
    printf("x= %d",this->x);
}
```

```
int main() {
    struct test A = {3, afis_implicit};
    A.afis(&A);
    return 0;
}
```

Q: Exista un mecanism prin care putem avea totusi functii in structuri in C?

A: Da, utilizand pointerii la functii

Q: Codul alaturat este valid si in C++?

A: Nu, pentru ca am folosit "this" ca identificator (mai tarziu despre "this").

Q: Daca putem folosi, totusi, functii in structuri in C, de ce folosim clase?

A: Pentru ca e dificil de emulat ascunderea informatiei, principiu de baza in POO.

4. Curs 1

4.2 Principiile programarii orientate pe obiecte

- ce este programarea
- definirea programatorului:
 - rezolva problema
- definirea informaticianului:
 - rezolva problema **bine**

4. Curs 1

4.2 Principiile programarii orientate pe obiecte

Rezolvarea “mai bine” a unei probleme

- “bine” depinde de caz
 - drivere: cat mai repede (asamblare)
 - jocuri de celulare: memorie mica
 - rachete, medicale: erori duc la pierderi de vieti
- programarea OO: cod mai corect
 - Microsoft: nu conteaza erorile minore, conteaza data lansarii
 - Utilizare principii S.O.L.I.D. (design patterns) – detalii mai tarziu!

4. Curs 1

4.2 Principiile programarii orientate pe obiecte

Principalele concepte utilizate in POO sunt:

Obiecte

Clase

Mostenire

Ascunderea informatiei

Polimorfism

Sabloane – nu sunt utilizate strict POO (mai general, se refera la Programarea generica)

4. Curs 1

4.2 Principiile programarii orientate pe obiecte

O **clasa** definește atribute și metode.

```
class X{  
    //date membre  
    //metode (functii membre – functii cu argument implicit  
    obiectul curent)  
};
```

- menționează proprietățile generale ale obiectelor din clasa respectivă
- folositoare la encapsulare (ascunderea informației)
- reutilizare de cod: moștenire

4. Curs 1

4.2 Principiile programarii orientate pe obiecte

Un **obiect** este o instanta a unei clase care are o anumita stare (reprezentata prin valoare) si are un comportament (reprezentat prin functii) la un anumit moment de timp.

- au stare si actiuni (metode/functii)
- au interfata (actiuni) si o parte ascunsa (starea)
- Sunt grupate in clase, obiecte cu aceleasi proprietati

Un **program orientat obiect** este o colectie de obiecte care interactioneaza unul cu celalalt prin mesaje (aplicand o metoda).

4. Curs 1

4.2 Principiile programarii orientate pe obiecte

Principalele concepte (caracteristici) ale POO sunt:

Incapsularea – contopirea datelor cu codul (metode de prelucrare si acces la date) în clase, ducând la o localizare mai bună a erorilor și la modularizarea problemei de rezolvat;

Moștenirea - posibilitatea de a extinde o clasa prin adaugarea de noi functionalitati;

- multe obiecte au proprietati similare
- reutilizare de cod

4. Curs 1

4.2 Principiile programarii orientate pe obiecte

Principalele concepte (caracteristici) ale POO sunt:

Ascunderea informatiei

foarte importanta

public, protected, private

Avem acces?	public	protected	private
Aceeasi clasa	da	da	da
Clase derivate	da	da	nu
Alte clase	da	nu	nu

4. Curs 1

4.2 Principiile programarii orientate pe obiecte

Principalele concepte (caracteristici) ale POO sunt:

Polimorfismul (la executie – *discutii mai ample mai tarziu*) – într-o ierarhie de clase obtinuta prin mostenire, o metodă poate avea implementari diferite la nivele diferite in acea ierarhie;

4. Curs 1

4.2 Principiile programarii orientate pe obiecte

Alt concept important in POO:

Sabloane

- cod mai sigur/reutilizare de cod
- putem implementa lista inlantuita de
 - intregi
 - caractere
 - float
 - obiecte

Perspective

1. Se vor discuta directiile principale ale cursului, feedback-ul studentilor fiind hotarator in acest aspect

- intelegerea notiunilor
- intrebari si sugestii

2. Cursul 2:

- Introducere in OOP. Clase. Obiecte



Programare orientată pe obiecte

- suport de curs -

Andrei Păun
Anca Dobrovăț

An universitar 2025 – 2026

Semestrul II

Seriile 13, 14, 15

Curs 2

Agenda cursului

- Recapitularea discuțiilor din cursul anterior
(Generalități despre curs, Reguli de comportament)
- Generalități despre OOP (Principiile programării orientate pe obiecte)
 - Clase, obiecte, modificatori de acces, funcții și clase prieten, constructori / destructor

Generalități despre curs

1. Curs – seria 15: luni (11 -14), seria 14: marti (14 – 17), seria 13: vineri (9 - 12)
2. Laborator – pe semigrupe, in fiecare saptamana
3. Seminar - o data la 2 saptamani

COLOCVIU: Data va fi anuntata (in principiu, ultima saptamana de curs) – se sustine fizic in facultate;

EXAMEN SCRIS: 11 iunie 2026, ora 9.00 – se sustine fizic in facultate

Daca cineva are o problema cu aceste date il/o rog sa ne anunte

In 2 saptamani datele acestea sunt fixate/finalizate

1. Scurta recapitulare C++

Completări aduse de limbajul C++ față de limbajul C

- C++98: a definit standardul inițial, toate chestiunile de limbaj, STL
- C++03: bugfix o unică chestie nouă: value initialization
- C++11: initializer lists, rvalue references, moving constructors, lambda functions, final, constant null pointer, etc.
- C++14: generic lambdas, binary literals, auto, variable template, etc.
- C++17: schimbări la STL pentru paralelizare, nested namespaces, inline variables, eliminat trigraphs, functii deprecated
- C++20: concepts, modules, three-way comparison, coroutines, constinit, volatile deprecated
- C++23: <expected>, <stacktrace>
- Următoarea plănuită în 2026 (C++2c)

1. Scurta recapitulare C++

Completări aduse de limbajul C++ față de limbajul C

•Intrări și ieșiri

- Obiectele **cin** și **cout**, în plus față de funcțiile scanf și printf din limbajul C.
- Nu necesită specificarea formatelor.
- Pentru I/O type-safe cu format: <format> (C++20)

•Comentarii pe o singură linie

•Citirea string-urilor până la primul caracter alb

1. Scurta recapitulare C++

Supraîncărcarea funcțiilor (un caz de *Polimorfism la compilare*)

Utilizarea mai multor funcții care au același nume

Identificarea se face prin numărul de parametri și tipul lor.

Tipul de întoarcere nu e suficient pentru a face diferența

Simplicitate/corectitudine de cod

1. Scurta recapitulare C++

Supraîncărcarea funcțiilor

Ambiguitati pentru polimorfism de functii

```
int myfunc(int i); // Error: differing return types are  
float myfunc(int i); // insufficient when overloading.
```

```
void f(int *p);  
void f(int p[]); // error, *p is same as p[]
```

```
void f(int x);  
void f(int& x);
```

```
int myfunc(int i) { return i; }  
int myfunc(int i, int j = 0) { return i*j; }
```

```
Apel: cout << myfunc(10);
```

1. Scurta recapitulare C++

Supraîncărcarea funcțiilor

Ambiguitati pentru polimorfism de functii

Obs.

```
int myfunc(double d); // ...  
cout << myfunc('c'); // not an error, conversion applied
```

Dar...

```
float myfunc(float i){ return i;}  
double myfunc(double i){ return -i;}  
  
int main(){  
    cout << myfunc(10.1) << " "; // unambiguous, calls myfunc(double)  
    cout << myfunc(10); // ambiguous  
    return 0; }
```

- problema nu e de definire a functiilor myfunc,
- problema apare la apelul functiilor

1. Scurta recapitulare C++

Funcții cu valori implicite

Într-o funcție se pot declara valori implicite pentru unul sau mai mulți parametri.

La apel se poate omite specificarea valorii pentru acei parametri formali care au declarate valori implicite.

Argumentele cu valori implicite trebuie să fie amplasate la sfârșitul listei.

Valorile implicite se specifică o singură dată în definiție (de obicei în prototip).

1. Scurta recapitulare C++

Funcții cu valori implicite

```
#include <iostream>
using namespace std;
```

```
void f (int a, int b = 12); // prototip cu mentionarea valorii implicite pentru b
```

```
int main()
{
    f(1);
    f(1,20);
    return 0;
}
```

```
void f (int a, int b) { cout<<a<<" - "<<b<<endl; }
```

1. Scurta recapitulare C++

Alocare dinamica

- new, delete (operatori nu funcții)
- se pot folosi încă malloc() și free() dar vor fi deprecated în viitor
- new: alocă memorie și întoarce un pointer la începutul zonei respective
- delete: sterge zona respectivă de memorie
- la eroare se “aruncă” excepția bad_alloc din <new>

1. Scurta recapitulare C++

Alocare dinamica

```
int *pi;
```

```
pi=new int;
```

```
delete pi; // elibereaza zona adresata de pi -o considera neocupata
```

```
pi=new int(2);// alocata zona si initializeaza zona cu valoarea 2
```

```
pi=new int[2]; // alocata un vector de 2 elemente de tip intreg
```

```
delete [ ] pi; //elibereaza intreg vectorul
```

```
//-pentru new se foloseste delete
```

```
//- pentru new [ ] se foloseste delete [ ]
```

1. Scurta recapitulare C++

Pointerii in C/C++

- O variabilă care ține o adresă din memorie
- Are un tip, compilatorul știe tipul de date către care se pointează
- Operațiile aritmetice țin cont de tipul de date din memorie
- `Pointer ++ == pointer+sizeof(tip)`
- Definiție: `tip *nume_pointer;`
 - Merge și `tip* nume_pointer;`

1. Scurta recapitulare C++

Pointerii in C/C++

```
#include <stdio.h>
int main(void)
{
    double x = 100.1, y;
    int *p;
    /* The next statement causes p (which
       is an integer pointer) to point to a
       double. */
    p = (int *)&x;
    /* The next statement does not operate
       as expected. */ y = *p;
    printf("%f", y); /* won't output 100.1 */
    return 0;
}
```

- Schimbarea de tip nu e controlată de compiler
- În C++ conversiile trebuie făcute cu schimbarea de tip

1. Scurta recapitulare C++

Pointerii in C/C++

pointeri și array-uri

- numele array-ului este pointer
- `lista[5]==*(lista+5)`
- array de pointeri, numele listei este un pointer către pointeri (dublă indirectare)
- `int **p;` (dublă indirectare)

1. Scurta recapitulare C++

Const in C++

- idee: să se elimine comenzile de preprocesor #define
- #define făceau substituție de valoare
- se poate aplica la pointeri, argumente de funcții, param de întoarcere din funcții, obiecte, funcții membru
- fiecare dintre aceste elemente are o aplicare diferită pentru const, dar sunt în aceeași idee/filosofie

1. Scurta recapitulare C++

Const in C/C++

```
// Using const for safety
#include <iostream>
using namespace std;

const int i = 100; // Typical constant
const int j = i + 10; // Value from const expr
long address = (long)&j; // Forces storage
char buf[j + 10]; // Still a const expression

int main() {
    cout << "type a character & CR:";
    const char c = cin.get(); // Can't change //valoarea
    nu e cunoscuta la compile time si necesita storage
    const char c2 = c + 'a';
    cout << c2;
    // ...
}
```

- dacă știm că variabila nu se schimbă să o declaram cu const
- dacă încercăm să o schimbăm primim eroare de compilare

1. Scurta recapitulare C++

Pointeri si Const in C/C++

- const poate fi aplicat valorii pointerului sau elementului pointat
- const se alătură elementului cel mai apropiat

`const int* u;`

- u este pointer către un int care este const

`int const* v;` la fel ca mai sus

1. Scurta recapitulare C++

Pointeri si Const in C/C++

- pentru pointeri care nu își schimbă adresa din memorie

```
int d = 1;
```

```
int* const w = &d;
```

- w e un pointer constant care arată către întregi+inițializare

1. Scurta recapitulare C++

Pointeri si Const in C/C++

```
//: C08:ConstPointers.cpp
const int* u;
int const* v;

int d = 1;

int* const w = &d;

const int* const x = &d; // (1)
int const* const x2 = &d; // (2)

int main() {} ///:~
```

1. Scurta recapitulare C++

Pointeri si Const in C/C++

- se poate face atribuire de adresă pentru obiect non-const către un pointer const
- nu se poate face atribuire pe adresă de obiect const către pointer non-const

1. Scurta recapitulare C++

Pointeri si Const in C/C++

```
int d = 1;
```

```
const int e = 2;
```

```
int* u = &d; // OK -- d not const
```

```
//! int* v = &e; // Illegal -- e const
```

```
int* w = (int*)&e; // Legal but bad practice
```

```
int main() {} ///:~
```

1. Scurta recapitulare C++

Const si argumente de funcții, param de întoarcere

- apel prin valoare cu const: param formal nu se schimbă în funcție
- const la întoarcere: valoarea returnată nu se poate schimba
- dacă se transmite o adresă: promisiune că nu se schimbă valoarea la adresa respectivă

1. Scurta recapitulare C++

Const si argumente de functii, param de întoarcere

```
void f1(const int i) {  
i++; // Illegal -- compile-time error  
}
```

cod mai clar echivalent mai jos:

```
void f2(int ic) {  
const int& i = ic;  
i++; // Illegal -- compile-time error  
}
```

1. Scurta recapitulare C++

Const si argumente de funcții, param de întoarcere

```
// Returning consts by value  
// has no meaning for built-in types
```

```
int f3() { return 1; }
```

```
const int f4() { return 1; }
```

```
int main() {
```

```
    const int j = f3(); // Works fine
```

```
    int k = f4(); // But this works fine too!
```

```
}
```

1. Scurta recapitulare C++

Tipul referinta

O referință este, in esenta, un pointer implicit, care actioneaza ca un alt nume al unui obiect (variabila).

```
int i;  
int *pi,j;
```

int & ri=i; //ri este alt nume pentru variabila i

```
pi=&i; // pi este adresa variabilei i  
*pi=3; //in zona adresata de pi se afla valoarea 3
```

Pentru a putea fi folosită, o referință trebuie inițializată in momentul declararii, devenind un alias (un alt nume) al obiectului cu care a fost inițializată.

1. Scurta recapitulare C++

Tipul referinta

```
int a = 20;  
int& ref = a;  
cout<<a<<" "<<ref<<endl; // 20 20
```

ref++;

```
cout<<a<<" "<<ref<<endl; // 21 21
```

Obs: spre deosebire de pointeri care la incrementare trec la un alt obiect de acelasi tip, incrementarea referintei implica, de fapt, incrementarea valorii referite.

1. Scurta recapitulare C++

Tipul referinta

```
int a = 20;  
int& ref = a;  
cout<<a<<" "<<ref<<endl; // 20 20
```

```
int b = 50;  
ref = b;
```

```
ref--;  
cout<<a<<" "<<ref<<endl; // 49 49
```

Obs: in afara initializarii, nu puteti modifica obiectul spre care indica referinta.

1. Scurta recapitulare C++

Tipul referinta

- o variabilă de tip referință trebuie să fie inițializată când este definită
- membrii unei clase de tip referință trebuie inițializați în constructori
- un parametru de funcție de tip referință se inițializează implicit la apel
- referințele nule sunt interzise într-un program C++ valid.
 - putem întoarce referințe dangling (warning la compilare); //
dangling – un pointer/referinta care “arata” catre o zona de memorie deja dealocata
- nu se poate obține adresa unei referințe (&ref ne dă adresa variabilei)
 - intern sunt tot pointeri, dar așa e mai greu să îi "stricăm"
- se pot crea tablouri de referințe cu [std::reference_wrapper](#)

1. Scurta recapitulare C++

Întoarcere de referințe

```
#include <iostream>
using namespace std;

char &replace(int i); // return a reference

char s[80] = "Hello There";

int main()
{
    replace(5) = 'X'; // assign X to space
    after Hello
    cout << s;
    return 0;
}

char &replace(int i)
{
    return s[i];
}
```

- putem face atribuiri către apel de funcție
- replace(5) este un element din s care se schimbă
- e nevoie de atenție ca obiectul referit să nu iasă din scopul de vizibilitate

1. Scurta recapitulare C++

Transmiterea parametrilor

<pre>C void f(int x) { x = x *2;} void g(int *x) { *x = *x + 30;} int main() { int x = 10; f(x); printf("x = %d\n",x); g(&x); printf("x = %d\n",x); return 0; }</pre>	<pre>C++ void f(int x){ x = x *2;} //prin valoare void g(int *x){ *x = *x + 30;} // prin pointer void h(int &x){ x = x + 50;} //prin referinta int main() { int x = 10; f(x); cout<<"x = "<<x<<endl; g(&x); cout<<"x = "<<x<<endl; h(x); cout<<"x = "<<x<<endl; return 0; }</pre>
---	---

1. Scurta recapitulare C++

Transmiterea parametrilor

Observatii generale

- parametrii formali - sunt creati la intrarea intr-o functie si distrusi la retur;
- apel prin valoare - copiaza valoarea unui argument intr-un parametru formal $\Rightarrow$ modificarile parametrului nu au efect asupra argumentului;
- apel prin referinta - in parametru este copiata adresa unui argument $\Rightarrow$ modificarile parametrului au efect asupra argumentului.
- functiile, cu exceptia celor de tip void, pot fi folosite ca operand in orice expresie valida.

1. Scurta recapitulare C++

Functii in structuri

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct test {
    int x;
    void (*afis)(struct test *this);
};
```

```
void afis_implicit(struct test *this) {
    printf("x= %d",this->x);
}
```

```
int main() {
    struct test A = {3, afis_implicit};
    A.afis(&A);
    return 0;
}
```

Q: Exista un mecanism prin care putem avea totusi functii in structuri in C?

A: Da, utilizand pointerii la functii

Q: Codul alaturat este valid si in C++?

A: Nu, pentru ca am folosit "this" ca identificator (mai tarziu despre "this").

Q: Daca putem folosi, totusi, functii in structuri in C, de ce folosim clase?

A: Pentru ca e dificil de emulat ascunderea informatiei, principiu de baza in POO.

1. Scurta recapitulare C++

Cand tipul returnat de o functie nu este declarat explicit, i se atribuie automat int.

Tipul trebuie cunoscut inainte de apel.

```
f (double x) {  
    return x;  
}
```

Prototipul unei functii: permite declararea in afara si a numarului de parametri / tipul lor:

```
void f(int); // antet / prototip  
int main() { cout<< f(50); }  
void f( int x) { // corp functie; }
```

2. Principiile programarii orientate pe obiecte

Clasa

Obiect

Incapsularea

Modularitatea

Ierarhizarea.

Polimorfism la compilare si polimorfism la executie, etc.

2. Principiile programarii orientate pe obiecte

Obiecte

- au stare și acțiuni (metode/funcții)
- au interfață (acțiuni) și o parte ascunsă (starea)
- Sunt grupate în clase, obiecte cu aceleași proprietăți
- Un **program orientat obiect** este o colecție de obiecte care interactionează unul cu celălalt prin mesaje (aplicand o metodă).

2. Principiile programarii orientate pe obiecte

Clase

O **clasa** definește atribute și metode.

```
class X{  
    //date membre  
    //metode (functii membre – functii cu argument  
implicit obiectul curent)  
};
```

- menționează proprietățile generale ale obiectelor din clasa respectivă
- folosite la încapsulare (ascunderea informației)
- reutilizare de cod: moștenire

2. Principiile programarii orientate pe obiecte

Clase

- cu “class”
- obiectele instanțiază clase
- similare cu struct-uri și union-uri
- au funcții
- specificatorii de acces: public, private, protected
- default: private
- protected: pentru moștenire, vorbim mai târziu

2. Principiile programarii orientate pe obiecte

Clase

```
class nume_clasă {  
    private variabile și funcții membru  
    specificator_de_acces:  
        variabile și funcții membru  
    specificator_de_acces:  
        variabile și funcții membru  
    // ...  
    specificator_de_acces:  
        variabile și funcții membru  
} listă_obiecte;
```

- putem trece de la public la private și iar la public, etc.

2. Principiile programarii orientate pe obiecte

Clase

- variabilele de instanta (instance variables)
- membri de tip date ai clasei
 - in general private
 - pentru viteza se pot folosi “public” dar **NU LA ACEST CURS**

2. Principiile programarii orientate pe obiecte

Clase

- se folosește mai mult a doua variantă
- un membru (ne-static) al clasei nu poate avea inițializare
- nu putem avea ca membri obiecte de tipul clasei (putem avea pointeri la tipul clasei)
- nu auto, extern, register

2. Principiile programarii orientate pe obiecte

Clase

```
class employee {  
    // private din oficiu  
        char name[80];  
public:  
    // acestea sunt publice  
        void putname(char *n);  
        void getname(char *n);  
private:  
    // acum din nou private  
        double wage;  
public:  
    // înapoi la public  
        void putwage(double w);  
        double getwage();  
};
```

```
class employee {  
        char name[80];  
        double wage;  
public:  
        void putname(char *n);  
        void getname(char *n);  
        void putwage(double w);  
        double getwage();  
};
```

2. Principiile programarii orientate pe obiecte

Clase

```
#define SIZE 100
// This creates the class stack.
class stack {
    int stck[SIZE];
    int tos;
public:
    void init();
    void push(int i);
    int pop();
};
```

- init(), push(), pop() sunt funcții membru
- stck, tos: variabile membru

2. Principiile programarii orientate pe obiecte

Clase

- se creează un tip nou de date
- un obiect instanțiază clasa
- funcțiile membru sunt date prin semnătură
- pentru definirea fiecărei funcții se folosește ::

stack mystack;

```
void stack::push(int i) {  
    if(tos==SIZE) {  
        cout << "Stack is full.\n";  
        return; }  
    stck[tos] = i;  
    tos++;  
}
```

2. Principiile programarii orientate pe obiecte

Clase

- :: scope resolution operator
- și alte clase pot folosi numele push() și pop()
- după instantiere, pentru apelul push()

stack mystack;

- mystack.push(5);
- programul complet în continuare

2. Principiile programarii orientate pe obiecte

```
#include <iostream>
using namespace std;
#define SIZE 100
// This creates the class stack.
class stack {
    int stck[SIZE];
    int tos;

public:
    void init();
    void push(int i);
    int pop();
};

void stack::init()
{
    tos = 0;
}

void stack::push(int i) {
    if(tos==SIZE) {
        cout << "Stack is full.\n";
        return;
    }
    stck[tos] = i;
    tos++;
}
```

```
int stack::pop()
{
```

```
    if(tos==0) {
        cout << "Stack underflow.\n";
        return 0;
    }
    tos--;
    return stck[tos];
}
```

```
}

int main()
{
```

```
    stack stack1, stack2; // create two stack objects
    stack1.init();
    stack2.init();
    stack1.push(1);
    stack2.push(2);
    stack1.push(3);
    stack2.push(4);
    cout << stack1.pop() << " ";
    cout << stack1.pop() << " ";
    cout << stack2.pop() << " ";
    cout << stack2.pop() << "\n";
    return 0;
}
```

Clase

2. Principiile programarii orientate pe obiecte

Incapsularea

- ascunderea de informații (data-hiding);
- separarea aspectelor externe ale unui obiect care sunt accesibile altor obiecte de aspectele interne ale obiectului care sunt ascunse celorlalte obiecte;
- definește apartenența unor proprietăți și metode față de un obiect;
- doar metodele proprii ale obiectului pot accesa starea acestuia.

2. Principiile programarii orientate pe obiecte

Incapsularea (ascunderea informatiei)

foarte importanta

public, protected, private

Avem acces?	public	protected	private
Aceeasi clasa	da	da	da
Clase derivate	da	da	nu
Alte clase	da	nu	nu

2. Principiile programarii orientate pe obiecte

Incapsularea (ascunderea informatiei)

```
class Test {  
    private:  
    int x;  
    public:  
    void set_x(int a) {x = a;}  
};
```

```
int main() {  
    Test t;  
    t.x = 34; // inaccesibil  
    t.set_x(34); // ok  
    return 0;  
}
```

2. Principiile programarii orientate pe obiecte

- **Agregarea** (ierarhia de obiecte) compunerea unui obiect din mai multe obiecte mai simple. (relație de tip “has a”)
- **Moștenirea** (ierarhia de clase) relație între clase în care o clasă mosteneste structura și comportarea definită în una sau mai multe clase (relație de tip “is a” sau “is like a”);

2. Principiile programarii orientate pe obiecte

Agregarea

```
class Profesor
{
    string nume;
    int vechime;
};
```

```
class Curs
{
    string denumire;
    Profesor p;
};
```


2. Principiile programarii orientate pe obiecte

Moștenire

- multe obiecte au proprietăți similare
- reutilizare de cod

2. Principiile programarii orientate pe obiecte

Moștenire

- terminologie
 - clasă de bază, clasă derivată
 - superclasă subclasă
 - părinte, fiu
- mai târziu: funcții virtuale, identificare de tipuri în timpul rulării (RTTI)

2. Principiile programarii orientate pe obiecte

Moștenire

- încorporarea componentelor unei clase în alta
- refolosire de cod
- detalii mai subtile pentru tipuri și subtipuri
- clasă de bază, clasă derivată
- clasa derivată conține toate elementele clasei de bază, mai adăugă noi elemente

2. Principiile programarii orientate pe obiecte

Moștenire

```
class building {  
    int rooms;  
    int floors;  
    int area;  
public:  
    void set_rooms(int num);  
    int get_rooms(); // ...  
};
```

// house e derivată din building

```
class house : public building {  
    int bedrooms;  
    int baths;  
public:  
    void set_bedrooms(int num);  
    int get_bedrooms();};
```

tip acces: public, private, protected

mai multe mai târziu

public: membrii publici ai building devin publici pentru house

2. Principiile programarii orientate pe obiecte

Moștenire

- house NU are acces la membrii privați ai lui building
- așa se realizează encapsularea
- clasa derivată are acces la membrii publici ai clasei de baza și la toți membrii săi (publici și privați)

2. Principiile programarii orientate pe obiecte

Moștenire

```
class building {  
    int rooms;  
    int floors;  
    int area;  
  
    public:  
    void set_rooms(int num);  
    int get_rooms(); //...};
```

```
class house : public building {  
    int bedrooms;  
    int baths;  
  
    public:  
    void set_bedrooms(int num);  
    int get_bedrooms();  
    void set_baths(int num);  
    int get_baths();  
  
};
```

// school este de asemenea derivată din building

```
class school : public building {  
    int classrooms;  
    int offices;  
  
    public:  
    void set_classrooms(int num);  
    int get_classrooms();  
    void set_offices(int num);  
    int get_offices();  
};
```

Moştenire

```
void building::set_rooms(int num)
{ rooms = num; }
void building::set_floors(int num)
{ floors = num; }
void building::set_area(int num)
{ area = num; }
int building::get_rooms()
{ return rooms; }
int building::get_floors()
{ return floors; }
int building::get_area()
{ return area; }
void house::set_bedrooms(int num)
{ bedrooms = num; }
void house::set_baths(int num)
{ baths = num; }
int house::get_bedrooms()
{ return bedrooms; }
int house::get_baths()
{ return baths; }
void school::set_classrooms(int num)
{ classrooms = num; }
void school::set_offices(int num)
{ offices = num; }
int school::get_classrooms()
{ return classrooms; }
int school::get_offices()
{ return offices; }
```

```
int main()
{
    house h;
    school s;
    h.set_rooms(12);
    h.set_floors(3);
    h.set_area(4500);
    h.set_bedrooms(5);
    h.set_baths(3);
    cout << "house has " << h.get_bedrooms();
    cout << " bedrooms\n"; s.set_rooms(200);
    s.set_classrooms(180);
    s.set_offices(5);
    s.set_area(25000);
    cout << "school has " << s.get_classrooms();
    cout << " classrooms\n";
    cout << "Its area is " << s.get_area();
    return 0;
}
```

house has 5 bedrooms
school has 180 classrooms
Its area is 25000

2. Principiile programarii orientate pe obiecte

Polimorfism

- tot pentru claritate/ cod mai sigur
- Polimorfism la compilare: ex. `max(int)`, `max(float)`
- Polimorfism la execuție: RTTI

2. Principiile programarii orientate pe obiecte

Șabloane

- din nou cod mai sigur/reutilizare de cod
- putem implementa listă înlănțuită de
 - întregi
 - caractere
 - float
 - obiecte

Perspective

Curs 3

- Class, struct, union
- Functii si clase prieten
- Functii inline
- Constructori / destructor



Programare orientată pe obiecte

- suport de curs -

Andrei Păun
Anca Dobrovăț

An universitar 2025 – 2026
Semestrul II
Seriile 13, 14, 15

Curs 3

Cuprinsul cursului

- Class, struct, union
- Functii si clase prieten
- Functii inline
- Constructori / destructor

Struct si class

- singura diferenta: struct are default membri ca public iar class ca private
- struct defineste o clasa (tip de date)
- putem avea in struct si functii
- pentru compatibilitate cu cod vechi
- extensibilitate
- **a nu se folosi struct pentru clase**

// Utilizarea unei structuri pentru a defini o clasa.

```
#include <iostream>
#include <cstring>
using namespace std;
```

```
struct mystr {
    void buildstr(char *s)
        { if(!*s) *str = '\0';
          else strcat(str, s);}
    void showstr() {cout << str << "\n"; }
private:
    char str[255];
};
```

```
class mystr {
    char str[255];
public:
    void buildstr(char *s); //
public
    void showstr();
};
```

Union si class

- la fel ca struct
- toate elementele de tip data folosesc aceeaasi locatie de memorie
- membrii sunt publici (by default)

Union si class

```
#include <iostream>
```

```
using namespace std;
```

```
union swap_byte {  
    void swap() { unsigned char t = c[0]; c[0] = c[1]; c[1] = t; }  
    void set_byte(unsigned short i) { u = i; }  
    void show_word() { cout << u; }  
  
    unsigned short u;  
    unsigned char c[2];  
};
```

```
int main() {  
    swap_byte b;  
    b.set_byte(49034);  
    b.swap();  
    b.show_word();  
    return 0;  
}
```


Union ca o clasa

- union nu poate mosteni
- nu se poate mosteni din union
- nu poate avea functii virtuale (nu avem mostenire)
- nu avem variabile de instanta statice
- nu avem referinte in union
- nu avem obiecte care fac overload pe =
- obiecte cu (con/de)structor definiti nu pot fi membri in union

Union anonime

- nu au nume pentru tip
- nu se pot declara obiecte de tipul respectiv
- folosite pentru a spune compilatorului cum se aloc/procesez variabilele respective in memorie
 - folosesc aceeaasi locatie de memorie
- variabilele din union sunt accesibile ca si cum ar fi declarate in blocul respectiv

Union anonime

```
#include <iostream>
#include <cstring>
using namespace std;
```

```
int main()
{
```

```
    union {
        long l;
        double d;
        char s[4];
    };
```

```
    l = 100000;
    cout << l << " ";
    d = 123.2342;
    cout << d << " ";
    strcpy(s, "hi");
    cout << s;
    return 0;
```

```
}
```

Union anonime

- nu poate avea functii
- nu poate avea private sau protected (fara functii nu avem acces la altceva)
- union-uri anonime globale trebuiesc precizate ca statice

Functii prieten

- Cuvantul cheie: **friend**
- pentru accesarea campurilor protected, private din alta clasa
- folositoare la overload-area operatorilor, pentru unele functii de I/O, si portiuni interconectate (exemplu urmeaza)
- in rest nu se prea folosesc

Functii prieten pentru o clasa

```
#include <iostream>
using namespace std;
class myclass {
    int a, b;
public:
    friend int sum(myclass x); // poate accesa direct a si b private
    void set_ab(int i, int j) { a = i; b = j; }
};
```

```
int sum(myclass x) {    return x.a + x.b; }
```

```
int main() {
    myclass n;
    n.set_ab(3, 4);
    cout << sum(n);
    return 0;
}
```

Functii prieten pentru mai multe clase

```
#include <iostream>
```

```
using namespace std;
```

```
class C2;
```

```
class C1 {    int x;
```

```
public:
```

```
    void set_x(int a){x = a;}
```

```
    friend void f(C1, C2);
```

```
};
```

```
class C2 {    int y;
```

```
public:
```

```
    void set_y(int b){y = b;}
```

```
    friend void f(C1, C2);
```

```
};
```

```
void f(C1 ob1, C2 ob2)
{ cout<<ob1.x + ob2.y<<"\n";}
```

```
int main()
```

```
{
```

```
    C1 A;
```

```
    C2 B;
```

```
    A.set_x(10);
```

```
    B.set_y(20);
```

```
    f(A,B);
```

```
}
```

Functii prieten din alte obiecte

```
#include <iostream>
using namespace std;
class C2;
class C1 {    int x;
public:
    void set_x(int a){x = a;}
    void f(C2);
};

class C2 {    int y;
public:
    void set_y(int b){y = b;}
    friend void C1::f(C2);
};
```

```
void C1::f(C2 ob2)
{ cout<<this->x + ob2.y<<"\n";}
```

```
int main()
{
    C1 A;
    C2 B;
    A.set_x(10);
    B.set_y(20);
    A.f(B);
}
```


Clase prieten

- Declararea unei clase Y ca prieten al unei clase X, are ca efect ca toate functiile membre ale clasei Y au acces la membrii privati ai clasei X.

```
#include <iostream>
```

```
class C1 {    int x;
```

```
public:
```

```
    friend class C2;
```

```
};
```

```
class C2 {
```

```
public:
```

```
    void set_x(int a, C1& ob){ ob.x = a;}
```

```
    int get_x (C1 ob) {return ob.x;}
```

```
};
```

```
int main()
```

```
{
```

```
    C1 A;
```

```
    C2 B;
```

```
    B.set_x(10,A);
```

```
    std::cout<<B.get_x(A);
```

```
}
```

Funcții inline

- execuție rapidă
- este o sugestie/cerere pentru compilator
- pentru funcții foarte mici
- pot fi și membri ai unei clase
- foarte comune în clase
- două tipuri: explicit (**inline**) și implicit

Explicit inline

```
#include <iostream>
using namespace std;
```

```
inline int max(int a, int b)
{
    return a>b ? a : b;
}
```

```
int main()
{
    cout << max(10, 20);
    cout << " " << max(99, 88);
    return 0;
}
```

```
#include <iostream>
using namespace std;
```

```
int main()
{
    cout << (10>20 ? 10 : 20);
    cout << " " << (99>88 ? 99 : 88);
    return 0;
}
```

Explicit inline in clase

```
#include <iostream>
using namespace std;
```

```
class myclass {
    int a, b;
public:
    void init(int i, int j);
    void show();
};
```

```
inline void myclass::init(int i, int j)
{ a = i; b = j; }
```

```
inline void myclass::show()
{ cout << a << " " << b << "\n"; }
```

```
int main() {
    myclass x;
    x.init(10, 20);
    x.show();
    return 0;
}
```

Definirea functiilor inline implicit (in clase)

```
#include <iostream>
using namespace std;

class myclass {
    int a, b;
public:
    // automatic inline
    void init(int i, int j)
    {
        a = i;
        b = j;
    }

    void show() { cout << a << " " << b << "\n"; }
};

int main() {
    myclass x;
    x.init(10, 20);
    x.show();
    return 0;
}
```

Constructor/Destructor

- inițializare automată
- efectueaza operatii prealabile utilizarii obiectelor create
- obiectele nu sunt statice
- constructor: funcție specială, numele clasei
- constructorii nu pot întoarce valori (nu au tip de întoarcere)

Constructor/Destructor

Caracteristici speciale:

- numele = numele clasei (~ numele clasei pentru destructori);
- la declarare nu se specifica tipul returnat;
- nu pot fi mosteniti, dar pot fi apelati de clasele derivate;
- nu se pot utiliza pointeri către functiile constructor / destructor;
- constructorii pot avea parametri (inclusiv impliciti) și se pot supradefini. Destructorul este unic și fără parametri.

Constructori/Destructor

Constructorii de copiere (discuții mai ample mai tarziu)

- creaza un obiect preluand valorile corespunzatoare altui obiect;
- exista implicit (copiata bit-cu-bit, deci trebuie redefinit la date alocate dinamic).

Constructorii/Destructorii

Orice clasa, are by default:

- un constructor de initializare
- un constructor de copiere
- un destructor
- un operator de atribuire.

Constructorii parametrizati:

- argumente la constructori
 - putem defini mai multe variante cu mai multe numere si tipuri de parametri
- overload de constructori (mai multe variante, cu numar mai mare și tipuri de parametri).

Constructor/Destructor

Orice clasa, are by default:

```
class A
{
    int x;
    float y;
    string z;
};

int main()
{
    A a;      // apel constructor de initializare - fara parametri
    A b = a;  // apel constructor de copiere
    A e(a);   // apel constructor de copiere
    A c;      // apel constructor de initializare
    c = a;    //operatorul de atribuire (=)
}
```

Constructor/Destructor

Exemplu – necesitate rescriere constructori

```
#include <iostream>

using namespace std;

class A
{
    int *v;
public:
    A() {v = new int[3]; v[0] = v[1] = v[2] = 123; cout<<"C\n";}
    ~A() {delete[] v; cout<<"D\n";}
    void afis() {cout<<v[2]<<"\n";}
};

void functie(A ob) {ob.afis();}

int main()
{
    A ob1;
    functie(ob1);
    ob1.afis();
    return 0;
}
```

```
C
123
D
15925584
```

```
Process returned -1073740940 (0xC0000374)
```

Constructor/Destructor

Exemplu - Constructori parametrizati

```
class A
{
    int x;
    float y;
    string z;
public:
    A(){x = -45;}
    A(int x) {this->x = x; this->y = 5.67; z = "Seria 13";}
    // this -> camp e echivalent cu camp simplu: this -> z echivalent cu z
    A(int x, float y) {this->x = x; this->y = y; z = "Seria 13";}
    A(int x, float y, string z) {this->x = x; this->y = y; this->z = z;}

int main() {
    A a;
    A b(34);
    return 0;
}
```

Constructor/Destructor

Exemplu - Constructori parametrizati

```
class A
{
    int x;
    float y;
    string z;
public:
```

```
    A(int x = -45, float y = 5.67, string z = "Seria 13") // valori implicite pt param
    {this->x = x; this->y = y; this->z = z;}
};
```

```
int main()
{
    A a;
    A b(34);
    return 0;
}
```

Constructori/Destructori

Funcțiile constructor cu un parametru – caz special (sursa H. Schildt)

```
#include <iostream>
using namespace std;
```

```
class X {
    int a;
public:
    X(int j) { a = j; }
    int geta() { return a; }
};
```

Se creeaza o conversie
explicita de date!

```
int main()
{
    X ob = 99; // passes 99 to j
    cout << ob.geta(); // outputs 99
    return 0;
}
```

Constructorii/Destructorii

Tablouri de obiecte

Daca o clasa are constructori parametrizati, putem initializa diferit fiecare obiect din vector.

```
class X{int a,b,c; ... };  
X v[3] = {X(10,20) , X (1,2,3), X(0) };
```

Daca constr are un singur parametru, atunci se poate specifica direct valoarea.

```
class X {  
    int i;  
public:  
    X(int j) { i = j;}  
};  
  
X v[3] = {10,15,20 };
```

Constructorii/Destructorii

Exemplu – Ordine apel

```
class A
{
    int x;
public:
    A(int x = 0)    {
        x = x;
        cout<<"Constructor "<<x<<endl;    }
    ~A()    {
        cout<<"Destructor "<<endl;    }
    A(const A&o)    {
        x = o.x;
        cout<<"Constructor de copiere"<<endl;    }
    void f_cu_referinta(A& ob3)    {
        A ob4(456);    }
    void f_fara_referinta(A ob6)    {
        A ob7(123);    }
} ob;
```

```
int main()
{
    A ob1(20), ob2(55);
    ob2.f_cu_referinta(ob1);
    ob1.f_fara_referinta(ob2);
    A ob5;
    return 0;
}
```


Constructorii/Destructorii

Exemplu – Ordine apel

- 1) In acelasi domeniu de vizibilitate, constructorii se apeleaza in ordinea declararii obiectelor, iar destructorii in sens invers.
- 2) Variabilele globale se declara inaintea celor locale, deci constructorii lor se declara primii.
- 3) Daca o functie are ca parametru un obiect, care nu e transmis cu referinta atunci, se activeaza constructorul de copiere si, implicit, la iesirea din functie, obiectul copie se distruge, deci se apeleaza destructor

```
/// Ordine:  
/// 1. Constructor ob;  
/// 2. Constructor ob1;  
/// 3. Constructor ob2;  
/// 4. Constructor ob4; - ob3 e referinta/alias-ul ob1, nu se creeaza obiect nou  
/// 5. Destructor ob4;  
/// 6. Constructor copiere ob6  
/// 7. Constructor ob7  
/// 8. Destructor ob7  
/// 9. Destructor ob6  
/// 10. Constructor ob5  
/// 11. Destructor ob5  
/// 12. Destructor ob2  
/// 13. Destructor ob1  
/// 14. Destructor ob
```

Constructor/Destructor

Exemplu – Ordine apel

```
class A {  
public:  
    A(){cout<<"Constr A"<<endl;}  
};
```

```
class B {  
public:  
    B() {cout<<"Constr B"<<endl;}  
private:  
    A ob;  
};
```

```
int main()  
{  
    B ob2;  
    /// Apel constructor obiect A si apoi constructorul propriu  
}
```

Constructor/Destructor

```
class A {
    int x;
public:
    A(int x = 7){this->x = x; cout<<"Const "<<x<<endl;}
    void set_x(int x){this->x = x;}
    int get_x(){ return x;}
    ~A(){cout<<"Dest "<<x<<endl;}
};

void afisare(A ob) {
    ob.set_x(10);
    cout<<ob.get_x()<<endl; }

int main ( ) {
    A o1;
    cout<<o1.get_x()<<endl;
    afisare(o1);
    return 0;
}
```

Exemplu – Ce se afiseaza?

```
Const 7 // obiect o1
7 // o1.get_x()
10 // in functie ob.get_x()
Dest 10 // ob
Dest 7 // o1
```

Constructori/Destructori

```
class cls { public:  
    cls() { cout << "Inside constructor 1" << endl; }  
    ~cls() { cout << "Inside destructor 1" << endl; } };
```

```
class cls2 {  
    cls xx;  
public:  
    cls2() { cout << "Inside constructor 2" << endl; }  
    ~cls2() { cout << "Inside destructor 2" << endl; } };
```

```
class cls3 {  
    cls2 xx;  
    cls3 xxx;  
public:  
    cls3() { cout << "Inside constructor 3" << endl; }  
    ~cls3() { cout << "Inside destructor 3" << endl; } };
```

```
int main() {  
    cls3 s;  
}
```

Exemplu – Ce se afiseaza?

Polimorfism pe constructori

- foarte comun sa fie supraincarcati
- de ce?
 - flexibilitate
 - pentru a putea defini obiecte initializate si neinitializate
 - constructori de copiere: copy constructors

overload pe constructori: flexibilitate

- putem avea mai multe posibilitati pentru initializarea/construirea unui obiect
- definim constructori pentru toate modurile de initializare
- daca se incearca initializarea intr-un alt fel (decat cele definite): eroare la compilare

polimorfism de constructori: obiecte initializate si ne-initializate

- important pentru array-uri dinamice de obiecte
- nu se pot initializa obiectele dintr-o lista alocata dinamic
- asadar avem nevoie de posibilitatea de a crea obiecte neinitializate (din lista dinamica) si obiecte initializate (definite normal)

```

#include <iostream>
#include <new>
using namespace std;
class powers
{ int x;
public:
    // overload constructor two ways

```

- ofThree si lista p au nevoie de constructorul fara parametri

```

};

int main()
{
    powers ofTwo[] = {1, 2, 4, 8, 16}; // initialized
    powers ofThree[5]; // uninitialized
    powers *p;
    int i; // show powers of two
    cout << "Powers of two: ";
    for(i=0; i<5; i++) {
        cout << ofTwo[i].getx() << " ";
    }
    cout << "\n\n";

```

```

// set powers of three
ofThree[0].setx(1); ofThree[1].setx(3);
ofThree[2].setx(9); ofThree[3].setx(27);
ofThree[4].setx(81);

// show powers of three
cout << "Powers of three: ";
for(i=0; i<5; i++) { cout << ofThree[i].getx() << " ";}
cout << "\n\n";

```

```

return 1;}

```

```

// initialize dynamic array with powers of two
for(i=0; i<5; i++) { p[i].setx(ofTwo[i].getx());}

// show powers of two
cout << "Powers of two: ";
for(i=0; i<5; i++) { cout << p[i].getx() << " ";}
cout << "\n\n";
delete [] p;
return 0;
}

```


polimorfism de constructori:

constructorul de copiere

- pot apărea probleme când un obiect inițializează un alt obiect

MyClass B = A;

- aici se copiază toate câmpurile (starea) obiectului A în obiectul B
- problema apare la alocare dinamică de memorie: A și B folosesc aceeași zonă de memorie pentru că pointerii arată în același loc
- destructorul lui MyClass eliberează aceeași zonă de memorie de două ori (distruge A și B)

constructorul de copiere

- aceeași problema
 - apel de funcție cu obiect ca parametru
 - apel de funcție cu obiect ca variabilă de întoarcere
 - în aceste cazuri un obiect temporar este creat, se copiază prin constructorul de copiere în obiectul temporar, și apoi se continuă
 - deci vor fi din nou două distrugeri de obiecte din clasa respectivă (una pentru parametru, una pentru obiectul temporar)

constructorul de copiere

Cazuri de utilizare:

Initializare explicita:

MyClass B = A;

MyClass B (A);

Apel de functie cu obiect ca parametru:

void f(MyClass X) {...}

Apel de functie cu obiect ca variabila de intoarcere:

MyClass f() {MyClass obiect; ... return obiect;}

MyClass x = f();

Copierea se poate face și prin operatorul = (*detalii mai târziu*).

putem redefini constructorul de copiere

```
classname (const classname &o) {  
    // body of constructor  
}
```

- o este obiectul din dreapta
- putem avea mai multi parametri (dar trebuie sa definim valori implicite pentru ei)
- & este apel prin referinta
- putem avea si atribuire (o1=o2;)
 - redefinim operatorii mai tarziu, putem redefini =
 - = diferit de initializare

```

#include <iostream>
#include <new>
#include <cstdlib>
using namespace std;

class array {
    int *p;
    int size;
public:
    array(int sz) {
        try {
            p = new int[sz];
        } catch (bad_alloc xa) {
            cout << "Allocation Failure\n";
            exit(EXIT_FAILURE);
        }
        size = sz;
    }
    ~array() { delete [] p; }

    // copy constructor
    array(const array &a);
    void put(int i, int j) {if(i>=0 && i<size) p[i] = j;}
    int get(int i) { return p[i];}
};

```

```

// Copy Constructor
array::array(const array &a) {
    int i;
    try {
        p = new int[a.size];
    } catch (bad_alloc xa) {
        cout << "Allocation Failure\n";
        exit(EXIT_FAILURE);
    }
    for(i=0; i<a.size; i++) p[i] = a.p[i];
}

```

```

int main()
{
    array num(10);
    int i;
    for(i=0; i<10; i++) num.put(i, i);
    for(i=9; i>=0; i--) cout << num.get(i);
    cout << "\n";

    // create another array and initialize with num
    array x(num); // invokes copy constructor
    for(i=0; i<10; i++) cout << x.get(i);
    return 0;
}

```

- Observatie: constructorul de copiere este folosit doar la initializari
- daca avem

```
array a(10);  
array b(10);  
b=a;
```

 - nu este initializare, este copiere de stare
 - este posibil sa trebuiasca redefinit si operatorul = (mai tarziu)

Perspective

Curs 4

- Static, clase locale
- Operatorul ::
- supraincarcarea functiilor in C++
- supraincarcarea operatorilor in C++



Programare orientată pe obiecte

- suport de curs -

Andrei Păun
Anca Dobrovăț

An universitar 2025 – 2026
Semestrul II
Seriile 13, 14, 15

Curs 4

Cuprins

- Recapitulare curs 3
- Membrii statici ai unei clase
- Clase locale (în funcții și în alte clase)
- Operatorul ::
- supraîncărcarea funcțiilor în C++
- supraîncărcarea operatorilor în C++

Variabile locale (reminder)

- **declarate în interiorul unei funcții sau al unui bloc de cod** ({ ... })
- domeniul de vizibilitate (scope): local, există **doar în interiorul aceluși bloc** și nu poate fi accesată din afara lui
- memoria este de obicei alocată pe **stack**
- **nu sunt inițializate automat**

```
1  #include <iostream>
2
3  using namespace std;
4
5  void f()
6  {
7      int x = 56;
8      x++;
9      cout<<&x<<" "<<x<<endl;
10 }
11 int main()
12 {
13     f(); f(); f();
14     return 0;
15 }
```

Se va afișa 57, 57, 57

- Adresa de memorie (pe PC-ul propriu): 0x8900fffaac – probabil o adresă pe stack (în general, adresele mari aparțin stack-ului)

Variabile locale statice (reminder)

- **declarate în interiorul unei funcții sau al unui bloc de cod** ({ ... }) dar folosind “**static**”
- domeniul de vizibilitate (scope): local, există **doar în interiorul aceluiași bloc** și nu poate fi accesată din afara lui
- **durată de viață statică** – există pe toată durata programului

```
1  #include <iostream>
2
3  using namespace std;
4
5  void f()
6  {
7      static int x = 56;
8      x++;
9      cout<<&x<<" "<<x<<endl;
10 }
11 int main()
12 {
13     f(); f(); f();
14     return 0;
15 }
```

Se va afișa 57, 58, 59

- Adresa de memorie (pe PC-ul propriu): 0x7ff785424010

Membrii statici ai unei clase

- date membre:
 - nestatice (distincte pentru fiecare obiect);
 - **static** (unice pentru toate obiectele clasei, există o singură copie pentru toate obiectele).
- cuvânt cheie “**static**”
- create, inițializate și accesate – independent de obiectele clasei.
- Variabile - alocarea și inițializarea – în afara clasei.
- Const – alocarea și inițializarea – în interiorul clasei

Membrii statici ai unei clase

- Variabile - alocarea și inițializarea – în afara clasei.
- Const – alocarea și inițializarea – în interiorul clasei

```
1  #include <iostream>
2
3  using namespace std;
4
5  class Test
6  {
7      static int x;
8  };
9  int Test::x;
10 int main()
11 {
12     cout<<sizeof(Test);
13 }
```

```
1  #include <iostream>
2
3  using namespace std;
4
5  class Test
6  {
7      const static int x = 0;
8  };
9
10 int main()
11 {
12     cout<<sizeof(Test);
13 }
```

Membrii statici ai unei clase

- create, inițializate și accesate – independent de obiectele clasei.

```
1  #include <iostream>
2
3  using namespace std;
4
5  class Test
6  {
7      static int x;
8  };
9  int Test::x;
10 int main()
11 {
12     cout<<sizeof(Test);
13 }
```

```
1  #include <iostream>
2
3  using namespace std;
4
5  class Test
6  {
7      const static int x = 0;
8  };
9
10 int main()
11 {
12     cout<<sizeof(Test);
13 }
```

```
1  #include <iostream>
2
3  using namespace std;
4
5  class Test
6  {
7      int x;
8  };
9
10 int main()
11 {
12     cout<<sizeof(Test);
13 }
```

- Se afișează 1 (dimensiunea “onorifică” a unei clase fără date membre nestatice).

- Se afișează 4 (dimensiunea unui int – dată membră nestatică x).

Membrii statici ai unei clase

- funcțiile statice:
 - efectuează operații asupra întregii clase;
 - nu au cuvântul cheie “this”;
 - se pot referi doar la membrii statici.
- referirea membrilor statici:
 - `clasa :: membru`;
 - `obiect.membru` (identic cu `nestatic`).

Membrii statici ai unei clase

O funcție membră statică:

- poate fi apelată înainte de crearea obiectelor;
- poate referi membri statici
- poate fi apelată și ca funcțiile membre nestatice prin `obiect.nume_funcție`
- nu poate referi, în mod direct, date sau funcții membre nestatice
- poate referi, în mod indirect, prin intermediul unui obiect local, date sau funcții membre nestatice
- nu au pointerul **this**

```
class Test
{
    static int x;
    int y;
public:
    void functie_nestatica() {}
    static void f1() {x = 1;}
    static void f2() {
        y = 2;
        functie_nestatica();
    }

    static void f3(Test o) {o.x = 3; o.y = 4;}
    static void f4() {Test o; o.x = 5; o.y = 6;}
    static void f5() {this->x = 7;}
};

int Test::x;
```

```
int main()
{
    Test::f1();
    Test ob;
    ob.f1();
    Test::f2();
    Test::f3(ob);
    Test::f4();
    Test::f5();
    return 0;
}
```


Obiecte statice

- Precedate de cuvântul **static**
- Se inițializează o singură dată și se utilizează dacă se dorește păstrarea valorii la apelul multiplu al funcției
- 2 tipuri: - obiecte **locale** statice și obiecte **globale** statice
- Un obiect static local este distrus la finalul programului, dar, este vizibil doar în scope-ul în care a fost definit.
- Obiecte globale statice sunt ultimele distruse

Obiecte statice locale

Un obiect static local este distrus la finalul programului.

```
#include <iostream>

using namespace std;

class Test{ int a;
public:
    Test(int x = 0):a(x){cout<<"C " <<a<<endl;}
    ~Test(){cout<<"D " <<a<<endl;}
};

void f(int i){ static Test obiect(i); }

int main(){
    cout<<"start"<<'\n';
    f(1);
    cout<<"end"<<'\n';
}
```

```
start
C 1
end
D 1
```

Obiecte statice locale

Ce se întâmplă dacă se apelează funcția de mai multe ori?

```
#include <iostream>

using namespace std;

class Test{ int a;
public:
    Test(int x = 0):a(x){cout<<"C " <<a<<endl;}
    ~Test() {cout<<"D " <<a<<endl;}
};

void f(int i){ static Test obiect(i); cout<<&obiect<<endl;}

int main()
{
    cout<<"start"<<'\n';
    f(1);
    f(2);
    f(3);
    cout<<"end"<<'\n';
}
```

```
start
C 1
0x407034
0x407034
0x407034
end
D 1
```

De câte ori este creat și distrus obiectul static local? Răspuns: o singură dată

Obiecte statice locale (în funcții membru)

```
class Test
{
    int x;
public:
    Test(int x=0): x(x) {cout<<x<<" C\n";}
    ~Test() {cout<<x<<" D\n";}
    void f()
    {
        static Test obiect;
    }
};

int main()
{
    Test obiect1(1);
    cout<<"start"<<"\n";
    obiect1.f();
    cout<<"end"<<"\n";
}
```

```
1 C
start
0 C
end
1 D
0 D
```

De câte ori este creat și distrus obiectul static local? Răspuns: o singură dată

Obiecte statice globale (sunt distruse ultimele)

```
class Test
{
    int x;
public:
    Test(int x=0): x(x) {cout<<x<<" C\n";}
    ~Test() {cout<<x<<" D\n";}
    void f() { static Test obiect; cout<<&obiect<<endl; }
};

static Test A(2), B(3);

int main()
{
    Test obiect1(1);
    cout<<"start"<<'\\n';
    obiect1.f();
    obiect1.f();
    cout<<"end"<<'\\n';
}
```

```
2 C
3 C
1 C
start
0 C
0x4030c0
0x4030c0
end
1 D
0 D
3 D
2 D
```

Operatorul de rezolutie de scop ::

```
int i; // global i
```

```
void f()
```

```
{
```

```
    int i; // local i
```

```
    i = 10; // uses local i.
```

```
}
```

```
int i; // global i
```

```
void f()
```

```
{
```

```
    int i = 7; // local i
```

```
    ::i = 10; // now refers to global i
```

```
    Cout<<::i;
```

```
}
```

Clase locale

- putem defini **clase în clase** sau **în funcții**
- **class** este o declarație, deci definește un scop
- operatorul de rezoluție de scop ajută în aceste cazuri
- rar utilizate clase în clase

Clase locale (în funcții)

```
void f();  
int main() {  
    f(); /// myclass not known here  
    return 0; }  
void f() {  
    class myclass  
    {  
        int i;  
    public:  
        void put_i(int n) { i=n; }  
        int get_i() { return i; }  
    } ob;  
    ob.put_i(10);  
    cout << ob.get_i();  
}
```

Funcțiile membre:

- trebuie definite în clasă
- nu pot accesa variabilele locale ale funcției
- pot accesa variabilele locale statice
- pot accesa variabilele globale

Clasele locale:

- Nu pot avea variabile statice, dar pot folosi funcții statice
- Pot fi puse în legătură cu alte clase locale din aceeași funcție

! Mai multe exemple în fisierul cpp încărcat în Moodle

Clase locale (în alte clase) – imbricate (“nested”)

```
int global = 300;
class Out{
public:
    class In{
        static int A;
        int x;
    public:
        void set(int i);
        int get(){return A + x + global;}
    }ob2;
};

int Out::In::A = 90;
void Out::In::set(int i){ x = i;}

int main()
{
    Out ob;
    ob.ob2.set(100);
    cout<<ob.ob2.get();
    return 0;
}
```

- Funcțiile membre ale clasei locale trebuie declarate și definite în clasa interioară sau în afara clasei exterioare
- Pot avea date membre statice
- Pot accesa variabile globale

!atenție “public”: doar dpv didactic/demonstrativ - pt a avea acces direct la ob2 din main

Funcții care întorc obiecte

- o funcție poate întoarce obiecte
- un obiect temporar este creat automat pentru a ține informațiile din obiectul de întors
- acesta este obiectul care este întors
- după ce valoarea a fost întoarsă, acest obiect este distrus
- probleme cu memoria dinamică: soluție
polimorfism pe = și pe constructorul de copiere

// Returning objects from a function.

```
#include <iostream>
```

```
using namespace std;
```

```
class myclass
```

```
{
```

```
    int i;
```

```
public:
```

```
myclass(){ i=0;}
```

```
    void set_i(int n) { i=n; }
```

```
    int get_i() { return i; }
```

```
};
```

```
myclass f(); // return object of type myclass
```

```
int main()
```

```
{
```

```
    myclass o;
```

```
    o = f();
```

```
    cout << o.get_i() << "\n";
```

```
    return 0;
```

```
}
```

```
myclass f()
```

```
{
```

```
    myclass x;
```

```
    x.set_i(1);
```

```
    return x;
```

```
}
```

copierea prin operatorul =

- este posibil să dăm valoarea unui obiect altui obiect
- trebuie să fie de același tip (aceeași clasă)

Supraîncărcarea operatorilor în C++

- majoritatea operatorilor pot fi supraîncărcați
- similar ca la funcții
- una din proprietățile C++ care îi conferă putere
- s-a făcut supraîncărcarea operatorilor si pentru operații de I/O (<<,>>)
- supraîncărcarea se face definind o funcție operator: membru al clasei sau nu

Restricții

- nu se poate redefini și precedența operatorilor
- nu se poate redefini numărul de operanzi
 - rezonabil pentru ca redefinim pentru lizibilitate
 - putem ignora un operand dacă vrem
- nu putem avea valori implicite; excepție pentru ()
- **nu putem face overload pe**
 - . (acces de membru)**
 - :: (rezoluție de scop)**
 - .*(acces membru prin pointer)**
 - ?: (ternar)**
 - sizeof()**
 - typeid**
 - alignof (operator pentru alinierea tipurilor; din C++11)**
- Se recomandă operațiuni apropiate de înțelesul operatorilor respectivi

Funcții operator membri ai clasei

```
ret-type class-name::operator#(arg-list)  
{  
    // operations  
}
```

- # este operatorul supraîncărcat (+ - * / ++ -- = , etc.)
- de obicei *ret-type* este tipul clasei, dar avem flexibilitate
- pentru operatori unari *arg-list* este vidă
- pentru operatori binari: *arg-list* conține un element

```

class loc {
    int longitude, latitude;
public:
    loc() {}
    loc(int lg, int lt) {
        longitude = lg;
        latitude = lt; }
    void show() {
        cout << longitude << " ";
        cout << latitude << "\n";
    }
    loc operator+(loc op2);
};

```

// Overload + for loc.

```

loc loc::operator+(loc op2)
{
    loc temp;
    temp.longitude = op2.longitude + longitude;
    temp.latitude = op2.latitude + latitude;
    return temp;
}

```

```

int main(){
    loc ob1(10, 20), ob2( 5, 30);
    ob1.show(); // displays 10 20
    ob2.show(); // displays 5 30
    ob1 = ob1 + ob2;
    ob1.show(); // displays 15 50
    return 0;
}

```

- un singur argument pentru că avem **this**
- longitude==this->longitude
- obiectul din stânga face apelul la funcția operator
 - ob1a chemat operatorul + redefinit în clasa lui ob1

- **daca întoarcem același tip de date în operator putem avea expresii**
- **daca întorceam alt tip nu puteam face**
$$\text{ob1} = \text{ob1} + \text{ob2};$$
- **putem avea si**
`(ob1+ob2).show(); // displays outcome of ob1+ob2`
- **pentru ca funcția show() este definita in clasa lui ob1**
- **se generează un obiect temporar**
 - (constructor de copiere)

```

#include <iostream>
using namespace std;
class loc { int longitude, latitude;
public:
    loc() {} // needed to construct temporaries
    loc(int lg, int lt) { longitude = lg; latitude = lt; }
    void show() { cout<<longitude<<" "<<latitude<<"\n"; }
    loc operator+(loc op2);
    loc operator-(loc op2);
    loc operator=(loc op2);
    loc operator++();
};
// Overload + for loc.
loc loc::operator+(loc op2){ loc temp;
    temp.longitude = op2.longitude + longitude;
    temp.latitude = op2.latitude + latitude;
    return temp;}

loc loc::operator-(loc op2){ loc temp;
    temp.longitude = longitude - op2.longitude;
    temp.latitude = latitude - op2.latitude;
    return temp;}

```

```

// Overload assignment for loc.
loc loc::operator=(loc op2){
    longitude = op2.longitude;
    latitude = op2.latitude;
    return *this; } // object that generated call
// Overload prefix ++ for loc.
loc loc::operator++(){
    longitude++;
    latitude++;
    return *this;}

int main(){
    loc ob1(10, 20), ob2( 5, 30), ob3(90, 90);
    ob1.show(); ob2.show();
    ++ob1; ob1.show(); // displays 11 21
    ob2 = ++ob1; ob1.show(); // displays 12 22
    ob2.show(); // displays 12 22
    ob1 = ob2 = ob3; // multiple assignment
    ob1.show(); // displays 90 90
    ob2.show(); // displays 90 90
    return 0;}

```

- apelul la funcția operator se face din obiectul din stânga (pentru operatori binari)
 - din aceasta cauza pentru – avem funcția definita asa
- operatorul = face copiere pe variabilele de instanță, întoarce *this
- se pot face atribuiri multiple (dreapta spre stânga)

Formele prefix și postfix

- am văzut prefix, pentru postfix: definim un parametru int “dummy”

```
// Prefix increment  
type operator++( ) {  
    // body of prefix operator  
}
```

```
// Postfix increment  
type operator++( int x) {  
    // body of postfix operator  
}
```

Supraîncărcarea +=, *=, etc.

```
loc loc::operator+=(loc op2)
{
    longitude = op2.longitude + longitude;
    latitude = op2.latitude + latitude;
    return *this;
}
```

- Este posibil să facem o decuplare completă între înțelesul inițial al operatorului
 - exemplu: << >>
- moștenire: operatorii (mai puțin =) sunt moșteniți de clasă derivată
- clasă derivată poate să își redefinească operatorii

Supraîncărcarea operatorilor ca funcții prieten

- operatorii pot fi definiți și ca funcție nemembră a clasei
- o facem funcție prietenă pentru a putea accesa rapid câmpurile protejate
- **nu avem pointerul “this”**
- **deci vom avea nevoie de toți operanzii** ca parametri pentru funcția operator
- primul parametru este operandul din stânga, al doilea parametru este operandul din dreapta

```

#include <iostream>
using namespace std;
class loc { int longitude, latitude;
public:
    loc() {} // needed to construct temporaries
    loc(int lg, int lt) { longitude = lg; latitude = lt; }
    void show() { cout<<longitude<<" "<<latitude<<"\n";}
    friend loc operator+(loc op1, loc op2); // friend
    loc operator-(loc op2);
    loc operator=(loc op2);
    loc operator++();
};

```

// Now, + is overloaded using friend function.

```

    loc operator+(loc op1, loc op2){
        loc temp;
        temp.longitude = op1.longitude + op2.longitude;
        temp.latitude = op1.latitude + op2.latitude;
        return temp;
    }

```

```

loc loc::operator-(loc op2){ loc temp;
// notice order of operands
    temp.longitude = longitude - op2.longitude;
    temp.latitude = latitude - op2.latitude;
    return temp;}

```

// Overload asignment for loc.

```

loc loc::operator=(loc op2){
    longitude = op2.longitude;
    latitude = op2.latitude;
    return *this; }// object that generated call

```

```

loc loc::operator++(){
    longitude++;
    latitude++;
    return *this;}

```

```

int main(){
    loc ob1(10, 20), ob2( 5, 30);
    ob1 = ob1 + ob2;
    ob1.show();
    return 0;}

```


Restricții pentru operatorii definiți ca prieten

- **nu se pot supraîncărca utilizând funcții prieten:**
 - Operatorul de atribuire =
 - Operatorul apel de funcție ()
 - Operatorul de indexare []
 - Operatorul de acces prin pointer la funcții membre ->
- pentru ++ sau -- trebuie să folosim referințe

Funcții prieten pentru operatori unari

- pentru ++, -- folosim referința pentru a transmite operandul
 - pentru că trebuie să se modifice și nu avem pointerul this
 - apel prin valoare: primim o copie a obiectului și nu putem modifica operandul (ci doar copia)

```

#include <iostream>
using namespace std;
class loc { int longitude, latitude;
public:
    loc() {} // needed to construct temporaries
    loc(int lg, int lt) { longitude = lg; latitude = lt; }
    void show() { cout<<longitude<<" "<<latitude<<"\n";}
    loc operator=(loc op2);
    friend loc operator++(loc& op);
    friend loc operator--(loc& op);
};
// Overload assignment for loc.
loc loc::operator=(loc op2){
    longitude = op2.longitude;
    latitude = op2.latitude;
    return *this; }// object that generated call

// Now a friend, use a reference parameter.
    loc operator++(loc& op) {
        op.longitude++;
        op.latitude++;
    return op;
}

```

```

// Make – a friend. Use reference
    loc operator--(loc& op) {
        op.longitude--;
        op.latitude--;
    return op;
}

int main(){
    loc ob1(10, 20), ob2;
    ob1.show();
    ++ob1;
    ob1.show(); // displays 11 21
    ob2 = ++ob1;
    ob2.show(); // displays 12 22
    --ob2;
    ob2.show(); // displays 11 21
    return 0;}

```

pentru varianta postfix ++ --

- la fel ca la supraîncărcarea operatorilor prin funcții membru ale clasei: parametru int

```
// friend, postfix version of ++  
friend loc operator++(loc &op, int x);
```

Diferențe supraîncărcarea prin membri sau prieteni

- de multe ori nu avem diferențe,
 - atunci e indicat să folosim funcții membru
 - ca funcții membru: operatori unari, cei compuși ($+=$, $*=$ etc)
- uneori avem însă diferențe: poziția operanzilor
 - pentru funcții membru operandul din stânga apelează funcția operator supraîncărcată
 - dacă vrem să scriem expresie: `100+ob`; probleme la compilare=> funcții prieten
- Interesant: depinde de operator și de situație (sursa: <https://stackoverflow.com/questions/4421706>)

“Spaceship operator” $< = >$ in C++20

- Se supraîncărca toți operatorii de comparație “în același timp”

```
#include <compare>

class X {
    // defines ==, !=, <, >, <=, >=, <=>
    friend auto operator<=>(const X&, const X&) = default;
};
```

(sursa: <https://stackoverflow.com/questions/4421706>)

- în aceste cazuri trebuie să definim două funcții de supraîncărcare:
 - $\text{int} + \text{tipClasa}$
 - $\text{tipClasa} + \text{int}$

```

#include <iostream>
using namespace std;
class loc { int longitude, latitude;
public:
    loc() {} // needed to construct temporaries
    loc(int lg, int lt) { longitude = lg; latitude = lt; }
    void show() { cout<<longitude<<" "<<latitude<<"\n";}
    loc operator=(loc op2);
    friend loc operator+(loc op1, int op2);
    friend loc operator+(int op1, loc op2);
};
// + is overloaded for loc + int.
loc operator+(loc op1, int op2){
    loc temp;
    temp.longitude = op1.longitude + op2;
    temp.latitude =op1.latitude + op2;
    return temp;}

```

```

// + is overloaded for int + loc.
loc operator+(int op1, loc op2){
    loc temp;
    temp.longitude = op1 + op2.longitude;
    temp.latitude =op1 + op2.latitude;
    return temp;}

```

```

int main(){
    loc ob1(10, 20), ob2(5, 30) , ob3(7, 14);
    ob1.show();
    ob2.show();
    ob3.show();
    ob1 = ob2 + 10; // both of these
    ob3 = 10 + ob2; // are valid
    ob1.show();
    ob3.show();

    return 0;}

```


supraîncărcarea new și delete

- supraîncărcare op. de folosire memorie în mod dinamic pentru cazuri speciale

```
// Allocate an object.
```

```
void *operator new(size_t size){
```

```
    /* Perform allocation. Throw bad_alloc on  
    failure. Constructor called automatically. */
```

```
    return pointer_to_memory;
```

```
}
```

```
// Delete an object.
```

```
void operator delete(void *p){
```

```
    /* Free memory pointed to by p. Destructor called  
    automatically. */
```

```
}
```

- size_t: predefinit
- pentru new: constructorul este chemat automat
- pentru delete: destructorul este chemat automat
- supraîncărcare la nivel de clasă sau globală

```

#include <iostream>
#include <cstdlib>
#include <new>
using namespace std;
class loc {    int longitude, latitude;
public:
    loc() {}
    loc(int lg, int lt)
        {longitude = lg; latitude = lt;}
    void show() { cout << longitude << " "
cout << latitude << "\n";}
    void *operator new(size_t size);
    void operator delete(void *p);
};
// new overloaded relative to loc.
void *loc::operator new(size_t size){
    void *p;
    cout << "In overloaded new.\n";
    p = malloc(size);
    if(!p) { bad_alloc ba; throw ba; }
    return p;}

```

```

// delete overloaded relative to loc.
void loc::operator delete(void *p){
    cout << "In overloaded delete.\n";
    free(p);
}
int main(){
    loc *p1, *p2;
    try {p1 = new loc (10, 20); }
    catch (bad_alloc xa)
        { cout << "Allocation error for p1.\n"; return 1;}
    try {p2 = new loc (-10, -20); }
    catch (bad_alloc xa)
        { cout << "Allocation error for p2.\n"; return 1;}
    p1->show();
    p2->show();
    delete p1; delete p2;
    return 0; }

```

In overlo
 In overlo
 10 20
 -10 -20
 In overlo
 In overlo

- dacă new sau delete sunt folosiți pentru alt tip de date în program, versiunile originale sunt folosite
- se poate face overload pe new și delete la nivel global
 - se declara în afara oricărei clase
 - pentru new/delete definiți și global și în clasă, cel din clasă e folosit pentru elemente de tipul clasei, și în rest e folosit cel redefinit global

```

#include <iostream>
#include <cstdlib>
#include <new>
using namespace std;
class loc { int longitude, latitude;
public:
    loc() {}
    loc(int lg, int lt)
        {longitude = lg;latitude = lt;}
    void show() {cout << longitude << " ";
        cout << latitude << "\n";}
};

```

// Global new

```

void *operator new(size_t size) {
    void *p;
    p = malloc(size);
    if(!p) { bad_alloc ba; throw ba; }
return p;
}

```

// Global delete

```

void operator delete(void *p) { free(p); }
int main(){
    loc *p1, *p2;
    float *f;
    try {p1 = new loc (10, 20); }
    catch (bad_alloc xa) {
        cout << "Allocation error for p1.\n";
        return 1; }
    try {p2 = new loc (-10, -20); }
    catch (bad_alloc xa) {
        cout << "Allocation error for p2.\n";
        return 1; }
    try {
        f = new float; // uses overloaded new, too }
    catch (bad_alloc xa) {
        cout << "Allocation error for f.\n";
        return 1; }
    *f = 10.10F;
    cout << *f << "\n";
    p1->show();
    p2->show();
    delete p1; delete p2; delete f;
return 0; }

```

new și delete pentru array-uri

- facem overload de două ori

```
// Allocate an array of objects.  
void *operator new[](size_t size) {  
    /* Perform allocation. Throw bad_alloc on failure.  
    Constructor for each element called automatically. */  
    return pointer_to_memory;  
}  
  
// Delete an array of objects.  
void operator delete[](void *p) {  
    /* Free memory pointed to by p. Destructors for each  
    element called automatically. */  
}
```

supraîncărcarea []

```
#include <iostream>
using namespace std;
class atype { int a[3];
public:
    atype(int i, int j, int k) { a[0] = i; a[1] = j; a[2] = k; }
    int operator[](int i) { return a[i]; }
};
int main() {
    atype ob(1, 2, 3);
    cout << ob[1]; // displays 2
    return 0;
}
```

- operatorul [] poate fi folosit și la stânga unei atribuirii (obiectul întors este atunci referință)

supraîncărcarea []

```
#include <iostream>
using namespace std;
class atype { int a[3];
public:
    atype(int i, int j, int k) { a[0] = i; a[1] = j; a[2] = k; }
    int &operator[](int i) { return a[i]; }
};
int main() {
    atype ob(1, 2, 3);
    cout << ob[1]; // displays 2
    cout << " ";
    ob[1] = 25; // [] on left of =
    cout << ob[1]; // now displays 25
    return 0; }
```

- putem în acest fel verifica array-urile
- exemplul următor

// A safe array example.

```
#include <iostream>
```

```
#include <cstdlib>
```

```
using namespace std;
```

```
class atype { int a[3];
```

```
public:
```

```
    atype(int i, int j, int k) {a[0] = i; a[1] = j; a[2] = k;}
```

```
    int &operator[](int i);
```

```
};
```

// Provide range checking for atype.

```
int &atype::operator[](int i)
```

```
{
```

```
    if(i<0 || i> 2) { cout << "Boundary Error\n"; exit(1); }
```

```
    return a[i];
```

```
}
```

```
int main() {
```

```
    atype ob(1, 2, 3);
```

```
    cout << ob[1]; // displays 2
```

```
    cout << " ";
```

```
    ob[1] = 25; // [] appears on left
```

```
    cout << ob[1]; // displays 25
```

```
    ob[3] = 44;
```

```
        // generates runtime error, 3 out-of-range
```

```
    return 0; }
```


supraîncărcarea ()

- nu creăm un nou fel de a chema funcții
- definim un mod de a chema funcții cu număr arbitrar de parametri

double operator()(int a, float f, char *s);

O(10, 23.34, "hi");

echivalent cu O.operator()(10, 23.34, "hi");

supraîncărcarea ()

```
#include <iostream>
using namespace std;
class loc { int longitude, latitude;
public:
    loc() {}
    loc(int lg, int lt) {longitude = lg;
latitude = lt;}
    void show() {cout << longitude << " ";
cout << latitude << "\n";}
    loc operator+(loc op2);
    loc operator()(int i, int j);
};
// Overload ( ) for loc.
loc loc::operator()(int i, int j) {
    longitude = i; latitude = j;
return *this;
}
```

```
// Overload + for loc.
loc loc::operator+(loc op2) {
    loc temp;
    temp.longitude = op2.longitude + longitude;
    temp.latitude = op2.latitude + latitude; return
temp;
}
int main() { loc ob1(10, 20), ob2(1, 1);
ob1.show();
ob1(7, 8); // can be executed by itself ob1.show();
ob1 = ob2 + ob1(10, 10); // can be used in
expressions
ob1.show();
return 0; }
```

10 20
7 8
11 11

overload pe ->

- operator unar
- obiect->element
 - obiect generează apelul
 - element trebuie să fie accesibil
 - întoarce un pointer către un obiect din clasă

```
#include <iostream>
using namespace std;
class myclass {
    public:
    int i;
    myclass *operator->() {return this;}
};
```

```
int main() {
    myclass ob; ob->i = 10; // same as ob.i
    cout << ob.i << " " << ob->i;
    return 0;
}
```

supraîncărcarea operatorului ,

- operator binar
- ar trebui ignorate toate valorile mai puțin a celui mai din dreapta operand

```

#include <iostream>
using namespace std;
class loc { int longitude, latitude;
public:
    loc() {}
    loc(int lg, int lt) {longitude = lg; latitude = lt;
    void show() {cout << longitude << " ";
cout << latitude << "\n";}
    loc operator+(loc op2);
    loc operator,(loc op2);
};
// overload comma for loc
loc loc::operator,(loc op2){
    loc temp;
    temp.longitude = op2.longitude;
    temp.latitude = op2.latitude;
    cout << op2.longitude << " ";
    cout << op2.latitude << "\n";
return temp;
}

// Overload + for loc
loc loc::operator+(loc op2) {
    loc temp;
    temp.longitude = op2.longitude + longitude;
    temp.latitude = op2.latitude + latitude;
return temp; }

int main() {
loc ob1(10, 20), ob2( 5, 30), ob3(1, 1); ob1.show();
ob2.show(); ob3.show();
cout << "\n";
ob1 = (ob1, ob2+ob2, ob3);
ob1.show(); // displays 1 1, the value of ob3
return 0;
}

```

```

10 20
5 30
1 1
10 60
1 1
1 1

```

Perspective

Curs 5

Proiectarea descendenta a claselor. Moștenirea în C++.

- 1 Controlul accesului la clasa de bază.
- 2 Constructori, destructori și moștenire.
- 3 Redefinirea membrilor unei clase de bază într-o clasă derivată.
- 4 Declarații de acces.



Programare orientată pe obiecte

- suport de curs -

Anca Dobrovăț
Andrei Păun

An universitar 2025 – 2026

Semestrul I

Seriile 13, 14 și 15

Curs 5

Cuprins

Proiectarea descendentă a claselor. Moștenirea în C++.

- Controlul accesului la clasa de bază.
- Constructori, destructori și moștenire.
- Redefinirea membrilor unei clase de bază într-o clasă derivată.
- Declarații de acces.

Obs: în acest curs, o parte din exemple vor fi luate, în principal, din cartea lui B. Eckel - Thinking in C++.

Moștenirea în C++

- important în C++ - reutilizare de cod;
- reutilizare de cod prin creare de noi clase (nu se dorește crearea de clase de la zero);
- 2 modalități (compunere și moștenire);
- “compunere” - noua clasă “este compusă” din obiecte reprezentând instanțe ale claselor deja create;
- “moștenire” - se creează un nou tip al unei clase deja existente.

Moștenirea în C++

Exemplu: compunere

```
class Proiector { float scara;  
public:  
    Proiector() { scara = 1; }  
    void set(int sc) { scara = sc; }  
};  
class Sala { int numar; Proiector proiector; // compunere  
public:  
    Sala() { numar = 118; }  
    const Proiector& getProiector() { return proiector; }  
    void f(int nr) { numar = nr; }  
};  
int main() {  
    Sala sala;  
    sala.f(47);  
    sala.getProiector().set(37); // Access the embedded object  
}
```

Moștenirea în C++

C++ permite moștenirea, ceea ce înseamnă că putem deriva o clasă din altă clasă de bază sau din mai multe clase.

Prin derivare se obțin clase noi, numite clase derivate, care moștenesc proprietățile unei clase deja definite, numită clasă de bază.

Clasele derivate conțin toți membrii clasei de bază, la care se adaugă noi membri, date și funcții membre.

Dintr-o clasă de bază se poate deriva o clasă care, la rândul său, să servească drept clasă de bază pentru derivarea altora. Prin această succesiune se obține o **ierarhie de clase**.

Se pot defini clase derivate care au la bază mai multe clase, înglobând proprietățile tuturor claselor de bază, procedeu ce poartă denumirea de **moștenire multiplă**.

Moștenirea în C++

C++ permite moștenirea, ceea ce înseamnă că putem deriva o clasă din altă clasă de bază sau din mai multe clase.

Sintaxa:

```
class Clasa_Derivata : [modifier de acces] Clasa_de_Baza { .... } ;
```

sau

```
class Clasa_Derivata : [modifier de acces] Clasa_de_Baza1, [modifier de  
acces] Clasa_de_Baza2, [modifier de acces] Clasa_de_Baza3 .....
```

Clasa de bază se mai numește clasă părinte sau superclasă, iar clasa derivată se mai numește subclasă sau clasă copil.

Moștenirea multiplă: foarte utilă în unele situații, însă inclusă din motive istorice. Majoritatea limbajelor apărute ulterior au eliminat-o din cauza complexității.

Moștenirea în C++

Exemplu: moștenire

```
class Dispozitiv {  bool in_use;
public:
    Dispozitiv() { in_use = false; }
    void start() { in_use = true; }
    bool status() const { return in_use; }
    void stop() { in_use = false; }
};

class Proiector : public Dispozitiv {
    int scara;
public:
    Proiector() { scara = 1; }
    void foloseste() {
        start(); // Different name call
        std::cout << "proiecteaza lectie\n";
    }
    bool status() {
        return Dispozitiv::status() &&
               scara > 0; // Same-name function call
    }
};

int main() {
    std::cout << sizeof(Dispozitiv) << ' ' << sizeof(Proiector);

    Proiector pr;
    pr.start(); // funcție din interfața clasei Dispozitiv
    pr.stop();
    pr.foloseste();
    std::cout << pr.status() ? "on" : "off"; // Redefined
    functions hide base versions:
}
```

Moștenirea în C++

Moștenire vs. Compunere

Moștenirea este asemănătoare cu procesul de includere a obiectelor în obiecte (procedeu ce poartă denumirea de compunere), dar există câteva elemente caracteristice moștenirii:

- implementarea poate fi comună mai multor clase;
- clasele pot fi extinse, fără a recompila codul care depinde de clasa de bază inițială;
- funcțiile ce utilizează obiecte din clasa de bază pot utiliza și obiecte din clasele derivate din această clasă.

Moștenirea în C++

Inițializare de obiecte

Foarte important în C++: garantarea inițializării corecte => trebuie să fie asigurată și la compunere, și la moștenire.

La crearea unui obiect, compilatorul trebuie să garanteze apelul TUTUROR sub-obiectelor.

Problemă: - cazul sub-obiectelor care nu au constructori implicați sau schimbarea valorii unui argument default în constructor.

De ce? - constructorul noii clase nu are permisiunea să acceseze datele **private** ale sub-obiectelor, deci nu le poate inițializa direct.

Rezolvare: - o sintaxă specială: *lista de inițializare pentru constructori*.

Moștenirea în C++

Exemplu: lista de inițializare pentru constructori

```
class Dispozitiv {  
    int nr_butoane;  
    public:  
        Dispozitiv(int nr) {nr_butoane = nr;}  
};
```

```
class Proiector: public Dispozitiv {  
    public:  
        Proiector(int);  
};
```

```
Proiector::Proiector (int nr) : Dispozitiv (nr) { ... }
```

Moștenirea în C++

Exemplu 2: lista de inițializare pentru constructori

```
class Telecomanda {  
    int nr_baterii;  
    public: Telecomanda(int nr) {nr_baterii = nr;}  
};
```

```
class Dispozitiv {  
    int nr_butoane;  
    public: Dispozitiv(int nr) {nr_butoane = nr;}  
};
```

```
class Proiector: public Dispozitiv {  
    Telecomanda telecomanda; // obiect telecomanda = subobiect în cadrul clasei Proiector  
    Public: Proiector(int, int);  
};
```

```
Proiector::Proiector(int i, int j) :
```

```
    Dispozitiv (i), telecomanda (j) { ... }
```

Moștenirea în C++

Chestiune specială: “pseudo - constructori” pentru tipuri primitive

- membrii de tipuri primitive (int, char, double) nu au constructori;
- soluție: C++ permite tratarea tipurilor predefinite asemănător unei clase cu o singură dată membră și care are un constructor parametrizat.
- Obs: tipurile din STL sunt clase, nu tipuri primitive:
 - Exemple: std::string, std::vector

Moștenirea în C++

```
class Proiector {  
    int butoane; // sau int butoane{7}; // sau int butoane = 7;  
    float scara;  
    char categorie;  
public:  
    Proiector() : butoane(7), scara(1.4), categorie('a') {}  
};  
  
int main() {  
    Proiector pr;  
    int i(100); // Applied to ordinary definition  
    int* ip = new int(47); delete ip;  
}
```

Moștenirea în C++

Exemplu: compunere și moștenire

```
class A { int i;  
public:  
    A(int ii) : i(ii) {}  
    ~A() {}  
    void f() const {}  
};
```

```
class B { int i;  
public:  
    B(int ii) : i(ii) {}  
    ~B() {}  
    void f() const {}  
};
```

```
class C : public B {  
    A a;  
public:  
    C(int ii) : B(ii), a(ii) {}  
    ~C() {} // Calls ~A() and ~B()  
    void f() const  
        { // Redefinition  
            a.f();  
            B::f();  
        }  
};  
int main() {  
    C c(47);  
}
```

Moștenirea în C++

Constructorii clasei derivate

Pentru crearea unui obiect al unei clase derivate, **se creează inițial un obiect al clasei de bază** prin apelul constructorului acesteia, apoi se adaugă elementele specifice clasei derivate prin apelul constructorului clasei derivate.

Declarația obiectului derivat trebuie să conțină valorile de inițializare, **atât pentru elementele specifice, cât și pentru obiectul clasei de bază.**

Această specificare se atașează la antetul funcției constructor a clasei derivate.

În situația în care clasele de bază au definit **constructor implicit** sau **constructor cu parametri implicați**, nu se impune specificarea parametrilor care se transferă către obiectul clasei de bază.

Moștenirea în C++

Constructorii clasei derivate

Constructorul parametrizat

```
class Forma {  
    protected:  
        int h;  
    public:  
        Forma(int a = 0) : h(a) {}  
};
```

```
class Cerc: public Forma {  
    protected:  
        float raza;  
    public:  
        Cerc(int h=0, float r=0) : Forma(h), raza(r) {}  
};
```

Moștenirea în C++

Constructorii clasei derivate

Constructorul de copiere

Se pot distinge mai multe situații.

1) Dacă ambele clase, atât clasa derivată, cât și clasa de bază, nu au definit constructor de copiere, se apelează constructorul implicit creat de compilator. Copierea se face membru cu membru.

2) Dacă clasa de bază are constructorul de copiere definit, dar clasa derivată nu, pentru clasa derivată compilatorul creează un constructor implicit care apelează constructorul de copiere al clasei de bază. (poate fi considerată un caz particular al primei situații, deoarece și partea de bază poate fi privită ca un fel de membru, iar la copiere se apelează cc pentru fiecare membru).

3) Dacă se definește constructor de copiere pentru clasa derivată, acestuia îi revine în totalitate sarcina transferării valorilor corespunzătoare membrilor ce aparțin clasei de bază.

Moștenirea în C++

Constructorii clasei derivate

Constructorul de copiere

```
class Forma {  
protected:    int h;  
public:  
    Forma(const Forma& ob), h(ob.h) { }  
};
```

```
class Cerc: public Forma {  
protected:  
    float raza;  
public:  
    Cerc(const Cerc&ob):Forma(ob), raza(ob.raza) { }  
};
```

Moștenirea în C++

Ordinea apelării constructorilor și destructorilor

- constructorii sunt apelați în ordinea definirii obiectelor ca membri ai clasei și în ordinea moștenirii:
- la fiecare nivel se apelează:
 - **întâi constructorul de la moștenire,**
 - apoi **constructorii din obiectele membru** în clasa respectivă (care sunt apelați în ordinea definirii)
 - și la final se merge pe următorul nivel în ordinea moștenirii;
- destructorii sunt executați în ordinea inversă a constructorilor

Moștenirea în C++

Ordinea apelării constructorilor și destructorilor

```
class A{  
public:  
    A(){cout<<"A ";}  
    ~A(){cout<<"~A ";}  
};
```

```
class C{  
public:  
    C(){cout<<"C ";}  
    ~C(){cout<<"~C ";}  
};
```

Ordine: C B A D ~D ~A ~B ~C

```
class B{  
    C ob;  
public:  
    B(){cout<<"B ";}  
    ~B(){cout<<"~B ";}  
};
```

```
class D: public B{  
    A ob;  
public:  
    D(){cout<<"D ";}  
    ~D(){cout<<"~D ";}  
};
```

```
int main() {  
    D s;  
}
```

Moștenirea în C++

- *Ordinea chemării constructorilor și destructorilor*

```
#define CLASS(ID) class ID { \
public: \
    ID(int)
        { cout << #ID " constructor\n"; } \
    ~ID()
        { cout << #ID " destructor\n"; } \
};
```

```
CLASS(Base1);
CLASS(Member1);
CLASS(Member2);
CLASS(Member3);
CLASS(Member4);
```

```
class Derived1 : public Base1 {
    Member1 m1;
    Member2 m2;
public:
    Derived1(int) : m2(1), m1(2), Base1(3)
        { cout << "Derived1 constructor\n"; }
    ~Derived1() { cout << "Derived1 destructor\n"; }
};
```

```
class Derived2 : public Derived1 {
    Member3 m3;
    Member4 m4;
public:
    Derived2() : m3(1), Derived1(2), m4(3)
        { cout << "Derived2 constructor\n"; }
    ~Derived2() { cout << "Derived2 destructor\n"; }
};

int main() { Derived2 d2;}
```

Moștenirea în C++

- *Ordinea chemării constructorilor și destructorilor*

```
#define CLASS(ID) class ID { \  
public: \  
    ID(int)  
        { cout << #ID " constructor\n"; } \  
    ~ID()  
        { cout << #ID " destructor\n"; } \  
};
```

```
CLASS(Base1);  
CLASS(Member1);  
CLASS(Member2);  
CLASS(Member3);  
CLASS(Member4);
```

Se va afișa:

Base1 constructor
Member1 constructor
Member2 constructor
Derived1 constructor
Member3 constructor
Member4 constructor
Derived2 constructor
Derived2 destructor
Member4 destructor
Member3 destructor
Derived1 destructor
Member2 destructor
Member1 destructor
Base1 destructor

Moștenirea în C++

Redefinirea funcțiilor membre

Clasa derivată are acces la toți membrii cu acces **protected** sau **public** ai clasei de bază.

Este permisă supradefinirea funcțiilor membre clasei de bază cu funcții membre ale clasei derivate.

- 2 modalități de a redefini o funcție membră:
 - **cu același antet ca în clasa de bază** (“redefining” - în cazul funcțiilor oarecare / “overriding” - în cazul funcțiilor virtuale);
 - **cu schimbarea listei de argumente sau a tipului returnat.**

Moștenirea în C++

Redefinirea funcțiilor membre

Exemplu: - păstrarea antetului/tipului returnat

```
class Baza {  
public:  
    void afis( ) {    cout<<"Baza\n";  }  
};
```

```
class Derivata : public Baza {  
public:  
    void afis( ) {    Baza::afis();    cout<<"si Derivata\n";  }  
};
```

```
int main( ) {  
    Derivata d;  
    d.afis( ); // se afiseaza "Baza si Derivata"  
}
```

Moștenirea în C++

Redefinirea funcțiilor membre

Exemplu: - nepăstrarea antetului/tipului returnat

```
class Baza {  
public:  
    void afis() {      cout<<"Baza\n";  }  
};
```

```
class Derivata : public Baza {  
public:  
    void afis (int x) {  
        Baza::afis();  
        cout<<"si Derivata\n";  }  
};
```

```
int main() {  
    Derivata d;  
    d.afis(); //nu exista Derivata::afis( )
```

Obs: la redefinirea unei funcții din clasa de baza, toate celelalte versiuni sunt automat ascunse!

Moștenirea în C++

Redefinirea funcțiilor membre

Care este efectul codului următor?

```
class Base {  
public:  
    int f() const { cout << "Base::f()\n"; return 1; }  
    int f(string) const { return 1; }  
    void g() { }  
};  
  
class Derived1 : public Base {  
public:    void g() const { }  
};  
  
class Derived2 : public Base {  
public:  
    // Redefinition:  
    int f() const { cout << "Derived2::f()\n"; return 2; }  
};
```

```
int main() {  
    string s("hello");  
  
    Derived1 d1;  
    int x = d1.f();  
    cout<<d1.f(s);  
  
    Derived2 d2;  
    x = d2.f();  
    //! d2.f(s); // string version hidden  
}
```

Moștenirea în C++

Redefinirea funcțiilor membre

Care este efectul codului următor?

```
class Base {  
public:  
    int f() const { cout << "Base::f()\n"; return 1; }  
    int f(string) const { return 1; }  
    void g() { }  
};  
class Derived3 : public Base {  
public:  
    // Change return type:  
    void f() const { cout << "Derived3::f()\n"; }  
};  
class Derived4 : public Base {  
public:  
    // Change argument list:  
    int f(int) const {  
        cout << "Derived4::f()\n";  
        return 4;  
    }  
};
```

```
int main() {  
  
    Derived3 d3;  
    //! x = d3.f(); // return int version hidden  
  
    Derived4 d4;  
    //! x = d4.f(); // f() version hidden  
    x = d4.f(1);  
}
```

Moștenirea în C++

Redefinirea funcțiilor membre

Obs:

Schimbarea interfeței clasei de bază prin modificarea tipului returnat sau a semnăturii unei funcții, înseamnă, de fapt, utilizarea clasei în alt mod.

Scopul principal al moștenirii: polimorfismul.

Schimbarea semnăturii sau a tipului returnat = schimbarea interfeței = contravine exact polimorfismului (un aspect esențial este păstrarea interfeței clasei de bază).

Moștenirea în C++

Redefinirea funcțiilor membre

Particularități la funcții

constructorii și destructorii nu sunt moșteniți până în C++11 (se redefinesc noi constr. și destr. pentru clasa derivată)

similar operatorul = (un fel de constructor care poate refolosi resurse alocate anterior – obiectul există deja)

Funcții care nu se moștenesc automat

Operatorul=

```
class Forma {  
protected:  int h;  
public:  
    Forma& operator=(const Forma& ob)  {  
        if (this!=&ob)      {      h = ob.h;      }  
        return *this;  }  
};
```

```
class Cerc: public Forma {  
protected:  
    float raza;  
public:  
    Cerc& operator=(const Cerc& ob)  {  
        if (this != &ob)      {      this->Forma::operator=(ob);      }  
        return *this;  }  
};
```

Moștenirea în C++

Moștenirea și funcțiile statice

Funcțiile membre statice se comportă exact ca și funcțiile membre nestatice:

Se moștenesc în clasa derivată.

Redefinirea unei funcții membre statice duce la ascunderea celorlalte supraîncărcări.

Schimbarea semnăturii unei funcții din clasa de bază duce la ascunderea celorlalte versiuni ale funcției.

Dar: O funcție membră statică nu poate fi virtuală. (detalii mai târziu)

Moștenirea în C++

Moștenirea și funcțiile statice

```
class Base {  
public:  
    static void f() { cout << "Base::f()\n"; }  
    static void g() { cout << "Base::f()\n"; }  
};  
  
class Derived : public Base {  
public: // Change argument list:  
    static void f(int x) { cout << "Derived::f(x)\n"; }  
};  
  
int main() {  
    int x;  
    Derived::f(); // ascunsă de supradefinirea f(x)  
    Derived::f(x);  
    Derived::g();  
}
```

Moștenirea în C++

Modificatorii de acces la moștenire

```
class A : public B { /* declarații */};
```

```
class A : protected B { /* declarații */};
```

```
class A : private B { /* declarații */};
```

Dacă modificatorul de acces la moștenire este **public**, membrii din clasa de bază își păstrează tipul de acces și în derivată.

Dacă modificatorul de acces la moștenire este **private**, toți membrii din clasa de bază vor avea tipul de acces “private” în derivată, indiferent de tipul avut în bază.

Dacă modificatorul de acces la moștenire este **protected**, membrii “publici” din clasa de bază devin “protected” în clasa derivată, restul nu se modifică.

Moștenirea în C++

```
class Baza {  
public:    void f() {cout<<"B";} };
```

```
class Derivata : public Baza{ };
```

```
int main()  
{  
    Derivata ob1;  
    ob1.f( );  
}
```

Obs. Funcția f() este accesibilă din derivată;

- modificatorul de acces la moștenire este “public”, deci f() rămâne “public” și în Derivata, deci este accesibil la apelul din main().

```
class Baza {  
public:    void f() {cout<<"B";} };
```

```
class Derivata : private sau protected Baza{ };
```

```
int main()  
{  
    Derivata ob1;  
    ob1.f( ); // inaccesibil  
}
```

- Dacă modificatorul de acces la moștenire este “private”, f() devine private în Derivata, deci inaccesibilă în main.

- Dacă modificatorul de acces la moștenire este “protected”, f() devine protected în Derivata, deci inaccesibilă în main.

Moștenirea în C++

Moștenirea cu specificatorul “private”

- inclusă în limbaj pentru completitudine;
- este mai bine a se utiliza compunerea în locul moștenirii private;
- toți membrii private din clasa de bază sunt ascunși în clasa derivată, deci inaccesibili;
- toți membrii public și protected devin private, dar sunt accesibili în clasa derivată;
- un obiect obținut printr-o astfel de derivare se tratează diferit față de cel din clasa de bază, e similar cu definirea unui obiect de tip bază în interiorul clasei noi (fără moștenire).
- dacă în clasa de bază o componentă era public, iar moștenirea se face cu specificatorul private, se poate reveni la public utilizând:

- ***using Baza::nume_componenta***

Moștenirea în C++

Moștenirea cu specificatorul “private”

```
class Pet {  
public:  
    char eat() const { return 'a'; }  
    int speak() const { return 2; }  
    float sleep() const { return 3.0; }  
    float sleep( int) const { return 4.0; }  
};
```

```
class Goldfish : Pet { // Private inheritance  
public:
```

```
    using Pet::eat; // Name publicizes member  
    using Pet::sleep; // Both overloaded members exposed  
};
```

```
int main() {  
    Goldfish bob;  
    bob.eat();  
    bob.sleep();  
    bob.sleep(1);  
    //! bob.speak();// Error: private member  
    //! function  
}
```

Moștenirea în C++

Moștenirea cu specificatorul “protected”

- secțiuni definite ca protected sunt similare ca definiție cu private (sunt ascunse de restul programului), cu excepția claselor derivate;
- good practice: cel mai bine este ca variabilele de instanță să fie PRIVATE și funcții care le modifică să fie protected;
- Sintaxă: *class derivată: protected baza {...};*
- toți membrii publici și protected din baza devin protected în derivată;
- nu se prea folosește, inclusă în limbaj pentru completitudine.

Moștenirea în C++

Moștenirea cu specificatorul “public”

```
class Base {  
protected:  
    int i;  
public:  
    Base() : i(7) {}  
};
```

```
class Derived1 : public Base { };
```

```
class Derived2 : public Derived1 {  
public:  
    void set(int x) { i = x; }  
};
```

```
• int main() {  
•     Derived2;  
•     d.set(10);  
• }
```

- dacă în baza avem zone “protected” ele sunt transmise și în derivata 1,2 tot ca protected, deci i e accesibil în funcția set()

Moștenirea în C++

Moștenirea cu specificatorul “protected”

```
class Base {  
protected:  
    int i;  
public:  
    Base() : i(7) {}  
};
```

```
class Derived1 : protected Base { };
```

```
class Derived2 : public Derived1 {  
public:  
    void set(int x) { i = x; }  
};
```

```
• int main() {  
•     Derived2;  
•     d.set(10);  
• }
```

- dacă în baza avem zone “protected” ele sunt transmise și în derivata 1,2 tot ca protected, deci i e accesibil în funcția set()

Moștenirea în C++

Moștenirea cu specificatorul “private”

```
class Base {  
protected:  
    int i;  
public:  
    Base() : i(7) {}  
};
```

```
class Derived1 : private Base { };
```

```
class Derived2 : public Derived1 {  
public:  
    void set(int x) { i = x; }  
};
```

```
• int main() {  
•     Derived2;  
•     d.set(10);  
• }
```

- moștenire derivata1 din baza (private)
atunci zonele protected devin private în
derivata1 și neaccesibile în derivata2.

Moștenirea în C++

Compatibilitatea între o clasă derivată și clasa de bază. Conversii de tip

Deoarece clasa derivată moștenește proprietățile clasei de bază, între tipul clasă derivată și tipul clasă de bază se admite o anumită compatibilitate.

Compatibilitatea este valabilă numai pentru clase **derivate cu acces public** la clasa de bază și numai în sensul de la clasa derivată spre cea de bază, nu și invers.

Compatibilitatea se manifestă sub forma unor **conversii implicite de tip**:

- dintr-un obiect derivat într-un obiect de bază;
- dintr-un pointer sau referință la un obiect din clasa derivată într-un pointer sau referință la un obiect al clasei de bază.

Perspective

Cursul 6:

Funcții virtuale în C++.

- Parametrizarea metodelor (polimorfism la execuție).
- Funcții virtuale în C++. Clase abstracte.
- Destructori virtuali.



Programare orientată pe obiecte

- suport de curs -

Anca Dobrovăț
Andrei Păun

An universitar 2025 – 2026
Semestrul II
Seriile 13, 14 și 15

Curs 6

Agenda cursului

1. Proiectarea descendentă a claselor. Moștenirea în C++ (recapitulare și completări la cursul 5)

Modificatori de acces, constructori / destructori, redefinirea membrilor unei clase de bază într-o clasă derivată.

Mostenire din baza multipla. Mostenire in diamant

2. Polimorfims la execuție prin funcții virtuale în C++.

Parametrizarea metodelor (polimorfism la execuție).

Funcții virtuale în C++.

Clase abstracte.

Overloading pe funcții virtuale

Destructori și virtualizare

1. Moștenirea in C++

C++ permite moștenirea ceea ce înseamnă că putem deriva o clasă din altă clasă de bază sau din mai multe clase.

Sintaxa:

class Clasa_Derivată : [modificatori de acces] Clasa_de_Bază { } ;

sau

*class Clasa_Derivată : [modificatori de acces] Clasa_de_Bază1,
[modificatori de acces] Clasa_de_Bază2, [modificatori de acces]
Clasa_de_Bază3*

Clasă de bază se mai numește **clasa părinte** sau **superclasă**, iar clasă derivată se mai numește **subclasa** sau **clasa copil**.

1. Moștenirea in C++

Exemplu: moștenire

```
class X { int i;
public:
    X() { i = 0; }
    void set(int ii) { i = ii; }
    int read() const { return i; }
    int permute() { return i = i * 47; }
};

class Y : public X {
    int i; // Different from X's I
public:
    Y() { i = 0; }
    int change() {
        i = permute(); // Different name call
        return i;
    }
    void set(int ii) {
        i = ii; X::set(ii); // Same-name function call
    }
};
```

```
int main() {
    cout << sizeof(X) << sizeof(Y);

    Y D;
    D.change(); // X function interface comes through:
    D.read();
    D.permute(); // Redefined functions hide base
versions:
    D.set(12);
}
```

1. Moștenirea în C++

Inițializare de obiecte

Foarte important în C++: garantarea inițializării corecte => trebuie să fie asigurată și la compoziție și moștenire.

La crearea unui obiect, compilatorul trebuie să garanteze apelul TUTUROR subobiectelor.

Problema: - cazul subobiectelor care nu au constructori implicați sau schimbarea valorii unui argument default în constructor.

De ce? - constructorul noii clase nu are permisiunea să acceseze datele private ale subobiectelor, deci nu le pot inițializa direct.

Rezolvare: - o sintaxă specială: *listă de inițializare pentru constructori*.

1. Moștenirea in C++

Exemple: lista de inițializare pentru constructori

```
class Bar {  
    int x;  
    public:  
    Bar(int i) {x = i;}  
};
```

```
class MyType: public Bar {  
    public:  
    MyType(int);  
};
```

```
MyType :: MyType (int i) : Bar (i) { ... }
```

1. Moștenirea in C++

Exemple: compoziție și moștenire

```
class A { int i;  
public:  
    A(int ii) : i(ii) {}  
    ~A() {}  
    void f() const {}  
};
```

```
class B { int i;  
public:  
    B(int ii) : i(ii) {}  
    ~B() {}  
    void f() const {}  
};
```

```
class C : public B {  
    A a;  
public:  
    C(int ii) : B(ii), a(ii) {}  
    ~C() {} // Calls ~A() and ~B()  
    void f() const  
        { // Redefinition  
            a.f();  
            B::f();  
        }  
};  
  
int main() {  
    C c(47);  
}
```


1. Moștenirea in C++

Constructorii clasei derivate

Pentru crearea unui obiect al unei clase derivate, **se creează inițial un obiect al clasei de bază** prin apelul constructorului acesteia, apoi se adaugă elementele specifice clasei derivate prin apelul constructorului clasei derivate.

Declarația obiectului derivat trebuie să conțină valorile de inițializare, **atât pentru elementele specifice, cât și pentru obiectul clasei de bază.**

Această specificare se atașează la antetul funcției constructor a clasei derivate.

În situația în care clasele de bază au definit **constructor implicit** sau **constructor cu parametri implicați**, nu se impune specificarea parametrilor care se transferă către obiectul clasei de bază.

1. Moștenirea in C++

Constructorii clasei derivate

Constructorul de copiere

Se pot distinge mai multe situații.

- 1) Dacă ambele clase, atât clasa derivată cât și clasa de bază, nu au definit constructor de copiere, se apelează constructorul implicit creat de compilator. Copierea se face membru cu membru.
- 2) Dacă clasa de bază are constructorul de copiere definit, dar clasa derivată nu, pentru clasa derivată compilatorul creează un constructor implicit care apelează constructorul de copiere al clasei de bază. (poate fi considerata un caz particular al primei situații, deoarece și partea de bază poate fi privită ca un fel de membru, iar la copiere se apelează cc pentru fiecare membru).
- 3) Dacă se definește constructor de copiere pentru clasa derivată, acestuia îi revine în totalitate sarcina transferării valorilor corespunzătoare membrilor ce aparțin clasei de bază.

1. Moștenirea în C++

Ordinea chemării constructorilor și destructorilor

Constructorii sunt chemați în ordinea definirii obiectelor ca membri ai clasei și în ordinea moștenirii:

- la fiecare nivel se apelează întâi constructorul de la moștenire, apoi constructorii din obiectele membru în clasa respectivă (care sunt chemați în ordinea definirii) și la final constructorul propriu;
- se merge pe următorul nivel în ordinea moștenirii;

Destructorii sunt chemați în ordinea inversă a constructorilor

1. Moștenirea in C++

Redefinirea funcțiilor membre

Clasa derivată are acces la toți membrii cu acces **protected** sau **public** ai clasei de bază.

Este permisă supradefinirea funcțiilor membre clasei de bază cu funcții membre ale clasei derivate.

-2 modalități de a redefini o funcție membră:

- **cu același antet ca în clasa de bază** (“redefining” - în cazul funcțiilor oarecare / “overloading” - în cazul funcțiilor virtuale);
- **cu schimbarea listei de argumente sau a tipului returnat.**

1. Moștenirea in C++

Redefinirea funcțiilor membre

Exemplu: - pastrarea antetului/tipului returnat

```
class Baza {  
public:  
    void afis( ) {      cout<<"Baza\n";  }  
};
```

```
class Derivata : public Baza {  
public:  
    void afis( ) {      Baza::afis();      cout<<"si Derivata\n";  }  
};
```

```
int main( ) {  
    Derivata d;  
    d.afis( ); // se afiseaza "Baza si Derivata"  
}
```

1. Moștenirea in C++

Redefinirea funcțiilor membre

Exemplu: - nepastrarea antetului/tipului returnat

```
class Baza {  
public:  
    void afis( ) {      cout<<"Baza\n";  }  
};
```

```
class Derivata : public Baza {  
public:  
    void afis (int x) {  
        Baza::afis();  
        cout<<"si Derivata\n";  }  
};
```

```
int main( ) {  
    Derivata d;  
    d.afis(); //nu exista Derivata::afis( )  
    d.afis(3); }
```

Obs: la redefinirea unei funcții din clasa de baza, toate celelalte versiuni sunt automat ascunse!

1. Moștenirea in C++

Redefinirea funcțiilor membre

Obs:

Schimbarea interfeței clasei de bază prin modificarea tipului returnat sau a semnăturii unei funcții, înseamnă, de fapt, utilizarea clasei în alt mod.

Scopul principal al moștenirii: polimorfismul.

Schimbarea semnăturii sau a tipului returnat = schimbarea interfeței = contravine exact polimorfismului (un aspect esențial este păstrarea interfeței clasei de bază).

1. Moștenirea in C++

Moștenirea si funcțiile statice

Funcțiile membre statice se comportă exact ca și funcțiile nemembre:
Se moștenesc în clasa derivată.

Redefinirea unei funcții membre statice duce la ascunderea celorlalte supraîncărcări.

Schimbarea semnăturii unei funcții din clasa de bază duce la ascunderea celorlalte versiuni ale funcției.

Dar: O funcție membră statică nu poate fi virtuală.

1. Moștenirea in C++

Modificatorii de acces la moștenire

```
class A : public B { /* declarații */};
```

```
class A : protected B { /* declarații */};
```

```
class A : private B { /* declarații */};
```

Dacă modificatorul de acces la moștenire este **public**, membrii din clasa de bază își păstrează tipul de acces și în derivată.

Dacă modificatorul de acces la moștenire este **private**, toți membrii din clasa de bază vor avea tipul de acces “private” în derivată, indiferent de tipul avut în bază.

Dacă modificatorul de acces la moștenire este **protected**, membrii “publici” din clasa de bază devin “protected” în clasa derivată, restul nu se modifică.

1. Moștenirea în C++

Moștenirea cu specificatorul “private”

- inclusă în limbaj pentru completitudine;
- este mai bine a se utiliza compunerea în locul moștenirii private;
- toți membrii private din clasa de bază sunt ascunși în clasa derivată, deci inaccesibili;
- toți membrii public și protected devin private, dar sunt accesibile în clasa derivată;
- un obiect obținut printr-o astfel de derivare se tratează diferit față de cel din clasa de bază, e similar cu definirea unui obiect de tip bază în interiorul clasei noi (fără moștenire).
- dacă în clasa de bază o componentă era public, iar moștenirea se face cu specificatorul private, se poate reveni la public utilizând:

using Baza::nume_componenta

1. Moștenirea in C++

Moștenirea cu specificatorul “private”

```
class Pet {  
public:  
    char eat() const { return 'a'; }  
    int speak() const { return 2; }  
    float sleep() const { return 3.0; }  
    float sleep( int) const { return 4.0; }  
};
```

```
class Goldfish : Pet { // Private inheritance
```

```
public:
```

```
    using Pet::eat; // Name publicizes member
```

```
    using Pet::sleep; // Both overloaded members exposed
```

```
};
```

```
int main() {  
    Goldfish bob;  
    bob.eat();  
    bob.sleep();  
    bob.sleep(1);  
    //! bob.speak();// Error: private member  
    function  
}
```

1. Moștenirea in C++

Moștenirea cu specificatorul “protected”

- secțiuni definite ca protected sunt similare ca definire cu private (sunt ascunse de restul programului), cu excepția claselor derivate;
- good practice: cel mai bine este ca variabilele de instanță să fie PRIVATE și funcții care le modifică să fie protected;
- Sintaxă: *class derivată: protected baza {...};*
- toți membrii publici și protected din baza devin protected în derivată;
- nu se prea folosește, inclusă în limbaj pentru completitudine.

1. Moștenirea in C++

Moștenire multiplă (MM)

- putine limbaje au MM;
- moștenirea multiplă e complicată: ambiguitate LA MOSTENIREA IN ROMB / IN DIAMANT;
- nu e nevoie de MM (se simulează cu moștenire simplă);
- se moșteneste in același timp din mai multe clase;

Sintaxa:

*class Clasa_Derivată : [modificatori de acces] Clasa_de_Bază1,
[modificatori de acces] Clasa_de_Bază2, [modificatori de acces]
Clasa_de_Bază3*

1. Moștenirea in C++

Moștenire multiplă (MM)

Exemplu:

```
class Imprimanta { };  
class Scanner { };  
class Multifuncționala: public Imprimanta, public Scanner { };
```

Ce ar putea crea probleme in cazul urmator?

```
class Baza{ };  
class Derivata_1: public Baza{ };  
class Derivata_2: public Baza{ };  
class Derivata_3: public Derivata_1, public Derivata_2 { };
```

In **Derivata_3** avem de doua ori variabilele din **Baza**!!

1. Moștenirea in C++

Moștenire multiplă : ambiguitati (problema diamantului)

```
class base { public:    int i; };  
class derived1 : public base { public:    int j; };  
class derived2 : public base { public:    int k; };  
class derived3 : public derived1, public derived2 {public:    int sum; };
```

```
int main() {  
    derived3 ob;  
    ob.i = 10; // this is ambiguous, which i    // expl ob.derived1::i  
    ob.j = 20;  
    ob.k = 30;  
    ob.sum = ob.i + ob.j + ob.k; // i ambiguous here, too  
    cout << ob.i << " "; // also ambiguous, which i?  
    cout << ob.j << " " << ob.k << " ";  
    cout << ob.sum;  
    return 0;  
}
```

1. Moștenirea in C++

Moștenire multiplă (MM)

- dar dacă avem nevoie doar de o copie lui i?
- nu vrem să consumăm spațiu în memorie;
- *folosim moștenire virtuală:*

```
class base { public:    int i; };  
class derived1 : virtual public base { public:    int j; };  
class derived2 : virtual public base { public:    int k; };  
class derived3 : public derived1, public derived2 {public:    int sum; };
```

- Dacă avem moștenire de două sau mai multe ori dintr-o clasă de bază (fiecare moștenire trebuie să fie virtuală) atunci compilatorul alocă spațiu pentru o singură copie;
- În clasele derived1 și 2 moștenirea e la fel ca mai înainte (niciun efect pentru virtual în acel caz)

2. Polimorfismul la execuție prin funcții virtuale

Funcții virtuale

Funcțiile virtuale și felul lor de folosire: componentă IMPORTANTĂ a limbajului OOP.

Folosit pentru polimorfism la execuție ---> cod mai bine organizat cu polimorfism.

Codul poate “crește” fără schimbări semnificative: programe extensibile.

Funcțiile virtuale sunt definite în bază și redefinite în clasa derivată.

Pointer de tip bază care arată către obiect de tip derivat și cheamă o funcție virtuală în bază și redefinite în clasa derivată executa ***Funcția din clasa derivată.***

Poate fi vazuta ca exemplu de separare dintre interfata si implementare.

2. Polimorfismul la execuție prin funcții virtuale

Decuplare în privința tipurilor

Upcasting - Tipul derivat poate lua locul tipului de bază (foarte important pentru procesarea mai multor tipuri prin același cod).

Funcții virtuale: ne lasă să chemăm funcțiile pentru tipul derivat.

Problemă: apel la funcție prin pointer (tipul pointerului ne da funcția apelată).

2. Polimorfismul la execuție prin funcții virtuale

```
enum note { middleC, Csharp, Eflat }; // Etc.
```

```
class Instrument { public:  
    void play(note) const {  
        cout << "Instrument::play" << endl; }  
};
```

```
class Wind : public Instrument {  
public: // Redefine interface function:  
    void play(note) const {  
        cout << "Wind::play" << endl; }  
};
```

```
void tune(Instrument& i) { i.play(middleC); }
```

```
int main() {  
    Wind flute;  
    tune(flute); // Upcasting ==> se afiseaza Instrument::play  
}
```

2. Polimorfismul la execuție prin funcții virtuale

In C ---> early binding la apel de funcții - se face la compilare.

In C++ ---> putem defini late binding prin funcții virtuale (late, dynamic, runtime binding) - se face apel de funcție bazat pe tipul obiectului, la rulare (nu se poate face la compilare).

Late binding ==> prin pointeri!

Late binding pentru o funcție: se scrie virtual înainte de definirea funcției.

Pentru clasa de bază: nu se schimbă nimic!

Pentru clasa derivată: late binding înseamnă că un obiect derivat folosit în locul obiectului de bază își va folosi funcția sa, nu cea din bază (din cauză de late binding).

Utilitate: putem extinde codul precedent fara schimbări in codul deja scris.

2. Polimorfismul la execuție prin funcții virtuale

```
enum note { middleC, Csharp, Eflat }; // Etc.
```

```
class Instrument { public:  
    virtual void play(note) const {  
        cout << "Instrument::play" << endl; }  
};
```

```
class Wind : public Instrument {  
public: // Redefine interface funcțion:  
    void play(note) const {  
        cout << "Wind::play" << endl; }  
};
```

```
void tune(Instrument& i) { i.play(middleC); }
```

```
int main() {  
    Wind flute;  
    tune(flute); // se afiseaza Wind::play  
}
```

2. Polimorfismul la execuție prin funcții virtuale

```
#include <iostream>
using namespace std;
enum note { middleC, Csharp, Eflat }; // Etc.
```

```
class Instrument {
public:
    virtual void play(note) const {
        cout << "Instrument::play" << endl;
    }
};
```

```
// Wind objects are Instruments
// because they have the same interface:
```

```
class Wind : public Instrument {
public:
    // Override interface function:
    void play(note) const {
        cout << "Wind::play" << endl;
    }
};
```

```
void tune(Instrument& i) {
    // ...
    i.play(middleC);
}
```

```
class Percussion : public Instrument {
public:
    void play(note) const {
        cout << "Percussion::play" << endl; } };
class Stringed : public Instrument {
public:
    void play(note) const {
        cout << "Stringed::play" << endl; } };
class Brass : public Wind {
public:
    void play(note) const {
        cout << "Brass::play" << endl; } };
class Woodwind : public Wind {
public:
    void play(note) const {
        cout << "Woodwind::play" << endl; } };
int main() {
    Wind flute;
    Percussion drum;
    Stringed violin;
    Brass flugelhorn;
    Woodwind recorder;
    tune(flute); tune(flugehorn); tune(violin);
}
```

2. Polimorfismul la execuție prin funcții virtuale

Cum se face late binding

Tipul obiectului este ținut în obiect pentru clasele cu funcții virtuale.

Late binding se face (uzual) cu o tabelă de pointeri: vptr către funcții.

În tabelă sunt adresele funcțiilor clasei respective (funcțiile virtuale sunt din clasa, celelalte pot fi moștenite, etc.).

Fiecare obiect din clasă are pointerul acesta în componență.

La apel de funcție membru se merge la obiect, se apelează funcția prin vptr.

Vptr este inițializat în constructor (automat).

2. Polimorfismul la execuție prin funcții virtuale

Cum se face late binding

```
class NoVirtual { int a;  
public:  
    void x() const {}  
    int i() const { return 1; } };
```

```
class OneVirtual { int a;  
public:  
    virtual void x() const {}  
    int i() const { return 1; } };
```

```
class TwoVirtuals { int a;  
public:  
    virtual void x() const {}  
    virtual int i() const { return 1; } };
```

```
int main() {  
    cout << "int: " << sizeof(int) << endl;  
    cout << "NoVirtual: "  
        << sizeof(NoVirtual) << endl;  
    cout << "void* : " << sizeof(void*) << endl;  
    cout << "OneVirtual: "  
        << sizeof(OneVirtual) << endl;  
    cout << "TwoVirtuals: "  
        << sizeof(TwoVirtuals) << endl;  
}
```


2. Polimorfismul la execuție prin funcții virtuale

Cum se face late binding

```
class Pet { public:  
    virtual string speak() const { return " "; } };  
  
class Dog : public Pet { public:  
    string speak() const { return "Bark!"; } };  
  
int main() {  
    Dog ralph;  
    Pet* p1 = &ralph;  
    Pet& p2 = ralph;  
    Pet p3;  
    // Late binding for both:  
    cout << "p1->speak() = " << p1->speak() << endl;  
    cout << "p2.speak() = " << p2.speak() << endl;  
    // Early binding (probably):  
    cout << "p3.speak() = " << p3.speak() << endl;  
}
```

2. Polimorfismul la execuție prin funcții virtuale

Daca funcțiile virtuale sunt așa de importante de ce nu sunt toate funcțiile definite virtuale (din oficiu)?

Deoarece “costă” în viteza programului.

În Java sunt “default”, dar Java ocupa mai multa memorie.

Nu mai putem avea funcții inline (ne trebuie adresa funcției pentru **vp_{tr}**).

2. Polimorfismul la execuție prin funcții virtuale

Clase abstracte și funcții virtuale pure

Clasă abstractă = clasă care are cel puțin o funcție virtuală PURĂ

Necesitate: clase care dau doar interfață (nu vrem obiecte din clasă abstractă ci upcasting la ea).

Eroare la instantierea unei clase abstracte (nu se pot defini obiecte de tipul respectiv).

Permisă utilizarea de pointeri și referințe către clasă abstractă (pentru upcasting).

Nu pot fi trimise către funcții (prin valoare).

2. Polimorfismul la execuție prin funcții virtuale

Funcții virtuale pure

Sintaxa: **virtual** *tip_returnat nume_funcție(lista_parametri) =0;*

Ex: virtual int pura(int i)=0;

Obs: La moștenire, dacă în clasa derivată nu se definește funcția pură, clasa derivată este și ea clasă abstractă ---> nu trebuie definită funcție care nu se execută niciodată

UTILIZARE IMPORTANTĂ: prevenirea “object slicing”.

2. Polimorfismul la execuție prin funcții virtuale

Clase abstracte si funcții virtuale pure

```
class Pet { string pname;
public:
    Pet(const string& name) : pname(name) {}
    virtual string name() const { return pname; }
    virtual string description() const {
        return "This is " + pname;
    }
};

class Dog : public Pet { string favoriteActivity;
public:
    Dog(const string& name, const string& activity)
        : Pet(name), favoriteActivity(activity) {}
    string description() const {
        return Pet::name() + " likes to " + favoriteActivity;
    }
};
```

```
void describe(Pet p) { // Slicing
    cout << p.description() << endl;
}

int main() {
    Pet p("Alfred");
    Dog d("Fluffy", "sleep");
    describe(p);
    describe(d);
}
```

2. Polimorfismul la execuție prin funcții virtuale

Overload pe funcții virtuale

Obs. Nu e posibil overload prin schimbarea tipului param. de întoarcere (e posibil pentru ne-virtuale)

De ce. Pentru că se vrea să se garanteze că se poate chema baza prin apelul respectiv.

Excepție: pointer către bază întors în bază, pointer către derivată în derivată

2. Polimorfismul la execuție prin funcții virtuale

Overload pe funcții virtuale

```
class Base {  
public:  
    virtual int f() const {  
        cout << "Base::f()\n"; return 1; }  
    virtual void f(string) const {}  
    virtual void g() const {}  
};  
  
class Derived1 : public Base {public:  
    void g() const {}  
};  
  
class Derived2 : public Base {public:  
    // Overriding a virtual function:  
    int f() const { cout << "Derived2::f()\n";  
        return 2; }  
};
```

```
int main() {  
    string s("hello");  
    Derived1 d1;  
    int x = d1.f();  
    d1.f(s);  
    Derived2 d2;  
    x = d2.f();  
    //! d2.f(s); // string version hidden  
}
```

2. Polimorfismul la execuție prin funcții virtuale

Overload pe funcții virtuale

```
class Base {  
public:  
    virtual int f() const {  
        cout << "Base::f()\n"; return 1; }  
    virtual void f(string) const {}  
    virtual void g() const {}  
};  
  
class Derived3 : public Base {public:  
    //! void f() const{ cout << "Derived3::f()\n";}};  
  
class Derived4 : public Base {public:  
    // Change argument list:  
    int f(int) const  
        { cout << "Derived4::f()\n"; return 4; }  
};
```

```
int main() {  
    string s("hello");  
    Derived4 d4;  
    x = d4.f(1);  
    //! x = d4.f(); // f() version hidden  
    //! d4.f(s); // string version hidden  
    Base& br = d4; // Upcast  
    //! br.f(1); // Derived version  
    //! br.f(s); // Base version available  
    return 0;  
}
```


2. Polimorfismul la execuție prin funcții virtuale

Constructorii și virtualizare

Obs. NU putem avea constructori virtuali.

În general pentru funcțiile virtuale se utilizează late binding, dar în utilizarea funcțiilor virtuale în constructori, varianta locală este folosită (early binding)

De ce?

Pentru că funcția virtuală din clasa derivată ar putea crede că obiectul e inițializat deja

Pentru că la nivel de compilator în acel moment doar VPTR local este cunoscut

2. Polimorfismul la execuție prin funcții virtuale

Destructori si virtualizare

Este uzual să se întâlnească.

Se cheamă în ordine inversă decât constructorii.

Dacă vrem să eliminăm porțiuni alocate dinamic și pentru clasa derivată dar facem upcasting trebuie să folosim destructori virtuali.

2. Polimorfismul la execuție prin funcții virtuale

Destructori si virtualizare

```
class Base1 {public: ~Base1() { cout << "~Base1()\n"; } };
```

```
class Derived1 : public Base1 {public: ~Derived1() { cout << "~Derived1()\n"; } };
```

```
class Base2 {public:  
    virtual ~Base2() { cout << "~Base2()\n"; }  
};
```

```
class Derived2 : public Base2 {public: ~Derived2() { cout << "~Derived2()\n"; } };
```

```
int main() {  
    Base1* bp = new Derived1;  
    delete bp; // Afis: ~Base1()  
    Base2* b2p = new Derived2;  
    delete b2p; // Afis: ~Derived2() ~Base2()  
}
```

2. Polimorfismul la execuție prin funcții virtuale

Destructorii virtuali puri

Utilizare: recomandat să fie utilizat dacă mai sunt și alte funcții virtuale.

Restricție: trebuie să fie definiți în clasă (chiar dacă este abstractă).

La moștenire nu mai trebuie să fie redefiniți (se construiește un destructor din oficiu)

De ce? Pentru a preveni instantierea clasei.

Obs. Nu are nici un efect dacă nu se face upcasting.

```
class AbstractBase {  
public:  
    virtual ~AbstractBase() = 0;  
};
```

```
AbstractBase::~~AbstractBase() {}
```

```
class Derived : public AbstractBase {};  
// No overriding of destructor necessary?  
int main() { Derived d; }
```

2. Polimorfismul la execuție prin funcții virtuale

Funcții virtuale in destructori

La apel de funcție virtuală din funcții normale se apelează conform VPTR

În destructori se face early binding! (apeluri locale)

De ce? Pentru că acel apel poate să se bazeze pe porțiuni deja distruse din obiect

```
class Base { public:  
    virtual ~Base() { cout << "~Base1()\n"; this->f(); }  
    virtual void f() { cout << "Base::f()\n"; }  
};  
class Derived : public Base { public:  
    ~Derived() { cout << "~Derived()\n"; }  
    void f() { cout << "Derived::f()\n"; }  
};  
  
int main() {  
    Base* bp = new Derived;  
    delete bp; // Afis: ~Derived() ~Base1() Base::f()  
}
```

2. Polimorfismul la execuție prin funcții virtuale

Downcasting

Folosit in ierarhii polimorifice (cu funcții virtuale).

Problema: upcasting e sigur pentru că respectivele funcții trebuie să fie definite în bază, downcasting e problematic.

Explicit cast prin: **dynamic_cast**

Dacă știm cu siguranță tipul obiectului putem folosi “static_cast”.

Static_cast întoarce pointer către obiectul care satisface cerințele sau 0.

Folosește tabelele VTABLE pentru determinarea tipului.

2. Polimorfismul la execuție prin funcții virtuale

Downcasting

```
class Pet { public: virtual ~Pet(){};
class Dog : public Pet {};
class Cat : public Pet {};
```

```
int main() {
    Pet* b = new Cat; // Upcast
    Dog* d1 = dynamic_cast<Dog*>(b); // Afis - 0; Try to cast it to Dog*:
    Cat* d2 = dynamic_cast<Cat*>(b); // Try to cast it to Cat*:
    // b si d2 retin aceeași adresa
    cout << "d1 = " << d1 << endl;
    cout << "d2 = " << d2 << endl;
    cout << "b = " << b << endl;
}
```

2. Polimorfismul la execuție prin funcții virtuale

Downcasting

```
class Shape { public: virtual ~Shape() {} };  
class Circle : public Shape {};  
class Square : public Shape {};  
class Other {};
```

```
int main() {  
    Circle c;  
    Shape* s = &c; // Upcast: normal and OK  
    // More explicit but unnecessary:  
    s = static_cast<Shape*>(&c);  
    // (Since upcasting is such a safe and common  
    // operation, the cast becomes cluttering)  
    Circle* cp = 0;  
    Square* sp = 0;
```

// Static Navigation of class hierarchies
requires extra type information:

```
    if(typeid(s) == typeid(cp)) // C++ RTTI  
        cp = static_cast<Circle*>(s);  
    if(typeid(s) == typeid(sp))  
        sp = static_cast<Square*>(s);  
    if(cp != 0)  
        cout << "It's a circle!" << endl;  
    if(sp != 0)  
        cout << "It's a square!" << endl;  
    // Static navigation is ONLY an efficiency  
    hack;  
    // dynamic_cast is always safer. However:  
    // Other* op = static_cast<Other*>(s);  
    // Conveniently gives an error message,  
    while  
        Other* op2 = (Other*)s;  
    // does not  
}
```


Perspective

Cursul 7:

Mostenire – completari:

Copy constructor, operator =, destructor pt clasele cu attribute de tip pointer

- clone, copy&swap,
- RAI (*Resource Acquisition Is Initialization*)

Tratarea exceptiilor



Programare orientată pe obiecte

- suport de curs -

Anca Dobrovăț
Andrei Păun

An universitar 2025 – 2026

Semestrul II

Seriile 13, 14 și 15

Curs 7 & 8

Agenda cursului

1. Moșteniri și funcții virtuale în C++

Funcții virtuale în C++

Destructorii și virtualizare

Constructorii și virtualizare

Copy & swap, RAI

Interfețe non-virtuale

2. Downcasting

3. Moștenire multiplă în C++

4. Tratarea excepțiilor

Mulumiri Asist drd Marius Micluta – Campeanu pentru co-realizarea (intr-o mare masura) a acestui material!

1. Moștenire, funcții virtuale

Redefinirea funcțiilor membre

Clasa derivată are acces la toți membrii cu acces **protected** sau **public** ai clasei de bază.

Este permisă supradefinirea funcțiilor membre clasei de bază cu funcții membre ale clasei derivate.

-2 modalități de a redefini o funcție membră:

- **cu același antet ca în clasa de bază** (“redefining” - în cazul funcțiilor oarecare / “overriding” - în cazul funcțiilor virtuale);
- **cu schimbarea listei de argumente sau a tipului returnat.**

1. Moștenire, funcții virtuale

Redefinirea funcțiilor membre

Obs:

Schimbarea interfeței clasei de bază prin modificarea tipului returnat sau a semnăturii unei funcții, înseamnă, de fapt, utilizarea clasei în alt mod.

Scopul principal al moștenirii: polimorfismul.

Schimbarea semnăturii sau a tipului returnat = schimbarea interfeței = contravine exact polimorfismului (un aspect esențial este păstrarea interfeței clasei de bază).

Mod de prevenire la funcțiile virtuale: cuvântul cheie **override** (C++11)

1. Moștenire, funcții virtuale

Funcții virtuale

Codul poate “crește” fără schimbări semnificative: programe **ușor** de extins

Exemplu de separare dintre interfață și implementare

- Clasele care folosesc interfața definită în clasa de bază **nu se modifică** atunci când schimbăm implementarea sau când adăugăm o nouă derivată

Pointer de tip bază care arată către obiect de tip derivat și cheamă o funcție virtuală din bază execută funcția redefinită în cea mai specifică derivată - **late binding**

Upcasting - Tipul derivat poate lua locul tipului de bază (L-ul din SOLID – va urma)

Funcții virtuale pure: forțează derivatele să definească o implementare

Tipul de retur al unei funcții virtuale nu poate fi schimbat în derivate.

Excepție: tipuri covariante.

Tipuri covariante și în alte limbaje: C#, Dart, Java, Python, Scala, TypeScript

1. Moștenire, funcții virtuale

```
enum note { middleC, Csharp, Eflat }; // Etc.
```

```
class Instrument { public:  
    virtual void play(note) const = 0;  
};
```

```
void Instrument::play(note) const { std::cout << "Instrument::play" << std::endl; }
```

```
class Wind : public Instrument {  
public: void play(note) const override { std::cout << "Wind::play" << std::endl; } };
```

```
class String : public Instrument {  
public: void play(note) const override { std::cout << "String::play" << std::endl; } };
```

```
void tune(Instrument& i) { i.play(middleC); }
```

```
int main() {  
    Wind flute; String cello;  
    tune(flute); tune(cello);  
}
```

1. Moștenire, funcții virtuale

Constructori și virtualizare

Obs. **NU putem avea constructori virtuali** (în sensul "virtual Instrument() {}")

NU apelăm funcții virtuale în constructori sau destructori ([detalii](#))

- Se va apela definiția din clasa curentă, nu dintr-o clasă mai derivată
- Deși de obicei nu este o problemă în sine, poate cauza confuzie în proiecte mari
- În limbaje dinamice (e.g. Python) nu este o problemă, dar acolo nu avem verificări de tipuri prea stricte

Soluții:

- Constructori “virtuali” în sensul de (**funcții de clonare**)
- Clase și funcții de tip factory

Dacă avem nevoie de funcții virtuale în destructori, ar trebui să ne întrebăm de ce am avea nevoie de așa ceva

- "soluție": apel explicit: Base::f() sau Derived::f()

1. Moștenire, funcții virtuale

Destructor și virtualizare

Se cheamă în ordine inversă decât constructorii.

În general două situații:

- Destructor public și virtual
 - Dacă avem și alte funcții virtuale, deci folosim pointeri către bază
- Destructor protected și non-virtual
 - Dacă dorim să folosim doar obiecte derivate

Dacă vrem să eliminăm porțiuni alocate dinamic și pentru clasa derivată dar facem upcasting trebuie să folosim destructori virtuali.

1. Moștenire, funcții virtuale

Destructori și virtualizare

```
class Base1 {public: ~Base1() { cout << "~Base1()\n"; } };
```

```
class Derived1 : public Base1 {public: ~Derived1() { cout << "~Derived1()\n"; } };
```

```
class Base2 {public:  
    virtual ~Base2() { cout << "~Base2()\n"; }  
};
```

```
class Derived2 : public Base2 {public: ~Derived2() { cout << "~Derived2()\n"; } };
```

```
int main() {  
    Base1* bp = new Derived1;  
    delete bp; // Afis: ~Base1()  
    Base2* b2p = new Derived2;  
    delete b2p; // Afis: ~Derived2() ~Base2()  
}
```

1. Moștenire, funcții virtuale

Destructori virtuali puri

Utilizare: poate fi utilizat dacă avem funcții virtuale pe care nu vrem să le suprascriem în toate derivatele.

Restricție: trebuie să aibă o definiție (chiar dacă este abstractă).

La moștenire nu mai trebuie redefiniți (se construiește un destructor din oficiu)

De ce? Pentru a preveni instanțierea clasei.

Obs. Nu are nici un efect dacă nu se face upcasting, dar atunci am folosi destructor protected și non-virtual.

```
class AbstractBase {
```

```
public:
```

```
    virtual ~AbstractBase() = 0;
```

```
};
```

```
AbstractBase::~~AbstractBase() {}
```

```
class Derived : public AbstractBase {};
```

```
// No overriding of destructor necessary?
```

```
int main() { Derived d; }
```

1. Moștenire, funcții virtuale

Destructori protected și non-virtuali

Utilizare: dorim să ținem într-o clasă de bază comună attribute și funcții, dar construim doar obiecte derivate și nu avem nevoie de pointeri sau referințe de bază.

De ce nu public?

Pentru a preveni instanțierea clasei din exterior (nu avem object slicing).

De ce protected și nu private?

Pentru a putea fi apelat de clasele derivate.

```
class Base {
```

```
protected:
```

```
    ~Base() = default;
```

```
};
```

```
class Derived1 : public Base {};
```

```
class Derived2 : public Base {};
```

```
int main() { Derived1 d1; Derived2 d2; /* Base b; */ /*error */ }
```

1. Moștenire, funcții virtuale

Constructorii și virtualizare

Situație: într-o clasă reținem ca atribut un pointer de tip bază al altei clase pentru a putea apela funcții virtuale prin acel pointer.

```
class Instrument {  
public:  
    virtual ~Instrument() = 0;  
    virtual void play(int) const {} };  
Instrument::~~Instrument() = default;  
class Wind : public Instrument { /* implementare play */ };  
class String : public Instrument { /* implementare play */ };  
class Orchestra {  
    std::vector<Instrument*> instruments;  
public:  
    void add(Instrument* inst) { instruments.push_back(inst); }  
    void rehearse() { for(const auto& inst : instruments) inst->play(0); }  
};  
int main() { Orchestra o1; o1.add(new Wind); o1.add(new String); o1.rehearse(); }
```

1. Moștenire, funcții virtuale

Constructorii și virtualizare

Ce probleme pot apărea?

Avem memory leaks (am folosit [DrMemory](#)).

```
~~Dr.M~~ Error #3: LEAK 8 direct bytes 0x0000018f7e600840-0x0000018f7e600848 + 0 indirect bytes
~~Dr.M~~ # 0 replace_operator_new [D:\a\drmemory\drmemory\common\alloc_replace.c:2903]
~~Dr.M~~ # 1 main
~~Dr.M~~
~~Dr.M~~ Error #4: LEAK 8 direct bytes 0x0000018f7e6008a0-0x0000018f7e6008a8 + 0 indirect bytes
~~Dr.M~~ # 0 replace_operator_new [D:\a\drmemory\drmemory\common\alloc_replace.c:2903]
~~Dr.M~~ # 1 main
~~Dr.M~~
~~Dr.M~~ ERRORS FOUND:
~~Dr.M~~      1 unique,      2 total unaddressable access(es)
~~Dr.M~~      0 unique,      0 total uninitialized access(es)
~~Dr.M~~      0 unique,      0 total invalid heap argument(s)
~~Dr.M~~      0 unique,      0 total GDI usage error(s)
~~Dr.M~~      0 unique,      0 total handle leak(s)
~~Dr.M~~      0 unique,      0 total warning(s)
~~Dr.M~~      2 unique,      2 total,      16 byte(s) of leak(s)
~~Dr.M~~      1 unique,      1 total,      26 byte(s) of possible leak(s)
~~Dr.M~~ ERRORS IGNORED:
~~Dr.M~~      3 potential error(s) (suspected false positives)
```

1. Moștenire, funcții virtuale

Constructorii și virtualizare

Cine ar trebui să elibereze memoria? Adăugăm destructorul și nu mai avem leaks.

```
class Instrument {  
public:  
    virtual ~Instrument() = 0;  
    virtual void play(int) const {} };  
Instrument::~~Instrument() = default;  
class Wind : public Instrument { /* implementare play */ };  
class String : public Instrument { /* implementare play */ };  
class Orchestra {  
    std::vector<Instrument*> instruments;  
public:  
    void add(Instrument* inst) { instruments.push_back(inst); }  
    void rehearse() { for(const auto& inst : instruments) inst->play(0); }  
    ~Orchestra() { for(auto* inst : instruments) delete inst; }  
};  
int main() { Orchestra o1; o1.add(new Wind); o1.add(new String); o1.rehearse(); }
```

1. Moștenire, funcții virtuale

Constructori și virtualizare

Nu mai avem memory leaks, dar... este corect?

```
class Instrument {
public:
    virtual ~Instrument() = 0;
    virtual void play(int) const {} };
Instrument::~~Instrument() = default;
class Wind : public Instrument { /* implementare play */ };
class String : public Instrument { /* implementare play */ };
class Orchestra {
    std::vector<Instrument*> instruments;
public:
    void add(Instrument* inst) { instruments.push_back(inst); }
    void rehearse() { for(const auto& inst : instruments) inst->play(0); }
    ~Orchestra() { for(auto* inst : instruments) delete inst; }
};
int main() { Orchestra o1; o1.add(new Wind); o1.add(new String); o1.rehearse();
    Orchestra o2 = o1; // sau Orchestra o3(o1);
}
```


1. Moștenire, funcții virtuale

Constructorii și virtualizare

Nu, cedează la execuție.

Dacă avem atribute de tip pointeri, constructorul de copiere copiază adrese de memorie.

```
~~Dr.M~~ Error #2: UNADDRESSABLE ACCESS of freed memory: reading 0x00000176cb2a0840-0x00000176cb2a0848 8 byte(s)
~~Dr.M~~ # 0 vec::~vec
~~Dr.M~~ # 1 main
~~Dr.M~~ Note: @0:00:00.434 in thread 7412
~~Dr.M~~ Note: next higher malloc: 0x00000176cb2a08d0-0x00000176cb2a08e0
~~Dr.M~~ Note: prev lower malloc: 0x00000176cb2a06a0-0x00000176cb2a07a0
~~Dr.M~~ Note: 0x00000176cb2a0840-0x00000176cb2a0848 overlaps memory 0x00000176cb2a0840-0x00000176cb2a0848 that was freed here:
~~Dr.M~~ Note: # 0 replace_operator_delete_nothrow [D:\a\drmemory\drmemory\common\alloc_replace.c:2978]
~~Dr.M~~ Note: # 1 derivata1::~derivata1
~~Dr.M~~ Note: # 2 vec::~vec
~~Dr.M~~ Note: # 3 main
~~Dr.M~~ Note: instruction: mov (%rax) -> %rdx
~~Dr.M~~
~~Dr.M~~ Error #3: UNADDRESSABLE ACCESS beyond heap bounds: reading 0x00000176cb2a0888-0x00000176cb2a0890 8 byte(s)
~~Dr.M~~ # 0 vec::~vec
~~Dr.M~~ # 1 main
```

1. Moștenire, funcții virtuale

Constructori și virtualizare

Nu, cedează la execuție.

Dacă avem atribute de tip pointeri, constructorul de copiere copiază adrese de memorie.

```
~Dr.M~~ ERRORS FOUND:
~Dr.M~~      3 unique,      4 total unaddressable access(es)
~Dr.M~~      1 unique,      3 total uninitialized access(es)
~Dr.M~~      0 unique,      0 total invalid heap argument(s)
~Dr.M~~      0 unique,      0 total GDI usage error(s)
~Dr.M~~      0 unique,      0 total handle leak(s)
~Dr.M~~      0 unique,      0 total warning(s)
~Dr.M~~      1 unique,      1 total,      16 byte(s) of leak(s)
~Dr.M~~      0 unique,      0 total,      0 byte(s) of possible leak(s)
~Dr.M~~ ERRORS IGNORED:
~Dr.M~~      4 potential error(s) (suspected false positives)
~Dr.M~~      (details: C:\Users\marius\AppData\Roaming\Dr. Memory\DrM
~Dr.M~~      11 unique,      11 total,      2227 byte(s) of still-reachable al
~Dr.M~~      (re-run with "-show_reachable" for details)
~Dr.M~~ Details: C:\Users\marius\AppData\Roaming\Dr. Memory\DrMemory-main
~Dr.M~~ WARNING: application exited with abnormal code 0xc0000005

C:\Users\marius\Documents\facultate\ore\2023-2024>main.exe

C:\Users\marius\Documents\facultate\ore\2023-2024>echo %errorlevel%
-1073741819
```

1. Moștenire, funcții virtuale

Constructori și virtualizare

Soluție: suprascriem constructorul de copiere, dar... **Ce copiem?**

```
class Orchestra {  
    std::vector<Instrument*> instruments;  
public:  
    void add(Instrument* inst) { instruments.push_back(inst); }  
    void rehearse() { for(const auto& inst : instruments) inst->play(0); }  
    ~Orchestra() { for(auto* inst : instruments) delete inst; }  
    Orchestra() = default;  
    Orchestra(const Orchestra& other) {  
        for(const auto& inst : other.instruments)  
            instruments.push_back(new ???); // new Wind() sau new String()??  
    }  
};
```

1. Moștenire, funcții virtuale

Constructori și virtualizare

"Soluția" 1: atribut pentru subtip.

```
class Instrument {
public:
    enum tip {wind, string};
    virtual ~Instrument() = 0;    virtual void play(int) const {} };
Instrument::~~Instrument() = default;
class Wind : public Instrument { /* inițializare tip cu wind, implementare play */ };
class String : public Instrument { /* inițializare tip cu string, implementare play */ };
class Orchestra {
    std::vector<Instrument*> instruments;
public:    Orchestra() = default;
    Orchestra(const Orchestra& other) {
        for(const auto& inst : other.instruments)
            if(inst.getTip() == Instrument::wind)
                instruments.push_back(new Wind(static_cast<Wind*>(inst))); // apelăm cc
            else if // etc
    };
};
```

1. Moștenire, funcții virtuale

Constructori și virtualizare

Problema cu "soluția" 1: trebuie să actualizăm de fiecare dată clasa de bază atunci când adăugăm o nouă derivată (recompilăm toate subclasele).

"Soluția" 2: dynamic_cast.

```
class Instrument {
public:
    virtual ~Instrument() = 0;    virtual void play(int) const {} };
Instrument::~~Instrument() = default;
class Wind : public Instrument { /* implementare play */ };
class String : public Instrument { /* implementare play */ };
class Orchestra {
    std::vector<Instrument*> instruments;
public:
    Orchestra() = default;
    Orchestra(const Orchestra& other) {
        for(const auto& inst : other.instruments)
            if(auto* wind = dynamic_cast<Wind*>(inst))
                instruments.push_back(new Wind(*wind));
            else if // etc
    };
};
```

1. Moștenire, funcții virtuale

Constructori și virtualizare

Problema cu "soluția" 2: peste tot unde avem nevoie de o copie a unui instrument va trebui să avem ramuri if/else și să facem un `dynamic_cast` (sau `typeid + static_cast`).

Pentru fiecare nouă derivată va trebui să adăugăm **peste tot** câte o nouă ramură if/else.

Soluția corectă 1: nu permitem copieri, folosim doar mutări. Dezavantaj dpdv didactic: necesare multe apeluri `std::move`

```
class Orchestra {  
    std::vector<Instrument*> instruments;  
public:  
    Orchestra() = default;  
    Orchestra(const Orchestra& other) = delete;  
    Orchestra& operator=(const Orchestra& other) = delete;  
    Orchestra(Orchestra&& other) = default;  
    Orchestra& operator=(Orchestra&& other) = default;  
    ~Orchestra() = default;  
};
```

1. Moștenire, funcții virtuale

Constructori și virtualizare

Soluția corectă 2 (funcție clone): fiecare subclasă ar trebui să știe să se copieze pe sine.

Având în vedere că în clasa Orchestra avem doar pointeri de tip bază, pentru copiere vom folosi o **funcție virtuală**.

```
class Instrument {  
public:  
    virtual ~Instrument() = default; // sau = 0  
    virtual void play(int) const {} };  
    virtual Instrument* clone() const = 0; };  
  
class Wind : public Instrument { /* implementare play */  
public: Instrument* clone() const override { return new Wind(*this); } }; // apelăm cc  
};  
  
class String : public Instrument { /* implementare play */  
public: Instrument* clone() const override { return new String(*this); } };  
};
```

1. Moștenire, funcții virtuale

Constructori și virtualizare

Soluția corectă 2 (funcție clone)

Clasa Orchestra se transformă în felul următor:

```
class Orchestra {  
    std::vector<Instrument*> instruments;  
public:  
    void add(Instrument* inst) { instruments.push_back(inst->clone()); }  
    void rehearse() { for(const auto& inst : instruments) inst->play(0); }  
    Orchestra() = default;  
    ~Orchestra() { for(auto* inst : instruments) delete inst; }  
    Orchestra(const Orchestra& other) {  
        for(const auto& inst : other.instruments)  
            instruments.push_back(inst->clone());    }  
};
```

Apelăm funcția clone și în funcția "add" deoarece vrem să fim siguri că obiectul de tip Orchestra **deține toate resursele sale**, deci nu depinde de ce se va întâmpla cu parametrii.

1. Moștenire, funcții virtuale

Constructorii și virtualizare

Soluția corectă 2 (funcție clone)

Clasa Orchestra se transformă în felul următor:

```
class Orchestra {
    std::vector<Instrument*> instruments;
public:
    void add(const Instrument& inst) { instruments.push_back(inst.clone()); }
    Orchestra() = default;
    ~Orchestra() { for(auto* inst : instruments) delete inst; }
    Orchestra(const Orchestra& other) {
        for(const auto& inst : other.instruments)
            instruments.push_back(inst->clone());
    };
};

int main() {
    Orchestra o1; // o1.add(new Wind); // aici am avea memory leak
    String *s1 = new String; o1.add(*s1); delete s1; // o1 are în continuare o copie lui s1
    String s2; o1.add(s2); /* nu se pune problema să facem delete la o variabilă locală */
}
```

1. Moștenire, funcții virtuale

Constructorii și virtualizare

Este corect acum?

Copierile funcționează, dar nu și atribuirile:

```
int main() {  
    Orchestra o1;  
    String s; o1.add(s);  
    Orchestra o2 = o1;    // apel constructor de copiere  
    o2 = o1;              // apel operator=  
}
```

1. Moștenire, funcții virtuale

Constructorii și virtualizare

Este corect acum?

Copierile funcționează, dar nu și atribuirile:

```
int main() {  
    Orchestra o1;  
    String s; o1.add(s);  
    Orchestra o2 = o1;  
    o2 = o1;  
}
```

```
~~Dr.M~~ Error #2: UNADDRESSABLE ACCESS of freed memory: reading 0x000002d2690e0840-0x000002d2690e0848 8 byte(s)  
~~Dr.M~~ # 0 vec::~vec  
~~Dr.M~~ # 1 main  
~~Dr.M~~ Note: @0:00:00.422 in thread 1060  
~~Dr.M~~ Note: next higher malloc: 0x000002d2690e08d0-0x000002d2690e08e0  
~~Dr.M~~ Note: prev lower malloc: 0x000002d2690e06a0-0x000002d2690e07a0  
~~Dr.M~~ Note: 0x000002d2690e0840-0x000002d2690e0848 overlaps memory 0x000002d2690e0840-0x000002d2690e0848 that was freed here:  
~~Dr.M~~ Note: # 0 replace_operator_delete_nothrow [D:\a\drmemory\drmemory\common\alloc_replace.c:2978]  
~~Dr.M~~ Note: # 1 derivata1::~derivata1  
~~Dr.M~~ Note: # 2 vec::~vec  
~~Dr.M~~ Note: # 3 main  
~~Dr.M~~ Note: instruction: mov (%rax) -> %rdx
```

1. Moștenire, funcții virtuale

Constructorii și virtualizare

Este corect acum?

Copierile funcționează, dar nu și atribuirile:

```
int main() {  
    Orchestra o1;  
    String s; o1.add(s);  
    Orchestra o2 = o1;  
    o2 = o1;  
}
```

```
ERRORS FOUND:  
    3 unique,      4 total unaddressable access(es)  
    1 unique,      3 total uninitialized access(es)  
    0 unique,      0 total invalid heap argument(s)  
    0 unique,      0 total GDI usage error(s)  
    0 unique,      0 total handle leak(s)  
    0 unique,      0 total warning(s)  
    3 unique,      3 total,      32 byte(s) of leak(s)  
    0 unique,      0 total,      0 byte(s) of possible leak(s)  
ERRORS IGNORED:  
    4 potential error(s) (suspected false positives)  
    (details: C:\Users\marius\AppData\Roaming\Dr. Memory\DrMemory.log)  
  
    11 unique,     11 total,     2227 byte(s) of still-reachable memory  
    (re-run with "-show_reachable" for details)  
Details: C:\Users\marius\AppData\Roaming\Dr. Memory\DrMemory.log  
WARNING: application exited with abnormal code 0xc0000005
```

1. Moștenire, funcții virtuale

Constructori și virtualizare

Suprascriem și operator=

```
class Orchestra {  
    std::vector<Instrument*> instruments;  
public:  
    void add(const Instrument& inst) { instruments.push_back(inst.clone()); }  
    Orchestra() = default;  
    ~Orchestra() { for(auto* inst : instruments) delete inst; }  
    Orchestra(const Orchestra& other) {  
        for(const auto& inst : other.instruments)  
            instruments.push_back(inst->clone());  
    }  
    Orchestra& operator=(const Orchestra& other) {  
        if(this == &other) return *this;  
        for(auto* inst : instruments) delete inst;  
        instruments.clear();  
        for(const auto& inst : other.instruments)  
            instruments.push_back(inst->clone());  
        return *this;  
    }  
};
```

1. Moștenire, funcții virtuale

Regula celor trei

Dacă într-o clasă trebuie să suprascriem cc/op=/destr, cel mai probabil trebuie suprascrise toate cele trei funcții speciale.

```
class Orchestra {  
    std::vector<Instrument*> instruments;  
public:  
    void add(const Instrument& inst) { instruments.push_back(inst.clone()); }  
    Orchestra() = default;  
    ~Orchestra() { for(auto* inst : instruments) delete inst; }  
    Orchestra(const Orchestra& other) {  
        for(const auto& inst : other.instruments)  
            instruments.push_back(inst->clone());  
    }  
    Orchestra& operator=(const Orchestra& other) {    if(this == &other)    return *this;  
        for(auto* inst : instruments) delete inst;  
        instruments.clear();  
        for(const auto& inst : other.instruments)  
            instruments.push_back(inst->clone());  
        return *this; }  
};
```

1. Moștenire, funcții virtuale

Constructori și virtualizare

```
class Orchestra {  
    std::vector<Instrument*> instruments;  
public:  
    void add(const Instrument& inst) { instruments.push_back(inst.clone()); }  
    Orchestra() = default;  
    ~Orchestra() { for(auto* inst : instruments) delete inst; }  
    Orchestra(const Orchestra& other) {  
        for(const auto& inst : other.instruments)  
            instruments.push_back(inst->clone()); }  
    Orchestra& operator=(const Orchestra& other) { if(this == &other) return *this;  
        for(auto* inst : instruments) delete inst;  
        instruments.clear();  
        for(const auto& inst : other.instruments)  
            instruments.push_back(inst->clone()); // ce se întâmplă dacă nu reușește copierea?  
        return *this; }  
};
```

Obiectul va fi într-o stare invalidă! Am pierdut datele vechi și nu am copiat datele noi.

1. Moștenire, funcții virtuale

Copy and swap

Soluția 2.5: Trebuie să efectuăm copierea noilor atribute **înainte** de a șterge datele vechi.

Pentru copiere **refolosim** implementarea din constructorul de copiere, iar pentru eliberarea vechilor resurse **refolosim** implementarea destructorului:

```
class Orchestra {  
    std::vector<Instrument*> instruments;  
public:  
    Orchestra() = default;  
    ~Orchestra() { for(auto* inst : instruments) delete inst; }  
    Orchestra(const Orchestra& other) {  
        for(const auto& inst : other.instruments) instruments.push_back(inst->clone());  
    }  
    Orchestra& operator=(const Orchestra& other) { if(this == &other) return *this;  
        auto copie = other; // aici se apelează constructorul de copiere  
        std::swap(this->instrumente, copie.instrumente);  
        return *this;  
    } // aici se apelează destructorul pentru copie  
};
```


1. Moștenire, funcții virtuale

Copy and swap

Soluția 2.6: Pentru copiere putem apela constructorul de copiere transmițând parametrul de la operator= prin valoare, simplificând astfel codul:

```
class Orchestra {  
    std::vector<Instrument*> instruments;  
public:  
    Orchestra() = default;  
    ~Orchestra() { for(auto* inst : instruments) delete inst; }  
    Orchestra(const Orchestra& other) {  
        for(const auto& inst : other.instruments) instruments.push_back(inst->clone());  
    }  
    Orchestra& operator=(Orchestra other) // aici se apelează constructorul de copiere  
    {  
        if(this == &other) return *this;  
        std::swap(this->instrumente, other.instrumente);  
        return *this;  
    } // aici se apelează destructorul pentru copie  
};
```

1. Moștenire, funcții virtuale

Copy and swap

Caz general: partea de swap poate fi refolosită în alte situații, motiv pentru care definim o funcție separată de swap:

```
class Orchestra {  
    std::vector<Instrument*> instruments;  
public:  
    Orchestra() = default;  
    ~Orchestra() { for(auto* inst : instruments) delete inst; }  
    Orchestra(const Orchestra& other) {  
        for(const auto& inst : other.instruments) instruments.push_back(inst->clone());  
    }  
    Orchestra& operator=(Orchestra other) // aici se apelează constructorul de copiere  
    {  
        if(this == &other) return *this;  
        swap(this, other);  
        return *this;  
    } // aici se apelează destructorul pentru copie  
};
```

1. Moștenire, funcții virtuale

Copy and swap

Caz general: partea de swap poate fi refolosită în alte situații, motiv pentru care definim o funcție separată de swap:

```
class Orchestra {  
    // codul anterior  
    friend void swap(Orchestra& o1, Orchestra& o2) {  
        using std::swap;  
        swap(o1.instrumente, o2.instrumente);  
        //swap(o1.dirijor, o2.dirijor);  
    }  
};
```

De ce funcție friend?

- Dacă și în altă clasă avem o funcție de swap, aceasta va fi găsită de ADL
- ADL (argument dependent lookup) găsește (doar) funcții friend

De ce "using std::swap"?

- Pentru a scrie la fel toate apelurile de swap, nu unele cu std:: și altele doar swap

1. Moștenire, funcții virtuale

RAII (resource acquisition is initialization)

Ideea de gestionare a resurselor în C++ este aceea că resursele ar trebui alocate doar în constructori și eliberate doar în destructori.

De ce?

Constructorii și destructorii sunt apelați automat de limbaj. Dacă nu facem alocări/dezalocări în alte locuri, este imposibil să avem memory leaks într-un program corect.

Așadar, spre deosebire de multe alte limbaje OOP, nu apare necesitatea de garbage collection: folosind RAII nu lăsăm în urmă "gunoaie".

Exemplu de RAII din limbaj: smart pointers.

1. Moștenire, funcții virtuale

RAII (resource acquisition is initialization)

Pentru a folosi smart pointers în funcțiile de clone, avem de făcut doar aceste modificări:

- Înlocuim `Instrument*` cu `std::shared_ptr<Instrument>`
- Înlocuim `new Wind(args)` cu `std::make_shared<Wind>(args)` (idem pentru `String`)
- Eliminăm apelurile de `delete`

Avantaje:

- Nu avem nevoie de destructori expliți în care să facem `delete`
- Este corect să facem apeluri de forma
`func(std::make_shared<T1>(), std::make_shared<T2>())`

Dezavantaj:

- Nu putem folosi tipuri de date covariante

1. Moștenire, funcții virtuale

Tipuri de date covariante

```
#include <iostream>
class Baza {
public:
    virtual ~Baza() = default;
    virtual Baza* clone() const = 0;
};

class Derivata1 : public Baza {
public:
    Baza* clone() const override {
        return new Derivata1(*this);
    }
    void f() { std::cout << "f der1\n"; }
};

class Derivata2 : public Baza {
public:
    Derivata2* clone() const override {
        return new Derivata2(*this);
    }
    void g() { std::cout << "g der2\n"; }
};
```

```
int main() {
    Baza* b1 = new Derivata1;
    // Derivata1* d1 = b1->clone(); // eroare
    // b1->f(); // eroare
    delete b1;
    Baza* b2 = new Derivata2;
    Derivata2 d2;
    // Derivata2* d2_1 = b2->clone(); // eroare
    Derivata2* d2_2 = d2.clone(); // ok
    d2_2->g(); // ok
    delete b2;
    delete d2_2;
}
```

1. Moștenire, funcții virtuale

Interfață non-virtuală (NVI)

Utilitate: toate clasele derivate au o implementare comună și au nevoie să suprascrie doar anumite porțiuni.

Exemplu fără NVI: observăm că doar o mică parte din implementările din derivate diferă.

```
class Instrument {  
public:  
    virtual ~Instrument() = default; virtual void play(int) const {} };
```

```
class Wind : public Instrument {  
public:  
    void play(int note) const override {  
        readNote(note);  
        prepare();  
        std::cout << "play wind\n";  
        rest(10ms);  
    }  
};
```

```
class String : public Instrument {  
public:  
    void play(int note) const override {  
        readNote(note);  
        prepare();  
        std::cout << "play string\n";  
        rest(10ms);  
    }  
};
```

1. Moștenire, funcții virtuale

Interfață non-virtuală (NVI)

Exemplu cu NVI: mutăm partea comună în clasa de bază, iar derivatele suprascriu strict partea care diferă.

```
class Instrument {  
public:  
    virtual ~Instrument() = default;  
    void perform(int note) const {  
        readNote(note);  prepare();  play(note);  rest(10ms);  
    }  
private:  
    virtual void play(int) const {} };
```

```
class Wind : public Instrument {  
public:  
    void play(int note) const override {  
        std::cout << "play wind\n";  
    }  
};
```

```
class String : public Instrument {  
public:  
    void play(int note) const override {  
        std::cout << "play string\n";  
    }  
};
```


1. Moștenire, funcții virtuale

Interfață non-virtuală (NVI)

Prin această abordare, este mult mai ușor să modificăm în mod uniform structura implementării la nivelul întregii ierarhii (și avem de recompilat un singur fișier).

În plus, prin NVI forțăm ca apelul să se realizeze doar prin funcția publică non-virtuală, ceea ce înseamnă că o derivată nu va putea "ocoli" implementarea comună impusă de clasa de bază.

Exemple de situații:

- setup/cleanup comun, testare, benchmarks
- rezolvă unele probleme de la moștenirea multiplă
- vezi mai târziu și TemplateMethod pattern, D din SOLID

2. Downcasting

Folosit in ierarhii polimorfice (cu funcții virtuale) pentru a obține înapoi derivata către care arată un pointer de tip bază.

Problema: upcasting e sigur pentru că respectivele funcții/attribute trebuie să fie definite în bază, downcasting e problematic.

Explicit cast prin: **dynamic_cast**

Dacă știm cu siguranță tipul obiectului, putem folosi “static_cast”.

static_cast întoarce pointer către obiectul care satisface cerințele sau 0.

dynamic_cast folosește tabelele VTABLE pentru determinarea tipului.

Conversiile tip C++ documentează de ce avem nevoie de un anumit cast

- Numele conversiilor sunt lungi pentru a atrage atenția (și pentru a le evita)

Conversiile tip C nu exprimă ce intenție avem și anulează verificările compilatorului.

2. Downcasting

Problema: downcasting e problematic și din alt motiv: dacă avem de făcut "if" pe tipuri de date, ori nu este abstractizarea bună, ori trebuie să folosim de fapt funcții virtuale.

Un program cu multe comparații explicite de subclase este ușor de scris pe termen scurt (exemplu: un semestru), dar dificil de întreținut și extins pe termen mediu/lung.

dynamic_cast sau typeid + static_cast?

dynamic_cast este mai lent, dar codul va funcționa în continuare dacă transmitem un pointer și mai derivat (este "mai polimorfic"): "este un fel de derivata1"

typeid este mai rapid, dar compară un singur tip de date: "este exact derivata1"

2. Downcasting

dynamic_cast

```
class Pet { public: virtual ~Pet(){};
```

```
class Dog : public Pet {};
```

```
class Cat : public Pet {};
```

```
int main() {
```

```
    Pet* b = new Cat; // Upcast
```

```
    if (auto* d1 = dynamic_cast<Dog*>(b)) {
```

```
        std::cout << "d1 = " << d1 << std::endl;
```

```
    }
```

```
    else if (auto* d2 = dynamic_cast<Cat*>(b)) {
```

```
        std::cout << "d2 = " << d2 << std::endl; // b și d2 rețin aceeași adresă
```

```
        std::cout << "b = " << b << std::endl;
```

```
    }
```

```
    try{
```

```
        auto& d1 = dynamic_cast<Dog&>(*b); // dacă nu reușește, se aruncă excepție
```

```
        auto& d2 = dynamic_cast<Cat&>(*b);
```

```
    } catch(std::bad_cast&) { std::cout << "eroare cast\n"; }
```

```
}
```

2. Downcasting

typeid (sau atribut intern) + static_cast // Static Navigation of class hierarchies

```
class Shape { public: virtual ~Shape() {} };
class Circle : public Shape {};
class Square : public Shape {};
class Other {};
```

```
int main() {
    Circle c;
    Shape* s = &c; // Upcast: normal and OK
    // More explicit but unnecessary:
    s = static_cast<Shape*>(&c);
    // (Since upcasting is such a safe and common
    // operation, the cast becomes cluttering)
    Circle* cp = 0;
    Square* sp = 0;
```

requires extra type information:

```
if(typeid(s) == typeid(cp)) // C++ RTTI
    cp = static_cast<Circle*>(s);
if(typeid(s) == typeid(sp))
    sp = static_cast<Square*>(s);
if(cp != 0)
    cout << "It's a circle!" << endl;
if(sp != 0)
    cout << "It's a square!" << endl;
// Static navigation is ONLY an efficiency
hack;
// dynamic_cast is always safer. However:
// Other* op = static_cast<Other*>(s);
// Conveniently gives an error message,
while
    Other* op2 = (Other*)s;
// does not
}
```

3. Moștenire multiplă și virtuală

Moștenire multiplă (MM)

- puține limbaje au MM;
- moștenirea multiplă e complicată: ambiguitate LA MOȘTENIREA IN ROMB / IN DIAMANT;
 - Soluție: interfață non-virtuală
- nu e nevoie de MM (se simulează cu moștenire simplă);
- se moștenește în același timp din mai multe clase;

Sintaxa:

*class Clasa_Derivată : [modificatori de acces] Clasa_de_Bază1,
[modificatori de acces] Clasa_de_Bază2, [modificatori de acces]
Clasa_de_Bază3*

3. Moștenire multiplă și virtuală

Moștenire multiplă (MM)

- dar dacă avem nevoie doar de o copie a lui i?
- nu vrem să consumăm spațiu în memorie;
- *folosim moștenire virtuală:*

```
class base { public:    int i; };  
class derived1 : virtual public base { public:    int j; };  
class derived2 : virtual public base { public:    int k; };  
class derived3 : public derived1, public derived2 {public:    int sum; };
```

- Dacă avem moștenire de două sau mai multe ori dintr-o clasă de bază (fiecare moștenire trebuie să fie virtuală) atunci compilatorul alocă spațiu pentru o singură copie;
- În clasele derived1 și 2 moștenirea e la fel ca mai înainte (niciun efect pentru virtual în acel caz)
- Dar... fiecare moștenire virtuală crește sizeof-ul unui obiect al acelei clase

3. Moștenire multiplă și virtuală

Moștenire multiplă (MM)

- dacă în clasa de bază avem doar constructor cu parametri, derivatele trebuie să apeleze explicit acest constructor
 - de ce? Clasa de bază se inițializează o singură dată, la început

```
class base {    int i; public: base(int z) : i(z) {} };
class derived1 : virtual public base {    int j; public: derived1() : base(1) {} };
class derived2 : virtual public base {    int k; public: derived2() : base(2) {} };
class derived3 : public derived1, public derived2 {
    int sum;
public:
    derived3() : base(3), derived1(), derived2() {} };
```


3. Moștenire multiplă și virtuală

Moștenire multiplă (MM)

- În exemplul următor afișăm attributele din derivate printr-un pointer de bază folosind o implementare naivă a funcției virtuale de afișare

```
#include <iostream>
class Baza {
    int i{1};
protected: virtual void afis(std::ostream& os) const {    os << "i: " << i << "\n";    }
public:
    virtual ~Baza() = default;
    friend std::ostream& operator<<(std::ostream& os, const Baza& b) {    b.afis(os);    return os;    } };

class Der1 : public virtual Baza {
    int j{2};
protected: void afis(std::ostream& os) const override { Baza::afis(os); os << "j: " << j << "\n"; } };

class Der2 : public virtual Baza {
    int k{3};
protected: void afis(std::ostream& os) const override { Baza::afis(os); os << "k: " << k << "\n"; } };
```

3. Moștenire multiplă și virtuală

Moștenire multiplă (MM)

- Observăm că atributele din clasa de bază se afișează de două ori

```
class Der3 : public virtual Der1, public virtual Der2 {
    int l{4};
    void afis(std::ostream& os) const override {
        Der1::afis(os);
        Der2::afis(os);
        os << "l: " << l << "\n";
    }
};

int main() {
    Baza* b = new Der3;
    std::cout << *b;
    delete b;
}

// se va afișa
i: 1
j: 2
i: 1
k: 3
l: 4
```

3. Moștenire multiplă și virtuală

Moștenire multiplă (MM)

- Soluție: aplicăm ideea de interfață non-virtuală (NVI)
- `operator<<` este funcția publică non-virtuală
- Este suficient să modificăm clasa de bază ca mai jos, iar în `Der1` și `Der2` să eliminăm apelul `Baza::afis()`:

```
class Baza {
    int i{1};
protected:
    virtual void afis(std::ostream& os) const {}
public:
    virtual ~Baza() = default;
    friend std::ostream& operator<<(std::ostream& os, const Baza& b) {
        os << "i: " << b.i << "\n";
        b.afis(os);
        return os;
    }
};
```

// se va afișa
i: 1
j: 2
k: 3
l: 4

4. Tratarea excepțiilor în C++

În urma execuției unui program pot apărea diverse erori.

Câteva mecanisme de tratare a erorilor:

- Coduri de eroare
- Aserțiuni
- Excepții
- [Tipuri de date rezultat](#) (vezi anul 3 semestrul 2)

Excepțiile (în C++) pot fi cauzate

- în mod implicit de către limbaj (alocare dinamică) și de funcții din biblioteca standard (argumente invalide, erori de conversie)
- în mod explicit de către noi

Scop: simplificarea tratării erorilor

Sintaxă: într-un bloc try/catch prindem excepții aruncate cu throw, iar în fiecare clauză catch tratăm un anumit tip de eroare

4. Tratarea excepțiilor în C++

```
try {  
    // try block  
}  
catch (type1 arg) {  
    // catch block  
}  
catch (type2 arg) {  
    // catch block  
}  
catch (type3 arg) {  
    // catch block  
}...  
catch (typeN arg) {  
    // catch block  
}
```

tipul argumentului arg din catch arată care bloc catch este executat

dacă nu este generată excepție, nu se execută nici un bloc catch

instrucțiunile catch sunt verificate în ordinea în care sunt scrise, primul de tipul erorii este folosit

4. Tratarea excepțiilor în C++

Observații:

- dacă se face throw și nu există un bloc try din care a fost aruncată excepția sau o funcție apelată dintr-un bloc try: eroare
- dacă nu există un catch care să fie asociat cu throw-ul respectiv (tipuri de date egale sau compatibile) atunci programul se termină prin terminate()
- terminate() poate să fie redefinită să facă altceva
- Nu se recomandă folosirea excepțiilor dacă locul unde are loc eroarea este foarte apropiat de catch-ul asociat
 - Mai simplu și mai clar folosind coduri de eroare

4. Tratarea excepțiilor în C++

Toate excepțiile standard moștenesc din `std::exception`

Multe dintre ele sunt în headerele `<exception>` sau `<stdexcept>`

Standard exceptions

- `logic_error`
 - `invalid_argument`
 - `domain_error`
 - `length_error`
 - `out_of_range`
 - `future_error` (since C++11)
- `runtime_error`
 - `range_error`
 - `overflow_error`
 - `underflow_error`
 - `regex_error` (since C++11)
 - `system_error` (since C++11)
 - `ios_base::failure` (since C++11)
 - `filesystem::filesystem_error` (since C++17)
 - `tx_exception` (TM TS)
 - `nonexistent_local_time` (since C++20)
 - `ambiguous_local_time` (since C++20)
 - `format_error` (since C++20)

4. Tratarea excepțiilor în C++

- aruncarea de erori din clase de bază și derivate
- un catch pentru tipul de bază va fi executat pentru un obiect aruncat de tipul derivat
- să se pună catch-ul pe tipul derivat primul și apoi catchul pe tipul de bază

```
class B { };
class D: public B { };
int main()
{
    D derived;
    try {    throw derived;  }
    catch(B b) {    cout << "Caught a base class.\n";  }
    catch(D d) {    cout << "This won't execute.\n";  }
    // primim warning la compilare
    return 0;
}
```


4. Tratarea excepțiilor în C++

Când ar trebui să aruncăm excepții?

- În constructori și în funcții care aruncă obiecte dacă obiectul rezultat nu ar fi valid
 - o împiedicăm construirea unui obiect invalid
 - o Execuția sare la primul catch care se potrivește
- Atunci când alternativa cu coduri de eroare este mai complicată
- Atunci când codul este mai clar de înțeles cu excepții decât fără
 - o Dacă avem separare clară între [happy path și bad path](#)
- [Dacă avem voie](#)

Ce punem în catch?

- Prindem excepțiile prin referință (const dacă nu modificăm)
- Încercăm să găsim un echilibru între a prinde doar erori cât mai generale (cod simplu) și erori specifice (util la depanare)

4. Tratarea excepțiilor în C++

- Limbajul C++ ne permite să ne definim o ierarhie de excepții de la zero
- De obicei nu este recomandat, deoarece modulul scris de noi va trebui tratat în mod special față de restul codului
- Prima idee: moștenim din `std::runtime_error` (sau `logic_error`)
 - Putem moșteni direct din `std::exception`, dar nu avem constructor cu mesaj de eroare
- Problemă: nu putem face distincția dintre excepții aruncate de codul nostru și excepții aruncate de biblioteca standard (sau de alte module/biblioteci)

4. Tratarea excepțiilor în C++

```
#include <fmi>
class StudentError : public std::runtime_error {
    using std::runtime_error::runtime_error;
};
class TeacherError : public std::runtime_error {
    using std::runtime_error::runtime_error;
};
int main() {
    try {
        Student st1, st2; Teacher t1, t2;
        st1.studyLate(); t1.prepareLate(); // might throw StudentError/TeacherError
        // might throw PPTError : public std::runtime_error
        FMIComputer::loadPPT(t1.getLecture());
        st2.studyLate(); t2.prepareLate(); // might throw StudentError/TeacherError
    } catch(std::runtime_error& err) {
        std::cout << err.what() << "\n"; // am prins eroare din codul nostru sau din <fmi>?
    }
}
```

4. Tratarea excepțiilor în C++

Soluție: fiecare modul ar trebui să aibă o ierarhie proprie de excepții cu baza derivată direct sau indirect din `std::exception`.

Ierarhia noastră devine următoarea:

```
class SchoolError : public std::runtime_error { using std::runtime_error::runtime_error; };  
class StudentError : public SchoolError { using SchoolError::SchoolError; };  
class TeacherError : public SchoolError { using SchoolError::SchoolError; };
```

4. Tratarea excepțiilor în C++

Soluție: fiecare modul ar trebui să aibă o ierarhie proprie de excepții cu baza derivată direct sau indirect din `std::exception`.

Codul din main:

```
int main() {
    try {
        Student st1, st2; Teacher t1, t2;
        st1.studyLate(); t1.prepareLate(); // might throw StudentError/TeacherError
        FMIComputer::loadPPT(t1.getLecture()); // might throw PPTError : public FMLError
        st2.studyLate(); t2.prepareLate(); // might throw StudentError/TeacherError
    } catch(SchoolError& err) {
        std::cout << err.what() << "\n";
    } catch(FMLError& err) {
        std::cout << err.what() << "\n";
    } catch(std::runtime_error& err) {
        std::cout << err.what() << "\n"; // alte erori
    }
}
```

4. Tratarea excepțiilor în C++

Observații:

void Xhandler1(int test) noexcept

void Xhandler2(int test) noexcept(false)

- se poate specifica dacă o funcție aruncă excepții sau nu
- re-aruncarea unei excepții: `throw; // fără excepție din catch`
 - Util pentru handlers care tratează erori comune
 - Atenție! `throw err;` efectuează o copie prin valoare
 - Poate cauza object slicing (la fel catch prin valoare)
 - Dacă totuși avem nevoie: <https://isocpp.org/wiki/faq/exceptions#throwing-polymorphically>

4. Tratarea excepțiilor în C++

Rearuncarea unei excepții

- re-aruncarea unei excepții: `throw;` // fără excepție din catch

Avem ierarhia următoare:

```
class AppError : public std::runtime_error {  
    using std::runtime_error::runtime_error;  
};  
class SomeCommonError : public AppError {  
    using AppError::AppError;  
};  
class OtherCommonError : public AppError {  
    using AppError::AppError;  
};  
class SomeSpecialError : public AppError {  
    using AppError::AppError;  
};  
class OtherSpecialError : public AppError {  
    using AppError::AppError;  
};
```

4. Tratarea excepțiilor în C++

Rearuncarea unei excepții

```
void handleCommonErrors() {  
    try {  
        throw;  
    } catch(SomeCommonError& err) { /* handle error */  
    } catch(OtherCommonError& err) { /* handle error */  
    } catch(AppError& err) { /* handle error */  
    }  
}
```

```
void f() {  
    try {  
        // cod care aruncă diverse erori  
    } catch(SomeSpecialError& err) { /* handle error */  
    } catch(OtherSpecialError& err) { /* handle error */  
    } catch(...) { handleCommonErrors(); }  
}
```


Perspective

Cursul 9:

Şabloane



Programare orientată pe obiecte

- suport de curs -

Andrei Păun
Anca Dobrovăț

An universitar 2025 – 2026
Semestrul II
Seriile 13, 14 și 15

Curs 10



Agenda cursului

1. Pointeri și referințe

2. Const

3. Mutable

4. Static



Pointerii în C/C++

- O variabilă care ține o adresă din memorie
- Are un tip, compilatorul știe tipul de date către care se pointează
- Operațiile aritmetice țin cont de tipul de date din memorie
- `Pointer ++ == pointer+sizeof(tip)`
- Definiție: `tip *nume_pointer;`
 - Merge și `tip* nume_pointer;`



Operatori pe pointeri

- *, &, schimbare de tip (*cast*)
- *== “la adresa”
- &== “adresa lui”

```
int i=7, *j;
```

```
j=&i;
```

```
*j=9;
```



Pe tipuri predefinite

```
int main()
{
    double x = 100.1, y;
    int *p;
    p = (int *) &x;
    y = *p;
    cout<<y; /// nu va afisa 100.1
    return 0;
}
```

- Schimbarea de tip nu e controlată de compilator
- În C++ conversiile trebuiesc făcute cu schimbarea de tip

Pe tipuri definite de utilizator

```
class Carte{ public: void afis(){cout<<"carte";}};
class Articol{public: void afis(){cout<<"articol";}};

int main()
{
    Carte x,y;
    Articol* p = (Articol*) &x;
    p->afis();
    y = *p;
    y.afis();
}
```

- Atentie la necesitatea supraincercarii operatorilor (=, cast etc)



Aritmetica pe pointeri

- `pointer++`; `pointer--`;
- `pointer+7`;
- `pointer-4`;
- `pointer1-pointer2`; întoarce un întreg
- comparații: `<`, `>`, `==`, etc.



pointeri și array-uri

- numele array-ului este pointer
- `lista[5]==*(lista+5)`
- array de pointeri, numele listei este un pointer către pointeri (dublă indirectare)
- `int **p;` (dublă indirectare)



C++: Array-uri de obiecte

- o clasă de un tip
- putem crea array-uri cu date de orice tip (inclusiv obiecte)
- se pot defini neinițializate sau inițializate

clasa lista[10];

sau

clasa lista[10]={1,2,3,4,5,6,7,8,9,0};

pentru cazul inițializat dat avem nevoie de constructor care primește un parametru întreg.



```
#include <iostream>
using namespace std;

class cl {
    int i;
public:
    cl(int j) { i=j; } // constructor
    int get_i() { return i; }
};

int main()
{
    cl ob[3] = {1, 2, 3}; // initializers
    int i;

    for(i=0; i<3; i++)
        cout << ob[i].get_i() << "\n";
    return 0;
}
```



- inițializare pentru constructori cu mai mulți parametri

```
clasa lista[3]={clasa(1,5), clasa(2,4), clasa(3,3)};
```



- pentru definirea listelor de obiecte neinițializate:
constructor fără parametri
- dacă în program vrem și inițializare și neinițializare:
overload pe constructor (cu și fără parametri)



pointeri către obiecte

- obiectele sunt în memorie
- putem avea pointeri către obiecte
- &obiect;
- accesarea membrilor unei clase:
-> în loc de .



- în C++ tipurile pointerilor trebuie să fie la fel

```
class Carte{ public: void afis(){cout<<"carte";}};
class Articol{public: void afis(){cout<<"articol";}};

int main(){
    int *x;
    float *y;
    x = y; /// eroare ---> nu se face conversie automata

    Carte *x;
    Articol* y;
    x = y; /// eroare ---> nu se face conversie automata
}
```

se poate face cu schimbarea de tip (type casting) dar
ieșim din verificările automate făcute de C++



pointerul **this**

- orice funcție membru are pointerul **this** (definit ca argument implicit) care arată către obiectul asociat cu funcția respectivă
- (pointer către obiecte de tipul clasei)
- funcțiile prieten nu au pointerul **this**
- funcțiile statice nu au pointerul **this**



pointeri către clase derivate

- clasa de bază B și clasa derivată D
- un pointer către B poate fi folosit și cu D;

```
B *p, o(1);
```

```
D oo(2);
```

```
p=&o;
```

```
p=&oo;
```




```
#include <iostream>
using namespace std;

class base {
    int i;
public:
    void set_i(int num) { i=num; }
    int get_i() { return i; }
};

class derived: public base {
    int j;
public:
    void set_j(int num) { j=num; }
    int get_j() { return j; }
};
```

```
int main()
{
    base *bp;
    derived d;
    bp = &d;

    bp->set_i(10);
    cout << bp->get_i() << " ";
    .

    bp->set_j(88); // error
    cout << bp->get_j(); // error

    ((derived *)bp)->set_j(88);
    cout << ((derived *)bp)->get_j();

    return 0;
}
```



pointeri către clase derivate

- de ce merge și pentru clase derivate?
 - pentru că acea clasă derivată funcționează ca și clasa de bază plus alte detalii
- aritmetica pe pointeri: nu funcționează dacă incrementăm un pointer către bază și suntem în clasa derivată
- se folosesc pentru polimorfism la execuție (funcții virtuale)



// This program contains an error.

```
#include <iostream>
```

```
using namespace std;
```

```
class base {
```

```
    int i;
```

```
public:
```

```
    void set_i(int num) { i=num; }
```

```
    int get_i() { return i; }
```

```
};
```

```
class derived: public base {
```

```
    int j;
```

```
public:
```

```
    void set_j(int num) {j=num;}
```

```
    int get_j() {return j;}
```

```
};
```

```
int main()
```

```
{
```

```
    base *bp;
```

```
    derived d[2];
```

```
    bp = d;
```

```
    d[0].set_i(1);
```

```
    d[1].set_i(2);
```

```
    cout << bp->get_i() << " ";
```

```
    bp++; // relative to base, not derived
```

```
    cout << bp->get_i(); // garbage value  
    displayed
```

```
    return 0;
```

```
}
```



pointeri către membri în clase

- pointer către membru
- nu sunt pointeri normali (către un membru dintr-un obiect) ci specifică un offset în clasă
- nu putem să aplicăm . și ->
- se folosesc .* și ->*



```
#include <iostream>
using namespace std;
class cl { int val;
public:
    cl(int i) { val=i; }
    int val;
    int double_val() { return val+val; }
};

int main()
{
    int cl::*data; // data member pointer
    int (cl::*func)(); // function member pointer
    cl ob1(1), ob2(2); // create objects
    data = &cl::val; // get offset of val
    func = &cl::double_val; // get offset of
                                //double_val()

    cout << "Here are values: ";
    cout << ob1.*data << " " << ob2.*data << "\n";
    cout << "Here they are doubled: ";
    cout << (ob1.*func)() << " ";
    cout << (ob2.*func)() << "\n";
    return 0;
}
```

```
#include <iostream>
using namespace std;
class cl { int val;
public: cl(int i) { val=i; }
        int val;
        int double_val() { return val+val; }
};

int main(){
    int cl::*data; // data member pointer
    int (cl::*func)(); // function member
                        //pointer
    cl ob1(1), ob2(2), *p1, *p2;
    p1 = &ob1; // access objects through a
                //pointer
    p2 = &ob2;
    data = &cl::val; // get offset of val
    func = &cl::double_val;
    cout << "Here are values: ";
    cout << p1->*data << " " << p2->*data;
    cout << "\n";
    cout << "Here they are doubled: ";
    cout << (p1->*func)() << " ";
    cout << (p2->*func)() << "\n";
    return 0;
}
```



```
int cl::*d;  
int *p;  
cl o;
```

```
p = &o.val // this is address of a  
specific val  
d = &cl::val // this is offset of generic  
val
```

- pointeri la membri nu sunt folosiți decât rar
în cazuri speciale



parametri referință

- nou la C++
- la apel prin valoare se adaugă și apel prin referință la C++
- nu mai e nevoie să folosim pointeri pentru a simula apel prin referință, limbajul ne dă acest lucru
- sintaxa: în funcție & înaintea parametrului formal



// Manually: call-by-reference using a pointer.

```
#include <iostream>
using namespace std;
```

```
void neg(int *i);
```

```
int main()
{
    int x;
    x = 10;
    cout << x << " negated is ";
    neg(&x);
    cout << x << "\n";
    return 0;
}
```

```
void neg(int *i)
{
    *i = -*i;
}
```

// Use a reference parameter.

```
#include <iostream>
using namespace std;
```

```
void neg(int &i); // i now a reference
```

```
int main()
{
    int x;
    x = 10;
    cout << x << " negated is ";
    neg(x); // no longer need the &
operator
    cout << x << "\n";
    return 0;
}
```

```
void neg(int &i)
{
    i = -i; // i is now a reference,
don't need *
}
```




```
#include <iostream>
using namespace std;

void swap(int &i, int &j);

int main()
{
    int a, b, c, d;
    a = 1;      b = 2;      c = 3;      d = 4;
    cout << "a and b: " << a << " " << b << "\n";
    swap(a, b); // no & operator needed
    cout << "a and b: " << a << " " << b << "\n";
    cout << "c and d: " << c << " " << d << "\n";
    swap(c, d);
    cout << "c and d: " << c << " " << d << "\n";
    return 0;
}

void swap(int &i, int &j)
{
    int t;
    t = i; // no * operator needed
    i = j;
    j = t;
}
```



referințe către obiecte

- dacă transmitem obiecte prin apel prin referință la funcții nu se mai creează noi obiecte temporare, se lucrează direct pe obiectul transmis ca parametru
- deci copy-constructorul și destructorul nu mai sunt apelate
- la fel și la întoarcerea din funcție a unei referințe



```
#include <iostream>
using namespace std;
class cl {
    int id;
public:
    int i;
    cl(int i);
    ~cl(){ cout << "Destructing " << id << "\n"; }
    void neg(cl &o) { o.i = -o.i; } // no temporary created
};

cl::cl(int num)
{
    cout << "Constructing " << num << "\n";
    id = num;
}

int main()
{
    cl o(1);
    o.i = 10;
    o.neg(o);
    cout << o.i << "\n";
    return 0;
}
```

Constructing 1
-10
Destructing 1



Întoarcere de referințe

```
#include <iostream>
using namespace std;

char &replace(int i); // return a reference

char s[80] = "Hello There";

int main()
{
    replace(5) = 'X'; // assign X to space
    after Hello
    cout << s;
    return 0;
}

char &replace(int i)
{
    return s[i];
}
```

- putem face atribuiri către apel de funcție
- `replace(5)` este un element din `s` care se schimbă
- e nevoie de atenție ca obiectul referit să nu iasă din scopul de vizibilitate



referințe independente

- nu e asociat cu apelurile de funcții
- se creează un alt nume pentru un obiect
- referințele independente trebuie să fie inițializate la definire pentru că ele nu se schimbă în timpul programului



```
#include <iostream>
using namespace std;
```

```
int main()
{
    int a;
    int &ref = a; // independent reference
    a = 10;
    cout << a << " " << ref << "\n";
    ref = 100;
    cout << a << " " << ref << "\n";
    int b = 19;
    ref = b; // this puts b's value into a
    cout << a << " " << ref << "\n";
    ref--; // this decrements a
    // it does not affect what ref refers to
    cout << a << " " << ref << "\n";
    return 0;
}
```

```
10 10
100 100
19 19
18 18
```



referințe independente ca membri ai unei clase

```
class Carte{
    int& cod; /// nu trebuie initializata aici
public:
    Carte(int c = 0) : cod(c) {cout<<"cod "<<c++<<" "<<cod<<" ";}
    void afis() {cout<<cod<<endl;}
};

int main()
{
    Carte a(100);
    a.afis();
    Carte b(200);
    b.afis();
    a.afis();
}
```

```
cod 100 101 101
cod 200 201 201
201
```

Atentie: cod este referinta, deci orice il va modifica, va fi aratat de catre toate obiectele



referințe către clase derivate

- putem avea referințe definite către clasa de bază și apelată funcția cu un obiect din clasa derivată
- exact ca la pointeri



alocare de obiecte

- cu new
- după creare, new întoarce un pointer către obiect
- după creare se execută constructorul obiectului
- când obiectul este șters din memorie (delete) se execută destructorul



obiecte create dinamic cu constructori parametrizați

```
class balance {...}  
...  
  
balance *p;  
// this version uses an initializer  
try {  
    p = new balance (12387.87, "Ralph Wilson");  
} catch (bad_alloc xa) {  
    cout << "Allocation Failure\n";  
    return 1;  
}
```



- array-uri de obiecte alocate dinamic
 - nu se pot inițializa
 - trebuie să existe un constructor fără parametri
 - delete poate fi apelat pentru fiecare element din array



- new și delete sunt operatori
- pot fi suprascriși pentru o anumită clasă
- pentru argumente suplimentare există o formă specială
 - `p_var = new (lista_argumente) tip;`
- există forma nothrow pentru new: similar cu malloc:
`p=new(nothrow) int[20]; // intoarce null la eroare`



const și volatile

- idee: să se elimine comenzile de preprocesor #define
- #define făceau substituție de valoare
- se poate aplica la pointeri, argumente de funcții, param de întoarcere din funcții, obiecte, funcții membru
- fiecare dintre aceste elemente are o aplicare diferită pentru const, dar sunt în aceeași idee/filosofie



- `#define BUFSIZE 100` (tipic in C)
- erori subtile datorită substituirii de text
- BUFSIZE e mult mai bun decât “valori magice”
- nu are tip, se comportă ca o variabilă
- mai bine: `const int bufsize = 100;`



- acum compilatorul poate face calculele la început:
“constant folding”: important pt. array: o expresie complicată e calculată la compilare
- `char buf[bufsize];`
- se poate face const pe: **char, int, float, double** și variantele lor
- se poate face const și pe obiecte



- const implică “***internal linkage***” adică e vizibilă numai în fișierul respectiv (la linkare)
- trebuie dată o valoare pentru elementul constant la declarare, singura excepție:

extern const int bufsiz;

- în mod normal compilatorul nu alocă spațiu pentru constante, dacă e declarat ca extern alocă spațiu (să poată fi accesat și din alte părți ale programului)



- pentru structuri complicate folosite cu `const` se alocă spațiu: nu se știe dacă se alocă sau nu spațiu și atunci `const` impune localizare (să nu existe coliziuni de nume)
- de aceea avem “*internal linkage*”



```
// Using const for safety
#include <iostream>
using namespace std;

const int i = 100; // Typical constant
const int j = i + 10; // Value from const expr
long address = (long)&j; // Forces storage
char buf[j + 10]; // Still a const expression

int main() {
    cout << "type a character & CR:";
    const char c = cin.get(); // Can't change //valoarea
    nu e cunoscuta la compile time si necesita storage
    const char c2 = c + 'a';
    cout << c2;
    // ...
}
```

- dacă știm că variabila nu se schimbă să o declaram cu const
- dacă încercăm să o schimbăm primim eroare de compilare



- `const` poate elimina memorie și acces la memorie
- `const` pentru agregate: aproape sigur compilatorul alocă memorie
- nu se pot folosi valorile la compilare



```
// Constants and aggregates
const int i[] = { 1, 2, 3, 4 };

//! float f[i[3]]; // Illegal

struct S { int i, j; };
const S s[] = { { 1, 2 }, { 3, 4 } };

//! double d[s[1].j]; // Illegal

int main() {}
```



diferențe cu C

- `const` în C: o variabilă globală care nu se schimbă
- deci nu se poate considera ca valoare la compilare

```
const int bufsize = 100;  
char buf[bufsize];
```

eroare în C



- în C se poate declara cu
const int bufsiz;
- în C++ nu se poate așa, trebuie extern:
extern const int bufsiz;
- diferența:
 - C external linkage
 - C++ internal linkage



- în C++ compilatorul încearcă să nu creeze spațiu pentru const-uri, dacă totuși se transmite către o funcție prin referință, extern etc. atunci se creează spațiu
- C++: const în afara tuturor funcțiilor: scopul ei este doar în fișierul respectiv: internal linkage,
- alți identificatori declarați în același loc (fara const)
EXTERNAL LINKAGE



pointeri const

- const poate fi aplicat valorii pointerului sau elementului pointat
- const se alătură elementului cel mai apropiat
`const int* u;`
- u este pointer către un int care este const
`int const* v;` la fel ca mai sus



pointeri constanți

- pentru pointeri care nu își schimbă adresa din memorie

```
int d = 1;
```

```
int* const w = &d;
```

- w e un pointer constant care arată către întregi+inițializare



const pointer catre const element

```
int d = 1;
```

```
const int* const x = &d; // (1)
```

```
int const* const x2 = &d; // (2)
```



```
//: C08:ConstPointers.cpp
const int* u;
int const* v;

int d = 1;

int* const w = &d;

const int* const x = &d; // (1)
int const* const x2 = &d; // (2)

int main() {} ///:~
```



- se poate face atribuire de adresă pentru obiect non-const către un pointer const
- nu se poate face atribuire pe adresă de obiect const către pointer non-const



```
int d = 1;
```

```
const int e = 2;
```

```
int* u = &d; // OK -- d not const
```

```
//! int* v = &e; // Illegal -- e const
```

```
int* w = (int*)&e; // Legal but bad practice
```

```
int main() {} ///:~
```



constante caractere

```
char* cp = "howdy";
```

dacă se încearcă schimbarea caracterelor din
“howdy” compilatorul ar trebui să genereze
eroare; nu se întâmplă în mod uzual
(compatibilitate cu C)

```
mai bine: char cp[] = "howdy";
```

și atunci nu ar mai trebui să fie probleme



argumente de funcții, param de întoarcere

- apel prin valoare cu const: **param formal nu se schimbă în funcție**
- const la întoarcere: **valoarea returnată nu se poate schimba**
- dacă se transmite o adresă: promisiune că nu se schimbă valoarea la adresa respectivă



```
void f1(const int i) {  
    i++; // Illegal -- compile-time error  
}
```

cod mai clar echivalent mai jos:

```
void f2(int ic) {  
    const int& i = ic;  
    i++; // Illegal -- compile-time error  
}
```




```
// Returning consts by value  
// has no meaning for built-in types
```

```
int f3() { return 1; }  
const int f4() { return 1; }
```

```
int main() {  
    const int j = f3(); // Works fine  
    int k = f4(); // But this works fine too!  
}
```



```
// Constant return by value
// Result cannot be used as an lvalue

class X {    int i;
public:    X(int ii = 0);
void modify();
};

X::X(int ii) { i = ii; }
void X::modify() { i++; }

X f5() {    return X(); }
const X f6() {    return X(); }

void f7(X& x) { // Pass by non-const
reference
    x.modify();
}

int main() {
    f5() = X(1); // OK -- non-const return
value
    f5().modify(); // OK
// Causes compile-time errors:
//!  f7(f5());
//!  f6() = X(1);
//!  f6().modify();
//!  f7(f6());
} ///:~
```



- `f7()` creează obiecte temporare, de aceea nu compilează
- aceste obiecte au constructor și destructor dar pentru că nu putem să le “atingem” sunt definite sub forma de obiecte constante
- `f7(f5())`; se creează ob. temporar pentru rezultatul lui `f5()`; și apoi apel prin referință la `f7`
- ca să compileze (dar cu erori mai târziu) trebuie apel prin referință const



- `f5() = X(1);`
- `f5().modify();`
- compilează fără probleme, dar procesarea se face pe obiectul temporar (modificările se pierd imediat, deci aproape sigur este bug)



parametrii de intrare și iesire: adrese

- e preferabil să fie definiți ca const
- în felul acesta pointerii și referințele nu pot fi modificați/modificate



```
// Constant pointer arg/return
```

```
void t(int*) {}

void u(const int* cip) {
    //! *cip = 2; // Illegal -- modifies value
    int i = *cip; // OK -- copies value
    //! int* ip2 = cip; // Illegal: non-const
}

const char* v() {
    // Returns address of static character
    array:
    return "result of function v()";
}

const int* const w() {
    static int i;
    return &i;
}
```

```
int main() {
    int x = 0;
    int* ip = &x;
    const int* cip = &x;
    t(ip); // OK
    //! t(cip); // Not OK
    u(ip); // OK
    u(cip); // Also OK
    //! char* cp = v(); // Not OK
    const char* ccp = v(); // OK
    //! int* ip2 = w(); // Not OK
    const int* const ccip = w(); // OK
    const int* cip2 = w(); // OK
    //! *w() = 1; // Not OK
} ///:~
```

- cip2 nu schimbă adresa întoarsă din w (pointerul constant care arată spre constantă) deci e ok; următoarea linie schimbă valoarea deci compilatorul intervine



comparații cu C

- în C dacă vrem param. o adresa: se face pointer la pointer
- în C++ nu se încurajează acest lucru: const referință
- pentru apelant e la fel ca apel prin valoare
 - nici nu trebuie să se gândească la pointeri
 - trimiterea unei adrese e mult mai eficientă decât transmiterea obiectului prin stivă, se face const deci nici nu se modifică



Ob. temporare sunt const

- e posibil să se transmită un obiect temporar către o funcție care primește referință const

```
//: C08:ConstTemporary.cpp
// Temporaries are const

class X {};

X f() { return X(); } // Return by value

void g1(X&) {} // Pass by non-const reference
void g2(const X&) {} // Pass by const reference

int main() {
    // Error: const temporary created by f():
    //! g1(f());
    // OK: g2 takes a const reference:
    g2(f());
} ///:~
```




- Obiectele temp sunt const (exemplu problema examen scris)

```
#include <iostream.h>
class problema
{   int i;
    public: problema(int j=5){i=j;}
        void schimba(){i++;}
        void afiseaza(){cout<<"starea curenta "<<i<<"\n";}
};
problema mister1() { return problema(6);}
void mister2(problema &o)
{   o.afiseaza();
    o.schimba();
    o.afiseaza();
}
int main()
{   mister2(mister1());
    return 0;
}
```



Const în clase

- const pentru variabile de instanță și
- funcții de instanță de tip const
- să construim un vector pentru clasa respectivă, în C folosim #define
- problemă în C: coliziune pe nume



- în C++: punem o variabilă de instanță const
- problemă: toate obiectele au această variabilă, și putem avea chiar valori diferite (depinde de inițializare)
- când se creează un const într-o clasă nu se poate inițializa (constructorul inițializează)
- în constructor trebuie să fie deja inițializat (altfel am putea să îl schimbăm în constructor)



- inițializare de variabile const în obiecte: lista de inițializare a constructorilor

```
// Initializing const in classes
#include <iostream>
using namespace std;

class Fred {
    const int size;
public:
    Fred(int sz);
    void print();
};

Fred::Fred(int sz) : size(sz) {}
void Fred::print() { cout << size << endl; }

int main() {
    Fred a(1), b(2), c(3);
    a.print(), b.print(), c.print();
} ///:~
```



rezolvarea problemei inițiale

- cu static
 - înseamnă că nu e decât un singur asemenea element în clasă
 - îl facem static const și devine similar ca un const la compilare
 - static const trebuie inițializat la declarare (nu în constructor)



```
#include <string>
#include <iostream>
using namespace std;

class StringStack {
    static const int size = 100;
    const string* stack[size];
    int index;
public:
    StringStack();
    void push(const string* s);
    const string* pop();
};

StringStack::StringStack() : index(0) {
    memset(stack, 0, size * sizeof(string*));
}

void StringStack::push(const string* s) {
    if(index < size)
        stack[index++] = s;
}

const string* StringStack::pop() {
    if(index > 0) {
        const string* rv = stack[--index];
        stack[index] = 0;
        return rv;
    }
    return 0;
}
```

```
string iceCream[] = {
    "pralines & cream",
    "fudge ripple",
    "jamocha almond fudge",
    "wild mountain blackberry",
    "raspberry sorbet",
    "lemon swirl",
    "rocky road",
    "deep chocolate fudge"
};

const int iCsz =
    sizeof iceCream / sizeof *iceCream;

int main() {
    StringStack ss;
    for(int i = 0; i < iCsz; i++)
        ss.push(&iceCream[i]);
    const string* cp;
    while((cp = ss.pop()) != 0)
        cout << *cp << endl;
}
```



enum hack

```
#include <iostream>
using namespace std;

class Bunch {
    enum { size = 1000 };
    int i[size];
};

int main() {
    cout << "sizeof(Bunch) = " << sizeof(Bunch)
         << ", sizeof(i[1000]) = "
         << sizeof(int[1000]) << endl;
}
```

- în cod vechi
- a nu se folosi cu C++ modern
- static const int size=1000;



obiecte const și funcții membru const

- obiecte const: nu se schimbă
- pentru a se asigura că starea obiectului nu se schimbă funcțiile de instanță apelabile trebuie definite cu const
- declararea unei funcții cu const nu garantează că nu modifică starea obiectului!



functii membru const

- compilatorul și linkerul cunosc faptul că funcția este const
- se verifică acest lucru la compilare
- nu se pot modifica părți ale obiectului în aceste funcții
- nu se pot apela funcții non-const



```
//: C08:ConstMember.cpp
class X {
    int i;
public:
    X(int ii);
    int f() const;
};

X::X(int ii) : i(ii) {}
int X::f() const { return i; }

int main() {
    X x1(10);
    const X x2(20);
    x1.f();
    x2.f();
} ///:~
```

Funcțiile const se pot apela atât de obiecte const cât și de obiecte neconst.



- toate funcțiile care nu modifică date să fie declarate cu `const`
- ar trebui ca “default” funcțiile membru să fie de tip `const`



```
#include <iostream>
#include <cstdlib> // Random number generator
#include <ctime> // To seed random generator
```

```
using namespace std;
```

```
class Quoter {
    int lastquote;
public:
    Quoter();
    int lastQuote() const;
    const char* quote();
};
```

```
Quoter::Quoter() {    lastquote = -1;
    srand(time(0)); // Seed random number
generator
}
```

```
int Quoter::lastQuote() const {    return
lastquote;}
```

```
const char* Quoter::quote() {
    static const char* quotes[] = {
        "Are we having fun yet?",
        "Doctors always know best",
        "Is it ... Atomic?",
        "Fear is obscene",
        "There is no scientific evidence "
        "to support the idea "
        "that life is serious",
        "Things that make us happy, make us wise",
    };
    const int qsize = sizeof quotes/sizeof *quotes;
    int qnum = rand() % qsize;
    while(lastquote >= 0 && qnum == lastquote)
        qnum = rand() % qsize;
    return quotes[lastquote = qnum];
}

int main() {
    Quoter q;
    const Quoter cq;
    cq.lastQuote(); // OK
    //! cq.quote(); // Not OK; non const function
    for(int i = 0; i < 20; i++)
        cout << q.quote() << endl;
} ///:~
```



schimbări în obiect din funcții const

- “casting away constness”
- se face castare a pointerului `this` la pointer către tipul de obiect
- pentru că în funcții `const` este de tip clasa `const*`
- după această schimbare de tip se modifica prin pointerul `this`



```
// "Casting away" constness

class Y {
    int i;
public:
    Y();
    void f() const;
};

Y::Y() { i = 0; }

void Y::f() const {
    //! i++; // Error -- const member function
    ((Y*)this)->i++; // OK: cast away const-ness
    // Better: use C++ explicit cast syntax:
    (const_cast<Y*>(this))->i++;
}

int main() {
    const Y yy;
    yy.f(); // Actually changes it!
}
```



- apare în cod vechi
- nu e ok pentru că funcția modifică și noi credem că nu modifică
- o metodă mai bună: în continuare



```
// The "mutable" keyword

class Z {
    int i;
    mutable int j;
public:
    Z();
    void f() const;
};

Z::Z() : i(0), j(0) {}

void Z::f() const {
    //! i++; // Error -- const member function
    j++; // OK: mutable
}

int main() {
    const Z zz;
    zz.f(); // Actually changes it!
}
```




Tot despre “mutable” (rest de discutii de la lambda expresii)

“Mutable” vs transmiterea prin referinta

```
int main()
{
    int i = 10;
    ///      auto A = [i]() { i = i*2; cout<<"local "<<i<<endl; return i;};
    /// i este transmis prin valoare, nu poate fi modificat
    auto A = [i]() mutable { i = i*2; cout<<"local "<<i<<endl; return i;};
    int j = A();
    cout<<i<<endl;
    i = 25; j = A(); cout<<i<<endl;
    i = 30; j = A(); cout<<i<<endl;
}
```

```
local 20
10
local 40
25
local 80
30
```

```
int main()
{
    int i = 10;
    auto A = [&i]() mutable { i = i*2; cout<<"local "<<i<<endl; return i;};
    int j = A();
    cout<<i<<endl;
    i = 25; j = A(); cout<<i<<endl;
    i = 30; j = A(); cout<<i<<endl;
}
```

```
local 20
20
local 50
50
local 60
60
```



Tot despre “mutable” (rest de discutii de la lambda expresii)

```
int main()
{
    int i = 8;
    auto f = [i]() mutable
    {
        int j = 2;
        auto m = [&i, j]() mutable { i /= j; };
        m();
        cout<<"inner " << i << endl;
    };
    f();
    cout<<"outer " << i << endl;
}
```

“*copia*” e mutable, deci modificabila, dar, totusi, o copie
i = 4 – interior
i = 8 – exterior



Tot despre “mutable” (rest de discutii de la lambda expresii)

```
int main()
{
    int i=1, j=2, k=3;
    auto f = [i, &j, &k]() mutable {
        auto m = [&i, j, &k]() mutable
            {i=4; j=5; k=6;};
        m();
        cout <<i<<" "<<j<<" "<<k;
    };
    f();
    cout <<" : "<<i<<" "<<j<<" "<<k;
}
```

4 2 6 : 1 2 6

- In f, i este transmis prin valoare, dar, mutable, deci doar copia se modifica
- In m, j este transmis prin valoare, dar, mutable, deci doar copia se modifica
- In ambele, k este transmis prin referinta, deci se modifica



volatile

- e similar cu const
- obiectul se poate schimba din afara programului

Expl: `volatile int x = 10; int a = x; int b = x;` (daca x este schimbat din afara programului, de exemplu de SO, atunci nu e garantat ca `a == b` intotdeauna)

- multitasking, multithreading, întreruperi
- nu se fac optimizări de cod

Expl: `int x = 10; while (x == 10) {};` deoarece x nu se schimba in while, compilatorul ar putea optimiza la `while(true){};`
daca `volatile int x = 10;` atunci nu face optimizare

- avem obiecte volatile, funcții volatile, etc.



static

- ceva care își tine poziția neschimbată
- alocare statică pentru variabile
- vizibilitate locală a unui nume



- variabile locale statice
- își mențin valorile între apelări
- inițializare la primul apel



```
#include "../require.h"
#include <iostream>
using namespace std;

char oneChar(const char* charArray = 0) {
    static const char* s;
    if(charArray) {
        s = charArray;
        return *s;
    }
    else
        require(s, "un-initialized s");
    if(*s == '\\0')
        return 0;
    return *s++;
}

char* a = "abcdefghijklmnopqrstuvwxyz";

int main() {
    // oneChar(); // require() fails
    oneChar(a); // Initializes s to a
    char c;
    while((c = oneChar()) != 0)
        cout << c << endl;
} ///:~
```



obiecte statice

- Sunt initializate o singura data, dar se distrug de-abia la finalizarea programului, indiferent unde apar (de exemplu, in functii)
- la fel ca la tipurile predefinite
- **avem nevoie de constructorul predefinit**



```
#include <iostream>
using namespace std;

class X {
    int i;
public:
    X(int ii = 0) : i(ii) {} // Default
    ~X() { cout << "X::~~X()" << endl; }
};

void f() {
    static X x1(47);
    static X x2; // Default constructor required
}

int main() {
    f();
} //::~
```



destructori statici

- când se termină main se distrug obiectele
- De obicei se cheamă `exit()` la ieșirea din main
- dacă se cheamă `exit()` din destructor e posibil să avem ciclu infinit de apeluri la `exit()`
- destructorii statici nu sunt executați dacă se iese prin `abort()`



- dacă avem o funcție cu obiect local static
- și funcția nu a fost apelată, nu vrem să apelăm destructorul pentru obiect neconstruit
- C++ ține minte care obiecte au fost construite și care nu



```
#include <fstream>
using namespace std;
ofstream out("statdest.out"); // Trace file
```

```
class Obj {
    char c; // Identifier
public:
    Obj(char cc) : c(cc) {
        out << "Obj::Obj() for " << c << endl;
    }
    ~Obj() {
        out << "Obj::~Obj() for " << c << endl;
    }
};
```

```
Obj a('a'); // Global (static storage)
// Constructor & destructor always called
```

```
void f() {
    static Obj b('b');
}
```

```
void g() {
    static Obj c('c');
}
```

```
int main() {
    out << "inside main()" << endl;
    f(); // Calls static constructor for b
    // g() not called
    out << "leaving main()" << endl;
} ///:~
```

Obj::Obj() for a
inside main()
Obj::Obj() for b
leaving main()
Obj::~Obj() for b
Obj::~Obj() for a



static pentru nume (la linkare)

- orice nume care nu este într-o clasă sau funcție este vizibil în celelalte părți ale programului (external linkage)
- dacă e definit ca static are internal linkage: vizibil doar în fișierul respectiv
- linkarea e valabilă pentru elemente care au adresă (clase, var. locale nu au)



- `int a=0;`
- în afara claselor, funcțiilor: este var globală, vizibilă pretutindeni
- similar cu: `extern int a=0;`
- `static int a=0; // internal linkage`
- nu mai e vizibilă pretutindeni, doar local în fișierul respectiv



```
//{L} LocalExtern2
#include <iostream>

int main() {
    extern int i;
    std::cout << i;
} ///:~

//: C10:LocalExtern2.cpp {0}
int i = 5;
///:~
```



funcții extern și static

- schimbă doar vizibilitatea
- `void f();` similar cu `extern void f();`
- restrictiv:
 - `static void f();`



- alți specificatori:
 - auto: aproape nefolosit; spune ca e var. locală
 - register: să se pună într-un registru



variabile de instanță statice

- când vrem să avem valori comune pentru toate obiectele
- static

```
class A {  
    static int i;  
public:  
    //...  
};  
int A::i = 1;
```

- `int A::i = 1;`
- se face o singura dată
- e obligatoriu să fie făcut de creatorul clasei, deci e ok



```
#include <iostream>
using namespace std;

int x = 100;

class WithStatic {
    static int x;
    static int y;
public:
    void print() const {
        cout << "WithStatic::x = " << x << endl;
        cout << "WithStatic::y = " << y << endl;
    }
};

int WithStatic::x = 1;
int WithStatic::y = x + 1;
// WithStatic::x NOT ::x

int main() {
    WithStatic ws;
    ws.print();
}
```



```
// Static members & local classes
#include <iostream>
using namespace std;

// Nested class CAN have static data members:
class Outer {
    class Inner {
        static int i; // OK
    };
};

int Outer::Inner::i = 47;

// Local class cannot have static data members:
void f() {
    class Local {
    public:
        //! static int i; // Error
        // (How would you define i?)
    } x;
}

int main() { Outer x; f(); } ///:~
```



funcții membru statice

- nu sunt asociate cu un obiect, nu au this

```
class X {  
    public:  
        static void f(){};  
};  
  
int main() {  
    X::f();  
} ///:~
```



Despre examen

- Descrieți pe scurt funcțiile șablon (template).

```
#include <iostream.h>
class problema
{   int i;
    public: problema(int j=5){i=j;}
           void schimba(){i++;}
           void afiseaza(){cout<<"starea
curenta "<<i<<"\n";}
};
problema mister1() { return problema(6); }
void mister2(problema &o)
{   o.afiseaza();
    o.schimba();
    o.afiseaza();
}
int main()
{   mister2(mister1());
    return 0;
}
```



```
#include<iostream.h>
class B
{ int i;
  public: B() { i=1; }
          virtual int get_i() { return i; }
};
class D: virtual public B
{ int j;
  public: D() { j=2; }
          int get_i() {return B::get_i()+j; }
};
class D2: virtual public B
{ int j2;
  public: D2() { j2=3; }
          int get_i() {return B::get_i()+j2; } };

class MM: public D, public D2
{ int x;
  public: MM() { x=D::get_i()+D2::get_i(); }
          int get_i() {return x; } };

int main()
{ B *o= new MM();
  cout<<o->get_i()<<"\n";
  MM *p= dynamic_cast<MM*>(o);
  if (p) cout<<p->get_i()<<"\n";
  D *p2= dynamic_cast<D*>(o);
  if (p2) cout<<p2->get_i()<<"\n";
  return 0;
}
```



```
#include <iostream.h>
#include <typeinfo>
class B
{ int i;
  public: B() { i=1; }
         int get_i() { return i; }
};

class D: B
{ int j;
  public: D() { j=2; }
         int get_j() {return j; }
};

int main()
{ B *p=new D;
  cout<<p->get_i();
  if (typeid((B*)p).name()=="D*")
  cout<<((D*)p)->get_j();
  return 0;
}
```




```
#include<iostream.h>
template<class T, class U>
T f(T x, U y)
{ return x+y;
}
int f(int x, int y)
{ return x-y;
}
int main()
{ int *a=new int(3), b(23);
  cout<<*f(a,b);
  return 0;
}
```



```
#include<iostream.h>
class A
{ int x;
  public: A(int i=0) { x=i; }
          A operator+(const A& a) { return x+a.x; }
          template <class T> ostream& operator<<(ostream&); };
template <class T>
ostream& A::operator<<(ostream& o) { o<<x; return o; }
int main()
{ A a1(33), a2(-21);
  cout<<a1+a2;
  return 0;
}
```



Perspective

Curs 12 – Design Patterns



Programare orientată pe obiecte

- suport de curs -

Andrei Păun
Anca Dobrovăț

An universitar 2024 – 2025
Semestrul II
Seriile 13, 14 și 15

Curs 10

Agenda cursului

Biblioteca Standard Template Library - STL

- Containere, iteratori și algoritmi.
- Clasele vector, list, map / multimap.
- Elemente avansate (lambda expresii)

Standard Template Library (STL)

-bibliotecă de clase C++, parte din Standard Library

Ofera:

- structuri de date și algoritmi fundamentali → dezvoltarea de programe în C++;
- componente generice, parametrizabile. Aproape toate clasele din STL sunt parametrizate (Template).

Componentele STL se pot compune cu ușurință fără a sacrifica performanța (generic programming)

STL conține clase pentru:

- **containere**
- **iteratori**
- **algoritmi**
- functori (function objects)
- allocators

Standard Template Library (STL)

Container

“Containers are the STL objects that actually store data” (H. Schildt)

- grupare de date în care se pot adauga (insera) si din care se pot sterge (extrage) obiecte;
- gestionează memoria necesară stocarii elementelor, oferă metode de acces la elemente (direct si prin iteratori);
- **funcționalități (metode):**
 - accesare elemente (ex.: [])
 - gestiune capacitate (ex.: size())
 - modificare elemente (ex.: insert, clear)
 - iterator (begin(), end())
 - alte operații (ie: find)

Standard Template Library (STL)

Container

Tipuri de containtere:

- ***de tip secventa*** (in terminologia STL, o secventa este o lista liniara):
 - vector
 - deque
 - list
- ***asociativi*** (permit regasirea eficienta a informatiilor bazandu-se pe chei)
 - set
 - multiset
 - map (permite accesul la informatii cu cheie unica)
 - multimap
- ***adaptor de containere***
 - stack
 - queue
 - priority_queue

Standard Template Library (STL)

Adaugarea si stergerea elementelor din containtere:

De tip secventa (vector, deque, list) si - **asociativi** (set, multiset, map, multimap):

- insert()
- erase()

De tip secventa (vector, deque, list) permit si:

- push_back()
- pop_back()

List , Deque

- pop_front()
- push_front()

Containerele asociative:

- find()

Accesarea uzuala: prin iteratori.

Standard Template Library (STL)

Containere de tip secventa

Vector (Dynamic Array)

Vector, Deque, List sunt containere de tip secvență, folosesc reprezentări interne diferite, astfel operațiile uzuale au complexități diferite.

template <class T, class Allocator = allocator<T>> class vector

T = tipul de data utilizat

Allocator = tipul de alocator utilizat (in general cel standard).

- elementele sunt stocate secvențial in zone continue de memorie.

Vector are performanțe bune la:

- Acesare elemente individuale de pe o poziție dată (constant time).
- Iterare elemente în orice ordine (linear time).
- Adăugare/Ștergere elemente de la sfârșit (constant amortized time).

Standard Template Library (STL)

Containere de tip secventa

Vector (Dynamic Array)

Construcitori:

explicit vector(const Allocator &a = Allocator());

*// expl. **vector<int> iv** - vector de int cu zero elemente;*

explicit vector(size_type num, const T &val = T (), const Allocator &a = Allocator());

*// expl. **vector<int> iv(10)** - vector de int cu 10 elemente;*

*// expl. **vector<int> iv(10,7)** - vector de int cu 10 elemente, fiecare egal cu 7;*

vector(const vector<T,Allocator> &ob);

*// expl. **vector<int> iv2(iv)** - vector de int reprezentand copia lui iv;*

template <class InIter>

vector(InIter start, InIter end, const Allocator &a = Allocator());

Standard Template Library (STL)

Containere de tip secventa

Vector (Dynamic Array)

Functii membre uzuale (sursa H. Schildt)

- **back()**: returneaza o referinta catre ultimul element;
- **front()**: returneaza o referinta catre primul element;
- **begin()**: returneaza un iterator catre primul element;
- **end()**: returneaza un iterator catre zona de memorie de dupa ultimul element;
- **clear()**: elimina toate elementele din vector.
- **empty()**: “true(false)” daca vectorul e (sau nu) gol.
- **erase(iterator i)**: stergerea elementului pointat de i; returneaza un iterator catre elementul de dupa cel sters.
- **erase(iterator start, iterator end)**: stergerea elementelor intre start si end.

Standard Template Library (STL)

Containere de tip secventa

Vector (Dynamic Array)

Functii membre uzuale (sursa H. Schildt)

- **insert(iterator i, const T &val)**: insereaza val inaintea elementului pointat;
- **insert(iterator i, size_type num, const T & val)** : insereaza un numar de “num” elemente de valoare “val” inaintea elementului pointat de i.
- **insert(iterator i, InIter start, InIter end)**: insereaza o secventa start-end inaintea elementului pointat de i.
- **operator[](size_type i)** : returneaza o referinta la elementul specificat de i;
- **pop_back()**: sterge ultimul element.
- **push_back(const T &val)**: adauga la final valoarea “val”
- **size()** : dimensiunea vectorului.

Standard Template Library (STL)

Containere de tip secventa

Vector (Dynamic Array)

Exemplu citirea unui vector cu accesarea elementelor prin iterator si prin operatorul []

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v(3);
    v[0] = 12;
    v[1] = 34;
    v[2] = 56;
    //v.resize(5); // necesar pentru reactualizarea dimensiunii vectorului
    v[3] = 78;
    v[4] = 90;
    for(int i = 0; i<v.size(); i++)
        cout<<v[i]<<" ";
}
```

Standard Template Library (STL)

Containere de tip secventa

Vector (Dynamic Array)

Exemplu citirea unui vector cu accesarea elementelor prin iterator si prin operatorul []

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v(3);
    v.push_back(12);
    v.push_back(34);
    v.push_back(56);
    v.push_back(78);
    v.push_back(90);
    for(int i = 0; i<v.size(); i++)
        cout<<v[i]<<" "; // Afisare 8 valori, resize automat
}
```

Standard Template Library (STL)

Containere de tip secventa Vector (Dynamic Array)

Exemplu

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<char> v(10);
    unsigned int i;
    cout << "Size = " << v.size() << endl;
    for(i=0; i<10; i++) v[i] = i + 'a';
    for(i=0; i<v.size(); i++) cout << v[i] << " ";
    cout << endl;
    for(i=0; i<10; i++) v.push_back(i + 10 + 'a');
    cout << "Size now = " << v.size() << endl;
    for(i=0; i<v.size(); i++) cout << v[i] << " ";
    cout << endl;
    for(i=0; i<v.size(); i++) v[i] = toupper(v[i]);
    for(i=0; i<v.size(); i++) cout << v[i] << " ";
}
```

```
Size = 10
a b c d e f g h i j
Size now = 20
a b c d e f g h i j k l m n o p q r s t
A B C D E F G H I J K L M N O P Q R S T
```


Standard Template Library (STL)

Container de tip secventa Vector (Dynamic Array)

Exemplu - accesarea unui vector cu iterator

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<char> v(10); //unsigned int i = 0;
    vector<char>::iterator p;

    for( p = v.begin(); p != v.end(); p++, i++)
        *p = i + 'a';

    for( p = v.begin(); p != v.end(); p++)
        cout << *p << " ";

    for( p = v.begin(); p != v.end(); p++)
        *p = toupper(*p);
}
```

Standard Template Library (STL)

Containere de tip secventa Vector (Dynamic Array)

Exemplu - inserarea si stergerea elementelor intr-un vector

```
vector<int> v(3);  
    v[0] = 12;    v[1] = 34;    v[2] = 56; /// 12,34,56  
    v.resize(4);  
    v[3] = 78;  
    v.push_back(90); /// 12, 34, 56, 78, 90  
  
    vector<int>::iterator p,pp;  
    p = v.begin();  
    p++;  
    v.insert(p,3,100); /// 12, 100, 100, 100, 34, 56, 78, 90  
  
    p = v.begin();  
    v.erase(p); /// 100, 100, 100, 34, 56, 78, 90  
  
    p = v.begin()+3;  
    v.erase(p,p+2); /// 100, 100, 100, 78, 90
```

Standard Template Library (STL)

Containere de tip secventa

Vector (Dynamic Array)

Exemplu - inserarea elementelor de tip definit de utilizator

```
class Test { int i;
public:
    Test(int x = 0) : i(x) { cout<<"C "; };
    Test(const Test& x) { i = x.i; cout<<"CC "; }
    ~Test() { cout<<"D "; } };

int main() {
    vector<Test> v;
    v.push_back(10); cout<<endl;    /// C CC D
    v.push_back(20); cout<<endl;    /// C CC CC D D
    Test ob(30);
    v.push_back(ob); cout<<endl;    /// C CC CC CC D D
    Test& ob2 = ob;
    v.push_back(ob2);    /// CC D D D D
}
```

Standard Template Library (STL)

Containere de tip secventa List

template <class T, class Allocator = allocator <T> > class list

T = tipul de data utilizat

Allocator = tipul de alocator utilizat (in general cel standard).

- implementat ca și listă dublu înlănțuită

List are performanțe bune la:

- Ștergere/adăugare de elemente pe orice poziție (constant time).
- Mutarea de elemente sau secvențe de elemente în liste sau chiar și între liste diferite(constant time).
- Iterare de elemente in ordine (linear time)

Standard Template Library (STL)

Containere de tip secventa List

Constructori:

```
explicit list(const Allocator &a = Allocator( ) );  
// expl. list<int> iv
```

```
explicit list(size_type num, const T &val = T( ), const Allocator &a =  
Allocator());  
// expl. list<int> iv(10);  
// expl. list<int> iv(10,7);
```

```
list(const list <T,Allocator> &ob);  
// expl. list<int> iv2(iv);
```

```
template <class InIter>  
list(InIter start, InIter end, const Allocator &a = Allocator( ));
```

Standard Template Library (STL)

Containere de tip secventa List

Functii membre uzuale (sursa H. Schildt)

- **back()**: returneaza o referinta catre ultimul element;
- **front()**: returneaza o referinta catre primul element;
- **begin()**: returneaza un iterator catre primul element;
- **end()**: returneaza un iterator catre zona de memorie de dupa ultimul element;
- **clear()**: elimina toate elementele.
- **empty()**: “true(false)” daca lista e (sau nu) goala.
- **erase(iterator i)**: stergerea elementului pointat de i; returneaza un iterator catre elementul de dupa cel sters.
- **erase(iterator start, iterator end)**: stergerea elementelor intre start si end.

Standard Template Library (STL)

Containere de tip secventa List

Functii membre uzuale (sursa H. Schildt)

- **insert(iterator i, const T &val)**: insereaza val inaintea elementului pointat;
- **insert(iterator i, size_type num, const T & val)** : insereaza un numar de “num” elemente de valoare “val” inaintea elementului pointat de i.
- **insert(iterator i, InIter start, InIter end)**: insereaza o secventa start-end inaintea elementului pointat de i.
- **merge(list <T, Allocator> &ob)**: concateneaza lista din ob; aceasta din urma devine vida.
- **merge(list &ob, Comp cmpfn)**: concateneaza si sorteaza;

Standard Template Library (STL)

Containere de tip secventa List

Functii membre uzuale (sursa H. Schildt)

- **pop_back()**: sterge ultimul element.
- **pop_front()**: sterge primul element.
- **push_back(const T &val)**: adauga la final valoarea “val”
- **push_front(const T &val)**: adauga la inceput valoarea “val”
- **remove(const T &val)**: elimina toate valorile “val” din lista;
- **reverse()**: inverseaza lista.
- **size()** : numarul de elemente.
- **sort()**: ordoneaza crescator
- **sort(Comp cmpfn)**: - sorteaza cu o functie de comparatie.

Standard Template Library (STL)

Containere de tip secventa List

Exemplu

```
#include <iostream>
#include <list>
using namespace std;
int main()
{
    list<int> lst; // create an empty list
    int i;
    for(i=0; i<10; i++) lst.push_back(i);
    cout << "Size = " << lst.size() << endl;
    list<int>::iterator p;
    for(p = lst.begin(); p != lst.end(); p++)
        cout << *p << " ";
    for(p = lst.begin(); p != lst.end(); p++)
        *p = *p + 100;
    for(p = lst.begin(); p != lst.end(); p++)
        cout << *p << " ";
    return 0;
}
```

Standard Template Library (STL)

Container de tip secventa List

Exemplu

```
list<int> lst; // lista vida
for(int i=0; i<10; i++) lst.push_back(i); // insereaza 0 .. 9

// Lista afisata in ordine inversa - mod 1"
list<int>::iterator p;
p = lst.end();
while(p != lst.begin()) {
    p--; // decrement pointer before using
    cout << *p << " ";
}

// Lista afisata in ordine inversa - mod 2"
list<int>::reverse_iterator q;
for(q = lst.rbegin(); q!=lst.rend(); q++)
    cout<<*q<<" ";
```

Standard Template Library (STL)

Container de tip secventa List

Exemplu - push_front, push_back si sort

```
#include <iostream>
#include <list>
using namespace std;
int main() {
    list lst1, lst2;
    int i;
    for(i=0; i<10; i++) lst1.push_back(i);
    for(i=0; i<10; i++) lst2.push_front(i);

    list::iterator p;
    for( p = lst1.begin(); p != lst1.end(); p++) cout << *p << " ";
    for( p = lst2.begin(); p != lst2.end(); p++) cout << *p << " ";

    // sort the list
    lst1.sort();
}
```

Standard Template Library (STL)

Container de tip secventa List

Exemplu - ordonare crescatoare si descrescatoare

```
#include<iostream>
#include<list>
using namespace std;
```

```
bool compare(int a, int b) {return a>b;}
```

```
int main() {
list<int> l;
l.push_back(5); l.push_back(3); l.push_front(2);
list<int>::iterator p;
l.sort(); //crescator
for(p=l.begin(); p!=l.end();p++) cout<<*p<<" ";
```

```
l.sort(compare); // descrescator prin functia comparator
for(p=l.begin(); p!=l.end();p++) cout<<*p<<" ";
}
```

Standard Template Library (STL)

Container de tip secventa List

Exemplu - concatenarea a 2 liste

```
#include <iostream>
#include <list>
using namespace std;
int main()
{
    list<int> lst1, lst2; /*...create liste ...*/
    list<int>::iterator p;
    // concatenare
    lst1.merge(lst2);
    if (lst2.empty())
        cout << "lst2 vida\n";
    cout << "Contents of lst1 after merge:\n";
    for (p = lst1.begin(); p != lst1.end(); p++) cout << *p << " ";
    return 0;
}
```

Standard Template Library (STL)

Container de tip secventa List

Exemplu - stocarea obiectelor in list

```
class Test { int i;
public:
    Test(int x = 0) : i(x) { cout<<"C "; };
    Test(const Test& x) { i = x.i; cout<<"CC "; }
    ~Test() { cout<<"D "; } };

int main() {
    list<Test> v;
    v.push_back(10); cout<<endl;    /// C CC D
    v.push_back(20); cout<<endl;    /// C CC D
    Test ob(30);
    v.push_back(ob); cout<<endl;    /// C CC
    Test& ob2 = ob;
    v.push_back(ob2);    /// CC D D D D D
}
```

Standard Template Library (STL)

Container de tip secventa List

Exemplu - stocarea obiectelor in list

```
#include <iostream>
#include <list>
#include <cstring>
using namespace std;
class myclass
{
    int a, b, sum;
public:
    myclass() {a = b = 0;}
    myclass(int i, int j) {a = i; b = j; sum = a + b;}
    int getsum() {return sum;}
    friend bool operator<(myclass &o1, myclass &o2) {return o1.sum < o2.sum;}
    friend bool operator>(myclass &o1, myclass &o2) {return o1.sum > o2.sum;}
    friend bool operator==(myclass &o1, myclass &o2) {return o1.sum == o2.sum;}
    friend bool operator!=(myclass &o1, myclass &o2) {return o1.sum != o2.sum;}
};
```

Standard Template Library (STL)

Container de tip secventa List

Exemplu - stocarea obiectelor in list

```
int main() {
    int i;
    list<myclass> lst1;
    list<myclass>::iterator p;

    for(i=0; i<10; i++) lst1.push_back(myclass(i, i));
    for (p = lst1.begin(); p != lst1.end(); p++) cout<< p->getsum()<< " ";
    cout << endl;

    // create a second list
    list<myclass> lst2;
    for(i=0; i<10; i++) lst2.push_back(myclass(i*2, i*3));
    for (p = lst2.begin(); p != lst2.end(); p++) cout<< p->getsum()<< " ";
    cout << endl;

    // now, merget lst1 and lst2
    lst1.merge(lst2);
    return 0;
}
```


Standard Template Library (STL)

Containere de tip secventa

Deque (double ended queue) - Coadă dublă (completă)

- elementele sunt stocate în blocuri de memorie (chunks of storage)
- elementele se pot adăuga/șterge eficient de la ambele capete

Vector vs Deque

- Accesul la elemente de pe orice poziție este mai eficient la vector
- Inserare/ștergerea elementelor de pe orice poziție este mai eficient la Deque (dar nu e timp constant)
- Pentru liste mari Vector alocă zone mari de memorie, deque alocă multe zone mai mici de memorie – Deque este mai eficient în gestiunea memoriei

Standard Template Library (STL)

Adaptor de container

- Încapsulează un container de tip secvență, și folosesc acest obiect pentru a oferi funcționalități specifice containerului (stivă, coadă, coadă cu priorități).

Stack: strategia LIFO (last in first out) pentru adaugare/ștergere elemente
Operații: empty(), push(), pop(), top().

```
template < class T, class Container = deque<T> > class stack;
```

Queue: strategia FIFO (first in first out)
Operații: empty(), front(), back(), push(), pop(), size();

```
template < class T, class Container = deque<T> > class queue;
```

Priority_queue: se extrag elemente pe baza priorităților
Operații: empty(), top(), push(), pop(), size();

```
template < class T, class Container = vector<T>, class Compare =  
less<typename Container::value_type> > class priority_queue;
```

Standard Template Library (STL)

Adaptor de container

Exemplu stiva

```
#include <stack>
#include<iostream>
using namespace std;
void sampleStack() {
    stack<int> s;
    s.push(3);
    s.push(4);
    s.push(1);
    s.push(2);

    while (!s.empty()) {
        cout << s.top() << " ";
        s.pop();
    }
}
int main() {
    sampleStack(); // 2, 1, 4, 3
}
```

Standard Template Library (STL)

Adaptor de container

Exemplu stiva

```
#include <stack>
#include <vector>
#include <iostream>
using namespace std;
void sampleStack() {
```

```
    stack<int,vector<int>> s; // primul parametru = tipul
    elementelor, al doilea parametru, stilul de stocare
```

```
    s.push(3);
    s.push(4);
    s.push(1);
    s.push(2);

    while (!s.empty()) {
        cout << s.top() << " ";
        s.pop();
    }

    int main() {
        sampleStack(); // 2, 1, 4, 3
    }
```

Standard Template Library (STL)

Adaptor de container

Exemplu stiva - influenta celui de-al doilea parametru (eventual)

```
class Test { int i;
public:
Test (int x = 0) : i(x) {cout<<"C ";};
Test (const Test& x) {i = x.i; cout<<"CC ";}
~Test () {cout<<"D ";}};

int main()
{
    stack<Test> s;
    s.push(1); // C CC D
    cout<<endl;
    s.push(3); // C CC D
    cout<<endl;
    s.push(5); // C CC D urmat de D D D (distrugerea listei)
}
```

Standard Template Library (STL)

Adaptor de container

Exemplu stiva - influenta celui de-al doilea parametru (eventual)

```
class Test { int i;
public:
Test (int x = 0) : i(x) {cout<<"C ";};
Test (const Test& x) {i = x.i; cout<<"CC ";}
~Test () {cout<<"D ";}};

int main()
{
    stack<Test, vector<Test> > s;
    s.push(1); // C CC D
    cout<<endl;
    s.push(3); // C CC CC D D
    cout<<endl;
    s.push(5); // C CC CC CC D D D urmat de D D D
}
```

Standard Template Library (STL)

Adaptor de container

Exemplu stiva - influenta celui de-al doilea parametru (eventual)

```
class Test { int i;
public:
Test (int x = 0) : i(x) {cout<<"C ";};
Test (const Test& x) {i = x.i; cout<<"CC ";}
~Test () {cout<<"D ";}};

int main()
{
    stack<Test, list<Test> > s;
    s.push(1); // C CC D
    cout<<endl;
    s.push(3); // C CC D
    cout<<endl;
    s.push(5); // C CC D urmat de D D D (distrugerea listei)
}
```

Standard Template Library (STL)

Adaptor de container

Exemplu coada

```
#include <queue>
#include<iostream>
using namespace std;
void sampleQueue() {
    queue<int> s;

    s.push(3);
    s.push(4);
    s.push(1);
    s.push(2);

    while (!s.empty()) {
        cout << s.top() << " ";
        s.pop();
    }
}
int main() {
    sampleQueue(); // 3, 4, 1, 2
}
```


Standard Template Library (STL)

Adaptor de container

Exemplu coada cu prioritate

Alocator default =
max-heap

Pentru utilizare cu
tipuri de date definite
de utilizator, trebuie
supraincarcat
operatorul <

```
#include <queue>
#include<iostream>
using namespace std;
void samplePriorQueue() {
    priority_queue<int> s;

    s.push(3);
    s.push(4);
    s.push(1);
    s.push(2);

    while (!s.empty()) {
        cout << s.top() << " ";
        s.pop();
    }
}
int main() {
    samplePriorQueue(); // 4, 3, 2, 1
}
```

Standard Template Library (STL)

Containere asociative

eficiente în accesare elementelor folosind chei (nu folosind poziții ca și în cazul containerelor de tip secvență).

- **Set**

- mulțime - stochează elemente distincte.
- se folosește arbore binar de căutare ca și reprezentare internă

- **Map**

- dictionar - stochează elemente formate din cheie și valoare
- nu putem avea chei duplicate
- **multimap** poate avea chei duplicate

- **Bitset**

- container special pentru a stoca biti.

Standard Template Library (STL)

Containere asociative Map

- in sens general, map = lista de perechi cheie - valoare

template <class Key, class T, class Comp = less<Key>, class Allocator = allocator<pair<const Key,T>> > class map

Key = tipul cheilor

T = tipul valorilor

Comp = functie care compara 2 chei

Allocator = tipul de alocator utilizat (in general cel standard).

Inserarea se face ordonat dupa chei.

Standard Template Library (STL)

Containere asociative Map

Constructori:

```
explicit map(const Comp &cmpfn = Comp( ), const Allocator &a = Allocator( ) );
```

```
// expl. map<char,int> m; - map cu zero elemente;
```

```
map(const map<Key,T,Comp,Allocator> &ob);
```

```
template <class InIter>
```

```
map(InIter start, InIter end, const Comp &cmpfn = Comp( ), const Allocator &a  
= Allocator( ));
```

Standard Template Library (STL)

Containere asociative Map

Functii membre uzuale (sursa H. Schildt)

Member	Description
<code>iterator begin();</code> <code>const_iterator begin() const;</code>	returneaza un iterator catre primul element;
<code>iterator end();</code> <code>const_iterator end() const;</code> <code>element;</code>	returneaza un iterator catre ultimul element;
<code>void clear();</code>	elimina toate elementele din map.
<code>size_type count(const key_type &k) const;</code>	- numarul de aparitii ale lui k
<code>bool empty() const;</code>	“true(false)” daca map e (sau nu) gol.

Standard Template Library (STL)

Containere asociative Map

Functii membre uzuale (sursa H. Schildt)

Member	Description
<code>iterator erase(iterator i);</code>	stergere elementului pointat de i; returneaza un iterator catre elementul de dupa cel sters.
<code>iterator erase(iterator start, iterator end);</code>	stergere elementelor intre start si end.
<code>size_type erase(const key_type &k)</code>	- sterger elementelor cu cheia k
<code>iterator find(const key_type &k);</code>	
<code>const_iterator find(const key_type &k) const;</code>	- returneaza un iterator catre valoarea cautata k sau catre sfarsitul map daca nu exista.

Standard Template Library (STL)

Containere asociative Map

Functii membre uzuale (sursa H. Schildt)

Member

`size_type size( ) const;`

Description

`iterator insert(iterator i, const value_type &val);` - insereaza val pe pozitia pointata de i sau dupa;

`template <class InIter>`

`void insert(InIter start, InIter end);` - insereaza o secventa start-end.

`pair <iterator,bool>`

`insert(const value_type &val);`

`mapped_type & operator[ ](const key_type &i)` - returneaza o referinta la elementul cheie i, daca nu exista, atunci se insereaza.

Standard Template Library (STL)

Containere asociative

Map

Exemplu

```
#include <iostream>
#include <map>
using namespace std;
int main() {
    map<int, int> m;
    m.insert(pair<int, int>(1,100));
    m.insert(pair<int, int>(7,300));
    m.insert(pair<int, int>(3,200));
    m.insert(pair<int, int>(5,400));
    m.insert(pair<int, int>(2,500));

    map<int, int>::iterator p;
    for(p = m.begin(); p != m.end(); p++)
        cout<<p->first<<" "<<p->second<<endl;

    //copierea intr-un alt map
    map<int,int>m2(m.begin(),m.end());
}
```


Standard Template Library (STL)

Container associative

Map

Exemplu

```
#include <iostream>
#include <map>
using namespace std;
int main() {
    map<char, int> m;
    for(int i=0; i<26; i++)
        m.insert(pair<char, int>('A'+i, 65+i));

    char ch;    cout << "Enter key: ";    cin >> ch;
    map<char, int>::iterator p;
    // find value given key
    p = m.find(ch);
    if(p != m.end())
        cout << "Its ASCII value is " << p->second;
    else cout << "Key not in map.\n";
    return 0;
}
```

Standard Template Library (STL)

Containere asociative Multimap

Exemplu - catalog studenti (cheia = nume si valoarea = numar matricol)

```
#include <iostream>
#include <map>
using namespace std;
int main( ) {
    multimap<string, int> m;
    m.insert(m.end(), make_pair("Ionescu",100));
    // echivalent cu m.insert(pair<string,int>("Ionescu",100));

    m.insert(m.end(), make_pair("Popescu",355));
    m.insert(m.end(), make_pair("Ionescu",234));

    map<string, int>::iterator p;
    for(p = m.begin(); p != m.end(); p++)
        cout<<p->first<<" "<<p->second<<endl;
```

Standard Template Library (STL)

Containere asociative Multimap

Exemplu - catalog studenti (cheia = nume si valoarea = numar matricol)

```
//Afisarea elementelor cu un anumit nume
string nume;
cin>>nume;
for(p = m.begin(); p!= m.end(); p++)
if (p->first == nume)
    cout<<p->first<<" "<<p->second<<endl;

//stergerea primei aparitii a unui nume
cin>>nume;
map<string, int>::iterator f;
f = m.find(nume);
if (f!=m.end()) m.erase(f);
for(p = m.begin(); p!= m.end(); p++)
    cout<<p->first<<" "<<p->second<<endl;
```

Standard Template Library (STL)

Containere asociative

Multimap

Exemplu - catalog studenti (cheia = nume si valoarea = numar matricol)

```
//Stergerea tuturor aparitiilor unui nume  
cin>>nume;  
  
m.erase(nume);
```

Standard Template Library (STL)

Containere asociative

Unordered_map

- **Bazat pe hash_table**; Principalul motiv pentru care se alege unordered_map este eficiența timpului de execuție

Performanță (Complexitate)

Căutare: $O(1)$ în medie, $O(n)$ în cel mai rău caz (foarte rar).

Inserare: $O(1)$ în medie.

Ștergere: $O(1)$ în medie

- **Worst case – $O(n)$** : Dacă funcția de hash este slabă, apar **coliziuni**. Toate elementele ajung în același bucket, transformând căutarea într-o listă liniară
- **load_factor()** ne ajută să monitorizăm “sănătatea” structurii.

Obs: volum mare de date + nu interesează ordinea elementelor -> unordered_map va fi aproape întotdeauna mai rapid decât std::map.

Standard Template Library (STL)

Containere asociative

Unordered_map

Caracteristici cheie

Nu menține ordinea: Elementele sunt stocate în "bucket-uri" bazate pe hash-ul cheii. Dacă parcurgi harta, ordinea va părea aleatorie și se poate schimba la inserări noi.

Chei unice: Nu poți avea două elemente cu aceeași cheie (pentru asta există `std::unordered_multimap`).

Consum de memorie: De obicei, consumă mai multă memorie decât `std::map` din cauza structurii tabeli de hash și a necesității de a evita coliziunile.

Tipuri de date complexe ca chei: `unordered_map` știe să facă hash implicit doar pentru tipuri de bază (`int`, `string`, `double`). Dacă vrei să folosești un `std::pair` sau un struct propriu drept cheie, va trebui să îți scrii propria funcție de hash.

Standard Template Library (STL)

Containere asociative

Unordered_map

Reprezentarea datelor in memorie

```
unordered_map<string,int> m = {{ "Popescu",10},{ "Ionescu",9}};  
for(int i = 0; i < m.bucket_count(); i++)  
{  
    cout<<"Bucket ["<<i<<"] contine: ";  
    for(auto it = m.begin(i); it != m.end(i); it++)  
        cout<<"("<<it->first<<") ";  
    cout<<endl;  
}
```

```
Bucket [0] contine:  
Bucket [1] contine:  
Bucket [2] contine:  
Bucket [3] contine:  
Bucket [4] contine:  
Bucket [5] contine:  
Bucket [6] contine:  
Bucket [7] contine:  
Bucket [8] contine:  
Bucket [9] contine:  
Bucket [10] contine:  
Bucket [11] contine:  
Bucket [12] contine: (Ionescu) (Popescu)
```

Standard Template Library (STL)

Containere asociative

Unordered_map

Exemplu:

```
std::unordered_map<std::string, int> varste;

// Inserare
varste["Alice"] = 25;
varste.insert({"Bob", 30});

// Accesare
std::cout << "Alice are " << varste["Alice"] << " ani." << std::endl;

// Verificare existență
if (varste.find("Charlie") == varste.end()) {
    std::cout << "Charlie nu este in lista." << std::endl;
}
```


Standard Template Library (STL)

Containere asociative

Map vs Unordered_map

Caracteristică	<code>std::map</code>	<code>std::unordered_map</code>
Implementare	Arbore Red-Black	Tabelă de Hash
Timp Căutare	$O(\log n)$	$O(1)$ (medie)
Ordine	Sortat după cheie	Neordonat
Cerințe Cheie	Operator <code><</code> definit	Funcție Hash + Operator <code>==</code>

Standard Template Library (STL)

Iterator

- un concept fundamental in STL, este elementul central pentru algoritmi oferiți de STL;
- obiect care gestionează o poziție (curentă) din containerul asociat;
- suport pentru traversare (++/--), dereferențiere (*it);
- permite decuplarea între algoritmi și containere.

Tipuri de iteratori:

- iterator input/output (istream_iterator, ostream_iterator)
- forward iterators, iterator bidirectional, iterator random access
- reverse iterators.

Adaptoarele de containere (container adaptors) - stack, queue, priority_queue - nu oferă iterator!

Standard Template Library (STL)

Iterator

Functii uzuale:

- **begin()** - returneaza pozitia de inceput a containerului
- **end()** - returneaza pozitia de dupa terminarea containerului
- **advance()** - incrementeaza pozitia iteratorului
- **next()** - returneaza noul iterator dupa avansarea cu o pozitie
- **prev()** - returneaza noul iterator dupa decrementarea cu o pozitie
- **inserter()** – insereaza elemente pe orice pozitie

Standard Template Library (STL)

Iterator

Exemplu - begin, end, advance, next, prev:

```
#include<iostream>
#include<iterator>
#include <vector>

using namespace std;
int main()
{
    vector<int> v(5);
    for(int i = 0; i<5; i++) v[i] = (i+1)*10;

    vector<int>::iterator p;
    for (p = v.begin(); p < v.end(); p++)
        cout << *p << " ";
    cout<<endl;
    p = v.begin();
    advance(p, 2);
    vector<int>::iterator n = next(p);
    vector<int>::iterator d = prev(p);
    for (; p < v.end(); p++)
        cout << *p << " ";
    cout<<endl<<*n<<" " <<*d;

}
```

10 20 30 40 50

30 40 50

40 20

Standard Template Library (STL)

Iterator

Exemplu - inserter:

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> v = { 1, 2, 3, 4, 5 };
    vector<int> v2 = { 10, 20, 30 };
    vector<int>::iterator p = v.begin();
    advance(p, 2);
    copy(v2.begin(), v2.end(), inserter(v, p));
    for (p = v.begin(); p < v.end(); p++)
        cout << *p << " ";
    return 0;
}
```

1 2 10 20 30 3 4 5

Standard Template Library (STL)

Reverse iterator

Exemplu

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> v = { 1, 2, 3, 4, 5 };

    vector<int>::iterator p;
    for (p = v.begin(); p < v.end(); p++)
        cout << *p << " ";
    cout<<endl;
    vector<int>::reverse_iterator r;
    for (r = v.rbegin(); r < v.rend(); r++)
        cout << *r << " ";
    return 0;
}
```

Standard Template Library (STL)

Algoritm

- colecție de funcții template care pot fi folosite cu iteratori. Funcțiile operează pe un domeniu (range) definit folosind iteratori;
- domeniu (range) este o secvență de obiecte care pot fi accesate folosind iteratori sau pointeri.
- headere: **<algorithm>**, **<numeric>**

Standard Template Library (STL)

Algoritm

- **Operații pe secvențe**
 - care nu modifică sursa: accumulate, count, find, count_if, etc
 - care modifică : copy, transform, swap, reverse, random_shuffle, etc.
- **Sortări:** sort, stable_sort, etc.
- **Pe secvențe de obiecte ordonate**
 - **Căutare binară** : binary_search, etc
 - **Interclasare (Merge)**: merge, set_union, set_intersection, etc.
- **Min/max:** min, max, min_element, etc.
- **Heap:** make_heap, sort_heap, etc.

Standard Template Library (STL)

Algoritm

Exemplu: – accumulate : calculează suma elementelor

```
vector<int> v;  
v.push_back(3);  
v.push_back(4);  
v.push_back(2);  
v.push_back(7);  
v.push_back(17);  
  
//compute the sum of all elements in the vector  
cout << accumulate(v.begin(), v.end(), 0) << endl;  
  
//compute the sum of elements from 1 inclusive, 4 exclusive [1,4)  
vector<int>::iterator start = v.begin()+1;  
vector<int>::iterator end = v.begin()+4;  
cout << accumulate(start, end, 0) << endl;
```

Standard Template Library (STL)

Algoritm

Exemplu: – copy

```
vector<int> v;  
v.push_back(3);  
v.push_back(4);  
v.push_back(2);  
v.push_back(7);  
v.push_back(17);  
  
//make sure there are enough space in the destination  
//allocate space for 5 elements  
vector<int> v2(5);  
  
//copy all from v to v2  
copy(v.begin(), v.end(), v2.begin());
```

Standard Template Library (STL)

Algoritm

Exemplu: – sort

Sorteaza elementele din intervalul [first,last) ordine crescătoare.

- Elementele se compară folosind **operator <**

```
vector<int> v;  
v.push_back(3);  
v.push_back(4);  
v.push_back(2);  
v.push_back(7);  
v.push_back(17);  
  
sort(v.begin(), v.end());
```

Standard Template Library (STL)

Algoritm

Exemplu: – sort cu o functie de comparare

```
bool asc(int i, int j) {  
    return (i < j);  
}  
  
vector<int> v;  
v.push_back(3);  
v.push_back(4);  
v.push_back(2);  
v.push_back(7);  
v.push_back(17);  
  
sort(v.begin(), v.end(), asc);
```

Standard Template Library (STL)

Algoritm

Exemplu: – for_each

```
void print(int elem) {  
    cout << elem << " ";  
}  
  
void testForEach() {  
    vector<int> v;  
    v.push_back(3);  
    v.push_back(4);  
    v.push_back(2);  
    v.push_back(7);  
    v.push_back(17);  
  
    for_each(v.begin(), v.end(), print);  
    cout << endl;  
}
```

Standard Template Library (STL)

Algoritm

Exemplu: – transform

- aplică funcția pentru fiecare element din secvență ([first1,last1))
- rezultatul se depune în secvența rezultat

```
int multiply3(int el) {  
    return 3 * el;  
}  
  
void testTransform() {  
    vector<int> v;  
    v.push_back(3); v.push_back(4);  
    v.push_back(2); v.push_back(7);  
    v.push_back(17);  
  
    vector<int> v2(5);  
    transform(v.begin(), v.end(), v2.begin(), multiply3);  
    //print the elements  
    for_each(v2.begin(), v2.end(), print);  
}
```

Standard Template Library (STL)

C++11

Expresii Lambda

- **funcții anonime** definite direct în cod, utile pentru operații scurte care nu merită o funcție separată.
- permite definirea funcției local, la momentul apelării;

Definite "on-the-fly"

Acces la variabilele locale

Sintaxă compactă și eficientă

Sintaxa: **[capture](parameters) mutable exception -> return-type {body}**

Standard Template Library (STL)

C++11

Expresii Lambda

Sintaxa: **[capture](parameters) mutable exception -> return-type {body}**

capture - partea introductiva, care spune compilatorului ca urmeaza o expresie lambda; aici se specifica si ce variabile si in ce mod (valoare sau referinta) se copiaza din blocul in care expresia lambda este definita;

parameters - parametrii expresiei lambda;

mutable (optional) – permite modificarea parametrilor transmisi prin valoare

exception (optional) – a se utiliza **noexcept** daca nu se arunca nicio expresie

return – type (optional) - tipul la care se evalueaza expresia lambda; compilatorul deduce implicit care este tipul expresiei.

body – corpul expresiei

Standard Template Library (STL)

C++11

Expresii Lambda

Blocul *capture* []

- Expresiile lambda pot “captura” variabilele din program sau poate introduce variabile locale (incepand cu C++14).
- Clauze de captura

[] - Nu captează nimic.

[=] - Toate prin **valoare** (copie).

[&] - Toate prin **referință**.

[x, &y] - Specific: x copie, y referință.

Standard Template Library (STL)

C++11

Expresii Lambda

Exemplu (preluat)

```
int a = 0; // Define an integer variable
auto f = []() { return a*9; }; // Error: 'a' cannot be accessed
auto f = [a]() { return a*9; }; // OK, 'a' is "captured" by value
auto f = [&a]() { return a++; }; // OK, 'a' is "captured" by reference
// Note: It is the responsibility of the programmer
// to ensure that a is not destroyed before the
// lambda is called.
auto b = f(); // Call the lambda function. a is taken from the capture
```

Standard Template Library (STL)

C++11

Lambda Expressions

Blocul *capture* [] – observatii / restrictii

- Daca [] contine & by default, atunci nu se poate declara un argument particular tot cu &
- Daca [] contine = by default, atunci nu se poate declara un argument particular tot prin valoare

Exemplu (preluat)

```
struct S { void f(int i); };

void S::f(int i) {
    [&, i]{};           // OK
    [&, &i]{};          // ERROR: i preceded by & when & is the default
    [=, this]{};        // ERROR: this when = is the default
    [=, *this]{ };      // OK: captures this by value. See below.
    [i, i]{};           // ERROR: i repeated
}
```

Standard Template Library (STL)

C++11

Lambda Expressions – exemplu

```
#include <iostream>
#include <string>
using namespace std;
int main()
{
    auto sum = [](int a, int b) {
        return a + b;
    };

    cout << "Sum of two integers:" << sum(5, 6) << endl;

    return 0;
}
```

Standard Template Library (STL)

C++11

Lambda Expressions – exemplu

```
#include <bits/stdc++.h>
#include <vector>

using namespace std;

int main() {
    vector<int> v={1,2,1,3,1,1};
    //afisare prin lambda expresie
    for_each(v.begin(), v.end(), [ ](int i){ cout << i << ' '; });
    cout << '\n';

    // contorizare prin lambda expresie
    int cont = 0; //modificat prin lambda expresie
    for_each(v.begin(), v.end(), [&cont](int i){
        if (i == 1)
            cont++;
    });
    cout<< cont <<" de 1 ";
    return 0;
}
```

Standard Template Library (STL)

C++11

Lambda Expressions + STL Algorithms – exemplu customizare sortari/cautari

```
vector<int> v = {5, 2, 8, 1, 9};  
  
/// sortare descrescatoare folosind lambda expresie  
  
sort(v.begin(), v.end(), [] (int a, int b) {return a>b;});  
  
///gasirea primului numar par din vector; obs: it este un iterator  
  
auto it = find_if(v.begin(), v.end(), [] (int n) {return n%2 == 0;});  
  
cout<<*it<<endl;
```

Standard Template Library (STL)

C++14

Lambda Expressions – exemplu cu tipuri de date generalizate (incepand cu C++14)

```
#include <iostream>
#include <string>
using namespace std;
int main()
{
    // generalized lambda
    auto sum = [](auto a, auto b) {
        return a + b;
    };

    cout << "Sum(5,6) = " << sum(5, 6) << endl; // sum of two integers

    cout << "Sum(2.0,6.5) = " << sum(2.0, 6.5) << endl; // sum of two floats

    cout << "Sum((string(\"SoftwareTesting\"), string(\"help.com\"))) = "
        << sum(string("SoftwareTesting"), string("help.com")) << endl; // sum of two strings

    return 0;
}
```

Standard Template Library (STL)

C++11

Deducerea tipului in mod automat (auto si decltype)

In C++03, fiecare variabila trebuie sa aiba un tip de data;

Cuvantul cheie "auto"

In C++11, acesta poate fi dedus din initializarea variabilei respective;

In cazul functiilor, daca tipul returnat este **auto**, este evaluata de expresia returnata la runtime;

Variabilele **auto** TREBUIE initializate, altfel eroare.

Standard Template Library (STL)

C++11

Deducerea tipului in mod automat (auto si decltype)

```
#include <bits/stdc++.h>
using namespace std;
class Test{ };
int main() {
    auto x = 1; // x e int
    float z;
    auto y = z; // y e float
    auto t = 3.37; // t e double
    auto p = &x; // pointer catre int

    Test ob;
    auto A = ob;
    auto B = &ob;

    cout << typeid(x).name() << endl << typeid(y).name() << endl
         << typeid(t).name() << endl << typeid(p).name() << endl
         << typeid(A).name() << endl << typeid(B).name() << endl;

    return 0;
}
```

Standard Template Library (STL)

C++11

Deducerea tipului in mod automat (auto si decltype)

```
#include <bits/stdc++.h>
#include <vector>
using namespace std;
int f(){}
int& g(){}

int main() {
vector<int> v = {1,2,4};

    auto x = v.begin();
    cout << typeid(x).name() << endl;
    for(auto p = v.begin(); p!=v.end(); p++)
        cout<<*p<<" ";
    cout<<endl;
    auto y = f();
    auto z = g();//    auto& z = g();
    cout << typeid(y).name() << endl << typeid(z).name() << endl;
    return 0;
}
```

Standard Template Library (STL)

C++11

Deducerea tipului in mod automat (auto si decltype)

```
int f(){}  
float g(){}  
// atentie: daca float& g() {}, eroare la decltype(g()) z, intrucat trebuie initializat  
  
int main()  
{  
  
    decltype(f()) y;  
    decltype(g()) z;  
    cout << typeid(y).name() << endl;  
    cout << typeid(z).name() << endl;  
  
    return 0;  
}
```

auto – permite declararea unei variabile cu un tip specific, iar decltype "ghiceste" tipul

Perspective

Cursul 11:

1. Pointeri și referințe
2. Const
3. Volatile
4. Static

Cursul 12:

Design Patterns



Programare orientată pe obiecte

- suport de curs -

Andrei Păun
Anca Dobrovăț

An universitar 2025 – 2026
Semestrul II
Seriile 13, 14, 15

Curs 12



Agenda cursului

Șabloane de proiectare (Design Patterns)

- Definiție și clasificare.
- Exemple de șabloane de proiectare (Singleton, Abstract Object Factory, Observer, Strategy Pattern).

Obs: Prezentare bazata pe GoF

(Erich Gamma, Richard Helm, Ralph Johnson si John Vlissides – Design Patterns, Elements of Reusable Object-Oriented Software (cunoscuta si sub numele “Gang of Four”), 1994)



Sabloane de proiectare (Design patterns)

Principiile proiectării de clase (S.O.L.I.D) – Robert C. Martin

Single-responsibility principle

A class should only have a single responsibility, that is, only changes to one part of the software's specification should be able to affect the specification of the class.

Open-closed principle

"Software entities ... should be open for extension, but closed for modification."

Liskov substitution principle

"Objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program."

Interface segregation principle

"Many client-specific interfaces are better than one general-purpose interface."

Dependency inversion principle

One should "depend upon abstractions, [not] concretions."



Sabloane de proiectare (Design patterns)

Principiile proiectarii de clase

Principiul “inchis-deschis”

“Entitatile software (module, clase, functii etc.) trebuie sa fie deschise la extensii si inchise la modificare” (Bertrand Meyer, 1988).

“deschis la extensii” = comportarea modulului poate fi extinsa pentru a satisface noile cerinte.

“inchis la modificare” = nu este permisa modificarea codului sursa.



Sabloane de proiectare (Design patterns)

Principiile proiectării de clase

Principiul substituirii

Funcțiile care utilizează pointeri sau referințe la clasa de bază trebuie să fie apte să utilizeze obiecte ale claselor derivate fără să le cunoască.

Principiul de inversare a dependentelor

- A. “Modulele de nivel înalt nu trebuie să depindă de modulele de nivel jos. Amândouă trebuie să depindă de abstracții.”
- B. “Abstracțiile nu trebuie să depindă de detalii. Detaliile trebuie să depindă de abstracții.”
 - programele OO bine proiectate inversează dependența structurală de la metoda procedurală tradițională
- metoda procedurală: o procedură de nivel înalt apelează o procedură de nivel jos, deci depinde de ea



Sabloane de proiectare (Design patterns)

Definitie si clasificare

Aplicarea principiilor pentru a crea arhitecturi OO → se ajunge repetat la aceleasi structuri, cunoscute sub numele de **sabloane de proiectare (design patterns)**.

Un **sablon de proiectare** descrie

- o problema care se intalneste in mod repetat in proiectarea programelor
- solutia generala pentru problema respectiva

Solutia este exprimata folosind **clase** si **obiecte**.

Cand si unde a aparut ideea?

- Arhitectura
- 1977: "A pattern language: Towns, Buildings, Constructiron"
- Christopher Alexander

https://en.wikipedia.org/wiki/Pattern_language



Sabloane de proiectare (Design patterns)

Definitie si clasificare

Clasificarea șabloanelor după scop:

- **creaționale** (creational patterns) privesc modul de creare al obiectelor.
- **structurale** (structural patterns) se referă la compoziția claselor sau a obiectelor.
- **comportamentale** (behavioral patterns) caracterizează modul în care obiectele și clasele interacționează și își distribuie responsabilitățile.

Clasificarea șabloanelor după domeniu de aplicare:

- sabloanele claselor** se referă la relații dintre clase, relații stabilite prin moștenire și care sunt statice (fixate la compilare).
- sabloanele obiectelor** se referă la relațiile dintre obiecte, relații care au un caracter dinamic .



Sabloane de proiectare (Design patterns)

Definitie si clasificare

In general, un sablon are 4 elemente esentiale:

1. **nume**
2. **descrierea problemei.** (contextul in care apare, cand trebuie aplicat sablonul).
3. **descrierea solutiei.** (elementele care compun proiectul, relatiile dintre ele, responsabilitatile si colaborarile).
4. **consecintele si compromisuri** aplicarii sablonului.

Un sablon de proiectare descrie de asemenea problemele de implementare ale sablonului si un exemplu de implementare a sablonului in unul sau mai multe limbaje de programare.



Sabloane de proiectare (Design patterns)

Structura unui sablon

În cartea de referință (GoF), descrierea unui sablon este alcătuită din următoarele secțiuni:

Numele sablonului și clasificarea

Intenția

Alte nume prin care este cunoscut, dacă există.

Motivația - scenariu care ilustrează o problemă de proiectare și rezolvarea ;

Aplicabilitatea

Structura - reprezentată grafic prin diagrame de clase și de interacțiune (UML) ;

Participanți - clasele și obiectele și responsabilitățile lor;

Colaborări

Consecințe - compromisurile și rezultatele utilizării sablonului.

Implementare - tehnici de implementare, aspectele dependente de limbaj

Exemplu de cod

Utilizări cunoscute

Sabloane corelate



Sabloane de proiectare (Design patterns)

Unele dintre sabloanele de proiectare cele mai folosite sunt descrise in cele ce urmeaza:

1. Abstract Server

Cand un client depinde direct de server, este incalcat principiul de inversare a dependentelor. Modificarile facute in server se vor propaga in client, iar clientul nu va fi capabil sa foloseasca alte servere similare cu acela pentru care a fost construit.

Situatia de mai sus se poate imbunatati prin inserarea unei interfete abstracte intre client si server,

Client -> Interfata <- Manager



Sabloane de proiectare (Design patterns)

2. Adapter

Cand inserarea unei interfete abstracte nu este posibila deoarece serverul este produs de o alta companie (third party ISV) sau are foarte multe dependente de intrare care-l fac greu de modificat, se poate folosi un ADAPTER pentru a lega interfata abstracta de server.

Client -> Interfata <- Adapter -> Manager.



Sabloane de proiectare (Design patterns)

3. *Singleton (clase cu o singura instanta)*

Intentia

proiectarea unei clase cu un singur obiect (o singura instanță)

Motivatia

Într-un sistem de operare:

- exista un sistem de fișiere

- exista un singur manager de ferestre

Aplicabilitate când trebuie să existe exact o instanta: clientii clasei trebuie sa aiba acces la instanta din orice punct bine definit.



Sabloane de proiectare (Design patterns)

Singleton (clase cu o singura instanta)

Consecințe

- acces controlat la instanta unica;
- reducerea spațiului de nume (eliminarea variab. globale);
- permite rafinarea operațiilor si reprezentării;
- permite un numar variabil de instanțe;
- mai flexibila decât operațiile la nivel de clasă (statice).



Sabloane de proiectare (Design patterns)

Singleton (clase cu o singura instanta) - exemplu cu referinte

```
class Singleton
{
public:
    static Singleton& instance()
    {
        return uniqueInstance;
    }
    int getValue() { return data;}
    void setValue(int value) { data = value;}

private:
    static Singleton uniqueInstance;
    int data;
    Singleton(int d = 0):data(d) {}
    Singleton & operator=(Singleton & ob){
        if (this != &ob) data = ob.data; return *this; }
    Singleton(const Singleton & ob) { data = ob.data; }
};

Singleton Singleton::uniqueInstance (0);

int main()
{
    Singleton& s1 = Singleton::instance();
    cout << s1.getValue() << endl;
    Singleton& s2 = Singleton::instance();
    s2.setValue(9);
    cout << s1.getValue() <<endl;
    return 0;
}
```



Sabloane de proiectare (Design patterns)

Singleton (clase cu o singura instanta) - exemplu cu pointeri

```
class Singleton
{
public:
    static Singleton* instance()
    {
        if (uniqueInstance == NULL)
            uniqueInstance = new Singleton();
        return uniqueInstance;
    }
    int getValue() { return data;}
    void setValue(int value) { data = value;}

private:
    static Singleton* uniqueInstance;
    int data;
    Singleton(int d = 0):data(d) { }
    Singleton & operator=(Singleton & ob){
        if (this != &ob) data = ob.data; return *this; }
    Singleton(const Singleton & ob) { data = ob.data; }
};

Singleton* Singleton::uniqueInstance = NULL;

int main()
{
    Singleton* s1 = Singleton::instance();
    cout << s1->getValue() << endl;
    Singleton* s2 = Singleton::instance();
    s2->setValue(9);
    cout << s1->getValue() <<endl;
    return 0;
}
```



Sabloane de proiectare (Design patterns)

Singleton (clase cu o singura instanta) - exemplu

```
#include <iostream>

using namespace std;

class Ceas_intern
{
    static Ceas_intern* instanta;
    int timestamp;

    Ceas_intern(int d = 0):timestamp(d) { }
    Ceas_intern & operator=(Ceas_intern & ob);
    Ceas_intern(const Ceas_intern & ob);

public:
    static Ceas_intern* get_instanta()
    {
        if (instanta == NULL) instanta = new Ceas_intern();
        return instanta;
    }
    void adauga_zile(int);
    void adauga_luni(int);
    int get_timestamp();
};

Ceas_intern* Ceas_intern::instanta = NULL;
```



Sabloane de proiectare (Design patterns)

Singleton (clase cu o singura instanta) - exemplu

```
Ceas_intern & Ceas_intern::operator=(Ceas_intern & ob)
{
    if (this != &ob)
        timestamp = ob.timestamp;
    return *this;
}

Ceas_intern::Ceas_intern(const Ceas_intern & ob)
{
    timestamp = ob.timestamp;
}

void Ceas_intern::adauga_zile(int d)
{
    timestamp+=d;
}

void Ceas_intern::adauga_luni(int m)
{
    timestamp+=22*m;
}

int Ceas_intern::get_timestamp()
{
    return timestamp;
}

int main()
{
    Ceas_intern* ob1 = Ceas_intern::get_instanta();
    cout<<ob1->get_timestamp()<<endl;
    ob1->adauga_zile(10);
    cout<<ob1->get_timestamp()<<endl;
    Ceas_intern* ob2 = Ceas_intern::get_instanta();
    cout<<ob2->get_timestamp()<<endl;
    ob2->adauga_zile(10);
    cout<<ob2->get_timestamp()<<endl;
    cout<<ob1->get_timestamp()<<endl;
}
```

0
10
10
20
20



Sabloane de proiectare (Design patterns)

4. Observer

Intentia: Defineste o dependenta “unul la mai multi” intre obiecte, astfel incat atunci cand unul dintre obiecte isi schimba starea toate obiectele dependente sunt notificate si actualizate automat.

Alte nume prin care este cunoscut: Dependents, Publish-Subscribe.

Motivatia descrie cum sa se stabileasca relatiile intre clase.

Obiectele cheie in acest sablon sunt **subiect** si **observator**.

Un subiect poate avea orice numar de observatori dependenti.

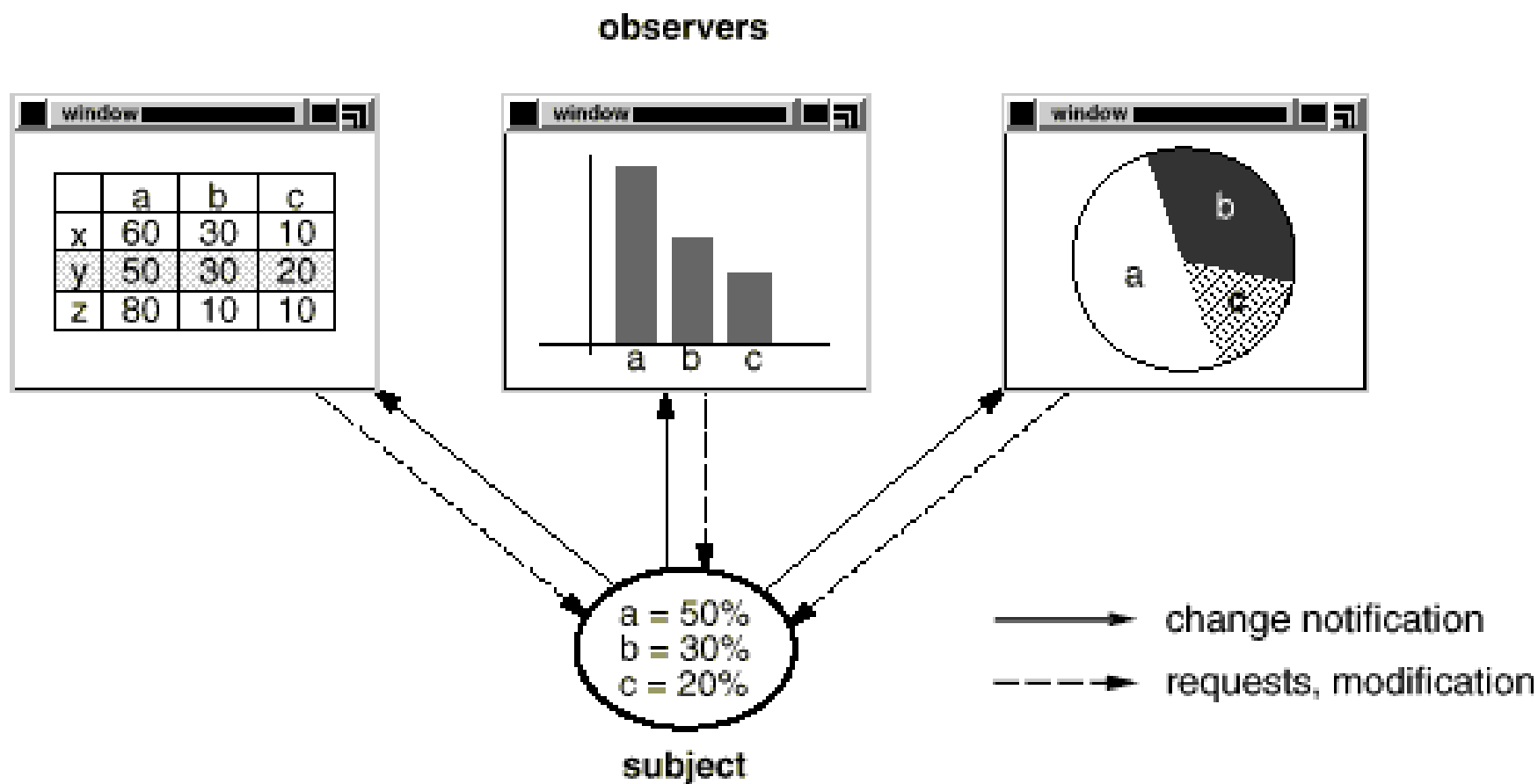
Toti observatorii sunt notificati ori de cate ori subiectul isi schimba starea.

Ca raspuns la notificare, fiecare observator va interoga subiectul pentru a-si sincroniza starea cu starea subiectului.



Sabloane de proiectare (Design patterns)

Observer

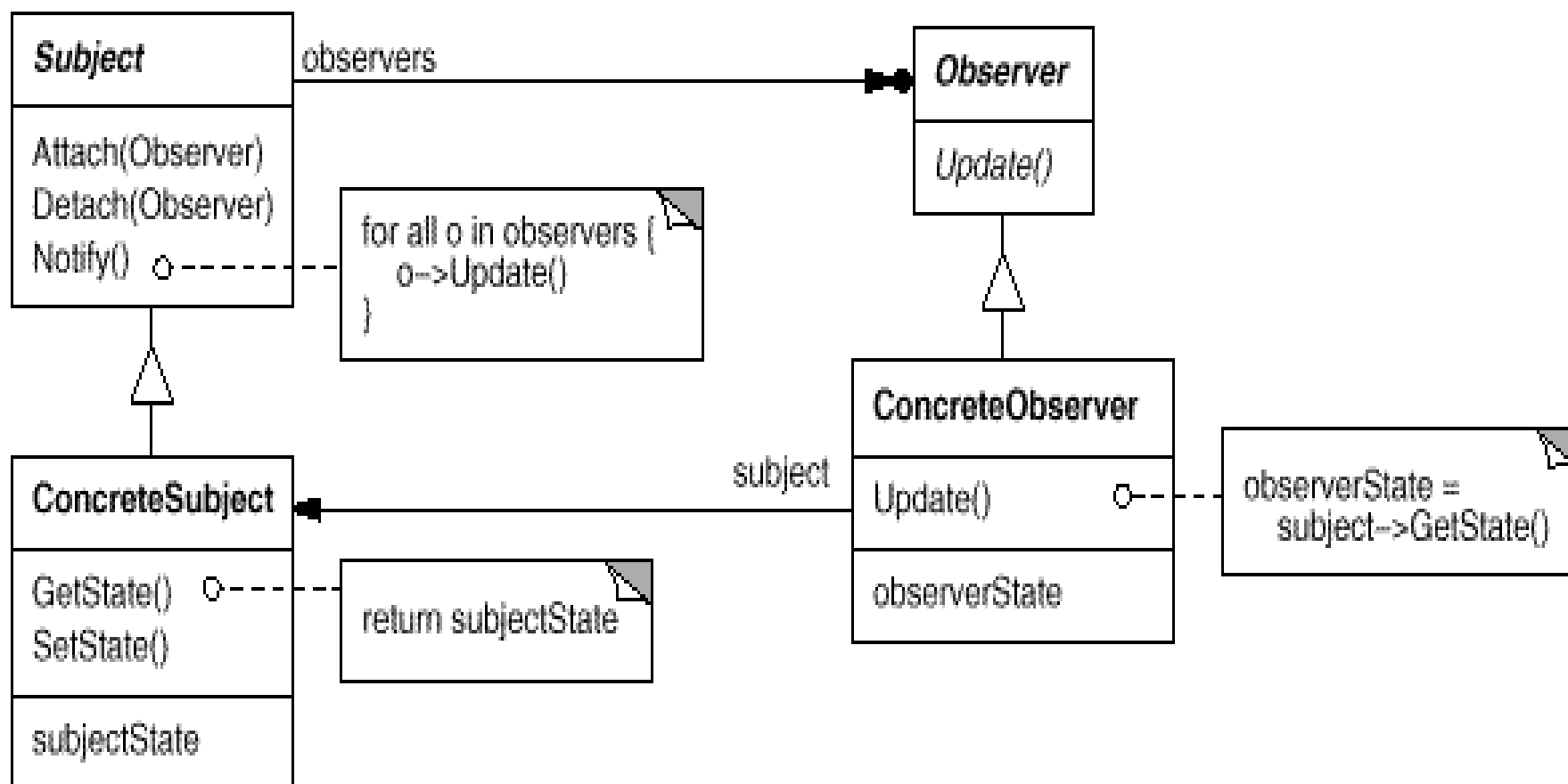




Sabloane de proiectare (Design patterns)

Observer

Structura





Sabloane de proiectare (Design patterns)

Observer

Aplicabilitatea

Sablonul poate fi utilizat in oricare dintre urmatoarele situatii:

- cand o abstractie are doua aspecte, unul dependent de celalalt (daca sunt incapsulate in obiecte separate, ele pot fi reutilizate independent).
- cand modificarea unui obiect necesita modificarea altor obiecte si nu se stie cate obiecte trebuie sa fie modificate.
- cand un obiect trebuie sa notifice alte obiecte fara a face presupuneri despre cine sunt aceste obiecte.



Sabloane de proiectare (Design patterns)

Observer

Participantii

Subject

Cunoaste observatorii sai.

Un obiect **Subject** poate fi observat de orice numar de obiecte **Observer**

Furnizeaza o interfata pentru atasarea si detasarea obiectelor **Observer**.

Observer

Defineste o interfata pentru actualizarea obiectelor care trebuie sa fie notificate (anuntate) despre modificarile din subiect.



Sabloane de proiectare (Design patterns)

Observer

Participantii

ConcreteSubject

Memoreaza starea de interes pentru obiectele ConcreteObserver.

Trimite o notificare observatorilor sai atunci cand i se schimba starea.

ConcreteObserver

Mentine o referinta la un obiect ConcreteSubject.

Memoreaza starea care trebuie sa ramana consistenta cu a subiectului.

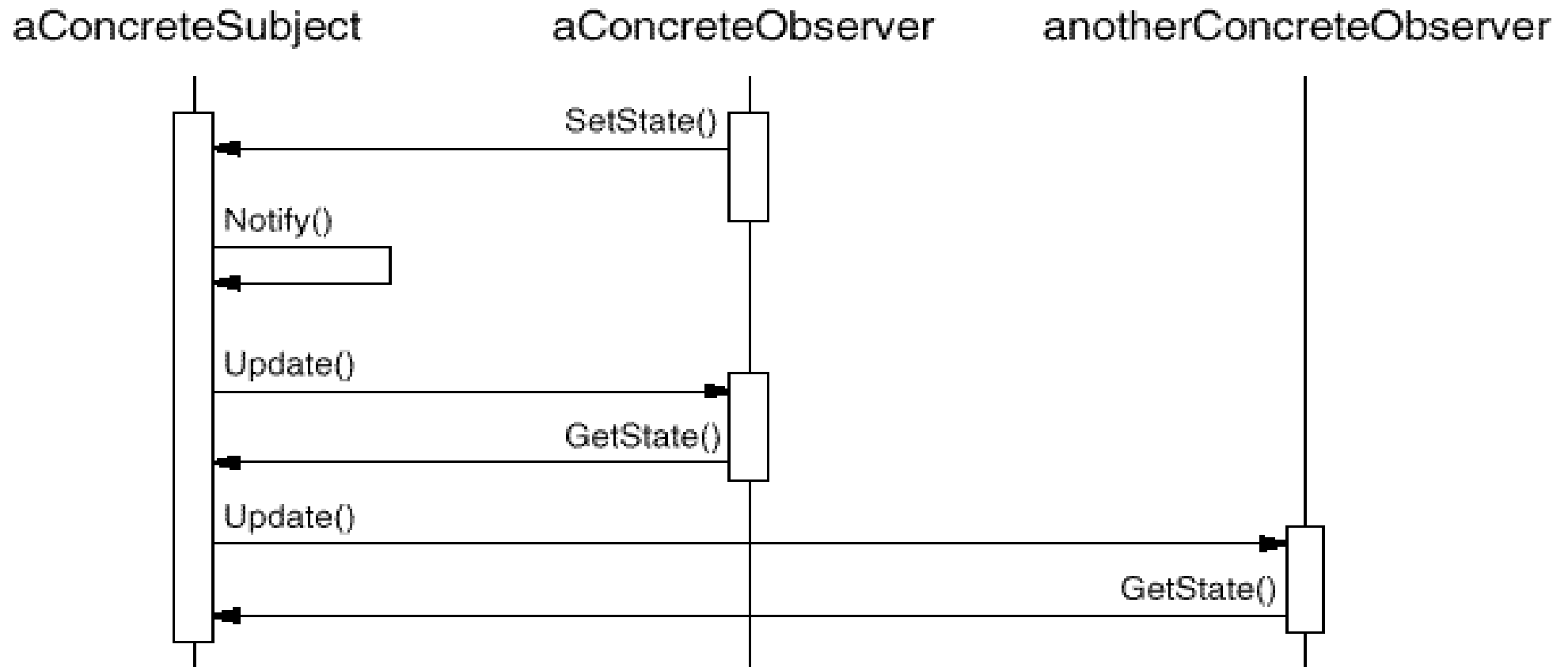
Implementeaza interfata de actualizare a clasei Observer



Sabloane de proiectare (Design patterns)

Observer

Colaborari





Sabloane de proiectare (Design patterns)

Observer

Exemplu de cod

Interfata **Observer** este definita printr-o clasa abstracta:

```
class Observer {  
public:  
    virtual ~ Observer();  
    virtual void Update (Subject* theChangedSubject) = 0;  
protected:  
    Observer();  
};
```



Sabloane de proiectare (Design patterns)

Observer

Exemplu de cod

Interfata **Subject** este definita prin urmatoarea clasa:

```
class Subject {
public:
    virtual ~Subject();
    virtual void Attach(Observer*);
    virtual void Detach(Observer*);
    virtual void Notify();
protected:
    Subject();
private:
    List<Observer*> *_observers;
};

void Subject::Attach (Observer* o) {_observers->Append(o); }
void Subject::Detach (Observer* o) {_observers->Remove(o); }
void Subject::Notify () {
    ListIterator<Observer*> i(_observers);
    //construieste un iterator, i, pentru containerul _observers
    for (i.First(); !i.IsDone(); i.Next()) { i.CurrentItem()->Update(this); }
}
```



Sabloane de proiectare (Design patterns)

Observer

un subiect concret

```
class ClockTimer : public Subject {
public:
    ClockTimer();
    virtual int GetHour();
    virtual int GetMinute();
    virtual int GetSecond();
    void Tick();
};

void ClockTimer::Tick () {
    // update internal time-keeping state
    // ...
    Notify();
}
```



Sabloane de proiectare (Design patterns)

Observer

un observator concret care mosteneste in plus o interfata grafica

```
class DigitalClock: public Widget, public Observer
{
public:
    DigitalClock(ClockTimer*);
    virtual ~DigitalClock();
    virtual void Update(Subject*);
        // overrides Observer operation
    virtual void Draw();
        // overrides Widget operation;
        // defines how to draw the digital clock
private:
    ClockTimer* _subject;
};
```




Sabloane de proiectare (Design patterns)

Observer

un observator concret care mosteneste in plus o interfata grafica

```
DigitalClock::DigitalClock (ClockTimer* s) {  
    _subject = s;  
    _subject->Attach(this); }  
  
DigitalClock::~~DigitalClock () {_subject->Detach(this);}  
  
void DigitalClock::Update (Subject* theChangedSubject) {  
    if (theChangedSubject == _subject) { Draw(); } }  
  
void DigitalClock::Draw () { // get the new values from the subject  
    int hour = _subject->GetHour();  
    int minute = _subject->GetMinute();  
    // draw the digital clock  
}
```



Sabloane de proiectare (Design patterns)

Observer

un alt observator

```
class AnalogClock : public Widget, public Observer {
public:
    AnalogClock(ClockTimer*);
    virtual void Update(Subject*);
    virtual void Draw();
    // ...
};

/* crearea unui AnalogClock si unui DigitalClock care arata
acelasi timp: */

ClockTimer* timer = new ClockTimer;
AnalogClock* analogClock = new AnalogClock(timer);
DigitalClock* digitalClock = new DigitalClock(timer);
```



Sabloane de proiectare (Design patterns)

5. Abstract Object Factory

intentie

- de a furniza o interfata pentru crearea unei familii de obiecte intercorelate sau dependente fara a specifica clasa lor concreta.

aplicabilitate

- un sistem ar trebui sa fie independent de modul in care sunt create produsele, compuse sau reprezentate
- un sistem ar urma sa fie configurat cu familii multiple de produse
- o familie de obiecte intercorelate este proiectata pentru astfel ca obiectele sa fie utilizate impreuna
- **se doreste furnizarea unei biblioteci de produse, dar se doreste accesibila numai interfata, nu si implementarea.**



Sabloane de proiectare (Design patterns)

Abstract Object Factory

colaborari

- normal se creeaza o singura instanta

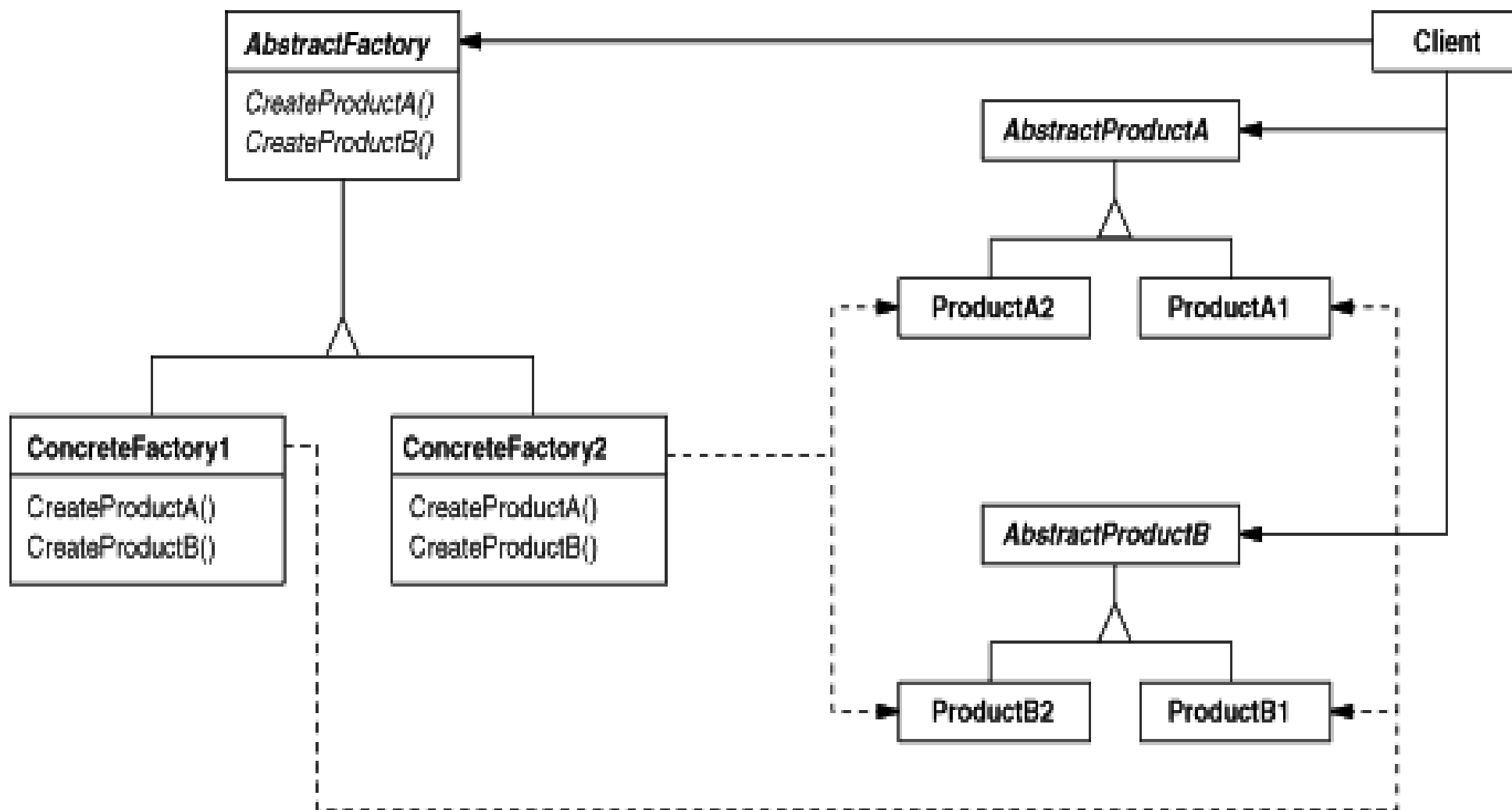
consecinte

- izoleaza clasele concrete
- simplifica schimbul familiei de produse
- promoveaza consistenta printre produse
- suporta noi timpul noi familii de produse usor
- respecta principiul deschis/inchis



Sabloane de proiectare (Design patterns)

Abstract Object Factory structura





Sabloane de proiectare (Design patterns)

Abstract Object Factory

implementare

o functie delegat (callback) este o functie care nu este invocata explicit de programator; responsabilitatea apelarii este delegata altei functii care primeste ca parametru adresa functiei delegat

Fabrica de obiecte utilizeaza functii delegat pentru crearea de obiecte: pentru fiecare tip este delegata functia carea creeaza obiecte de acel tip.



Sabloane de proiectare (Design patterns)

Abstract Object Factory

solutia

- definim mai intai clasa de baza ca si clasa abstracta

```
class Figura {  
    public:  
        virtual void afis() = 0;  
};
```

- Definim grupurile de produse / tipurile de produse



Sabloane de proiectare (Design patterns)

Abstract Object Factory

solutia

- Definim grupurile de produse / tipurile de produse

```
/****** Produse tip A *****/  
class Cerc : public Figura {  
public:  
    void afis() {  
        cout << "Cerc\n";  
    }  
};  
  
class Patrat : public Figura {  
public:  
    void afis() {  
        cout << "Patrat\n";  
    }  
};
```

```
/****** Produse tip B *****/  
class Elipsa : public Figura {  
public:  
    void afis() {  
        cout << "Elipsa\n";  
    }  
};  
  
class Dreptunghi : public Figura {  
public:  
    void afis() {  
        cout << "Dreptunghi\n";  
    }  
};
```




Sabloane de proiectare (Design patterns)

Abstract Object Factory

solutia

definim apoi o fabrica de figuri, adica o clasa care sa gestioneze tipurile de figuri

```
class Factory {  
    public:  
        virtual Figura* creeaza_figuri_fara_colturi() = 0;  
        virtual Figura* creeaza_figuri_cu_colturi() = 0;  
};
```



Sabloane de proiectare (Design patterns)

Abstract Object Factory

solutia

Generam factory pentru grupuri de produse

```
class Figuri_simple : public Factory {
public:
    Figura* creeaza_figuri_fara_colaturi() {
        return new Cerc;
    }
    Figura* creeaza_figuri_cu_colaturi() {
        return new Patrat;
    }
};

class Figuri_robuste : public Factory {
public:
    Figura* creeaza_figuri_fara_colaturi() {
        return new Elipsa;
    }
    Figura* creeaza_figuri_cu_colaturi() {
        return new Dreptunghi;
    }
};
```



Sabloane de proiectare (Design patterns)

Abstract Object Factory

solutia

Apel fara a numi efectiv figurile

```
int main() {  
  
    Factory* factory = new Figuri_simple();  
    Figura* f[3];  
  
    f[0] = factory->creeaza_figuri_fara_colture();  
    f[1] = factory->creeaza_figuri_cu_colture();  
    f[2] = factory->creeaza_figuri_fara_colture();  
  
    for (int i=0; i < 3; i++) f[i]->afis();  
}
```



Sabloane de proiectare (Design patterns)

6. *Strategy pattern*

intentie

- Presupune incapsularea separata a fiecarui algoritm dintr-o familie, facand astfel ca algoritmii respectivi sa fie interschimbabili.

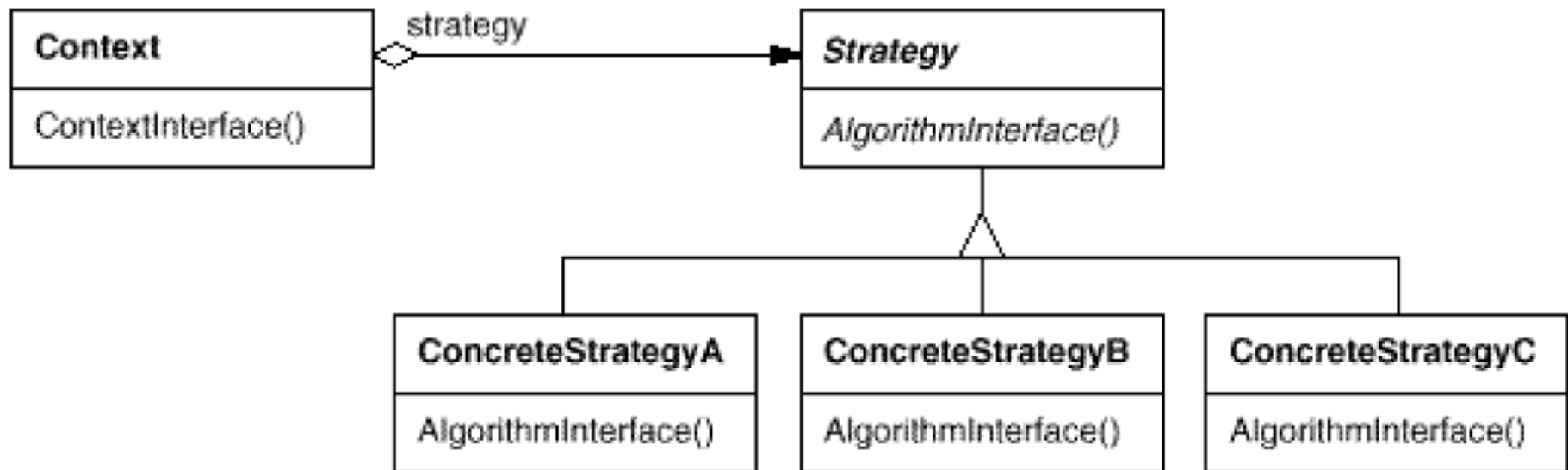
aplicabilitate

- mai multe clase inrudite difera doar prin comportament;
- sunt necesare mai multe variante ale unui algoritm, care difera intre ele, de exemplu, prin compromisul spatiu-timp adoptat;
- un algoritm utilizeaza date pe care clientul algoritmului nu trebuie sa le cunoasca;
- intr-o clasa sunt definite mai multe actiuni care apar ca structuri conditionale multiple. In loc de aceasta, se recomanda plasarea ramurilor conditionale inrudite in cate o clasa strategy separata.



Sabloane de proiectare (Design patterns)

Strategy pattern structura



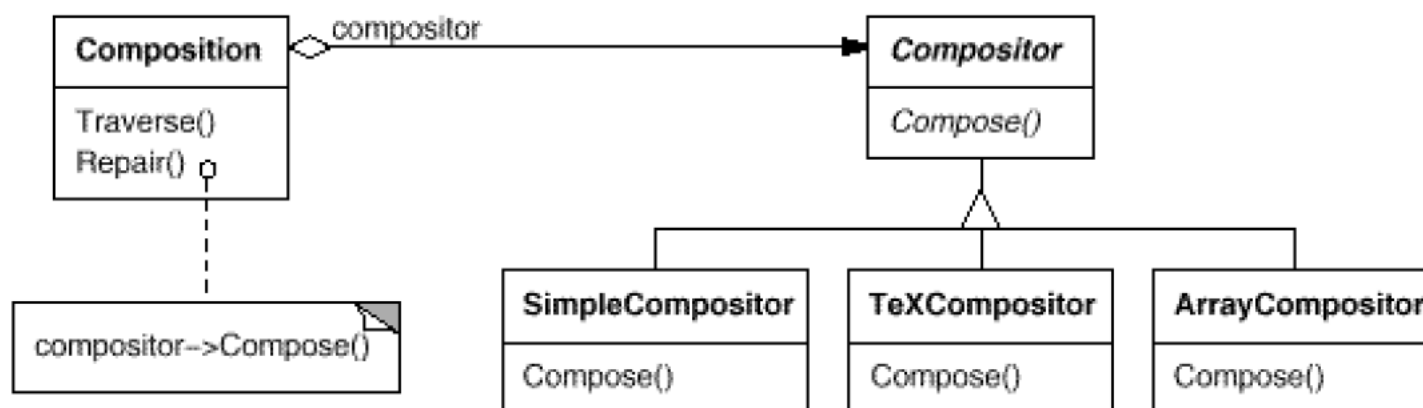


Sabloane de proiectare (Design patterns)

Strategy pattern exemplu

Algoritm care impart un text pe linii

- SimpleCompositor – algoritm simplu cu \n
- TeXCompositor – algoritm care grupeaza, mai eficient, pe paragrafe
- ArrayCompositor – algoritm care imparte textul astfel incat, pe fiecare linie, sa existe acelasi nr de caractere





Sabloane de proiectare (Design patterns)

Strategy pattern

Exemplu implementare

Problema!!

```
class Turist
{
    string nume;
public:
    void transport_cu_masina() {cout<<"masina\n";}
    void transport_cu_avion() {cout<<"avion\n";}
    void transport_cu_bicicleta() {cout<<"bicicleta\n";}
};
```



Sabloane de proiectare (Design patterns)

Strategy pattern

Exemplu implementare

Strategiile de transport

```
class I_Transport
{
public:
    virtual void transport() const = 0;
};

class Masina : public I_Transport
{
public:
    void transport() const {cout<<"masina\n";}
};

class Avion : public I_Transport
{
public:
    void transport() const {cout<<"avion\n";}
};

class Bicicleta : public I_Transport
{
public:
    void transport() const {cout<<"bicicleta\n";}
};
```




Sabloane de proiectare (Design patterns)

Strategy pattern

Exemplu implementare

Noua clasa turist (care contine un pointer catre modul de deplasare)

```
class Turist
{
    string nume;
    I_Transport* modTransport;
public:
    Turist (string n, I_Transport* modInitial) {nume = n; modTransport = modInitial;}

    void setModTransport(I_Transport* modNou)
    {
        if (modTransport != NULL) delete modTransport;
        modTransport = modNou;
    }
    void deplasare()
    {
        if (modTransport == NULL) throw "Nu ai selectat un mijloc de transport";
        modTransport->transport(); /// nu se mai leaga de implementare ca inainte
    }
    virtual ~Turist() {delete modTransport;}
};
```



Sabloane de proiectare (Design patterns)

Strategy pattern

Exemplu implementare

Apel

```
int main()  
{  
    Turist t("Popescu", new Avion());  
    t.deplasare();  
  
    t.setModTransport(new Masina());  
    t.deplasare();  
  
    t.setModTransport(new Bicicleta());  
    t.deplasare();  
}
```



Curs 13

Succes la colocviu si la examenul scris!